

PROMPT ENGINEERING

FROM PROMPT TO PROJECTS

N.S.Sai karun, Shamim , Dr.K.V.N.R Sai Krishna

Prompt Engineering

Copyright © 2024 by N.S.Sai karun, Shamim , Dr.K.V.N.R Sai Krishna

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means without written permission from the author.

ISBN: 978-81-980666-0-2

Printed in the India by KBN College

DEDICATION

Dedication

This book is dedicated to the curious minds, creative thinkers, and passionate innovators who are exploring the endless possibilities of AI and prompt engineering.

To all those who believe in the power of effective communication between humans and machines, may you find inspiration and endless opportunities within these pages.

To my family, friends, and mentors—your unwavering support has been my foundation.

With gratitude,

N.S. Sai Karun

Asst.Professor

Dept.of Computer Science, KBN College

TABLE OF CONTENTS

Foreword	10
Preface	11
Introduction	12
Key Terminology	13
What is Prompt?	17
Understanding Prompt Engineering	19
How Prompts Influence AI Responses	19
Introduction to Different Types of Prompts	19
Traditional Engineering vs. Prompt Engineering	21
The Basics of AI and Language Models	23
Explanation of AI Language Models and Their Architecture	23
The Role of Training Data in Shaping Model Responses	24
Examples of Popular Language Models	25
Advancements in AI Models: A Comprehensive Overview	26
Text-to-Image Models	26
Text-to-3D Models	27
Text-to-Video Models	27
Audio Processing Models	27
Conversational and Text Generation Models	28
Text-to-Text Models	28
Code Generation Models	28
Multimodal and Cross-Disciplinary Models	28
Other Innovative Models	29
Analysis of Technologies Behind the Models	29
Elements of a Prompt	30
1. Instruction	30
2. Context	30
3. Input Data	31
4. Output Indicator	31
Comparison of Prompt Elements	31
Analysis of Element Interactions	32
Advanced Prompting Techniques	35
1. Zero-shot Prompting	35
2. Few-shot Prompting	35

3. Chain-of-Thought Prompting	35
4. Meta Prompting	35
5. Self-Consistency	36
6. Generate Knowledge Prompting	36
7. Prompt Chaining	36
8. Tree of Thoughts	36
9. Retrieval Augmented Generation	36
10. Automatic Reasoning and Tool-use	37
11. Automatic Prompt Engineer	37
12. Active-Prompt	37
13. Directional Stimulus Prompting	37
14. Program-Aided Language Models	37
15. ReAct	38
Zero-Shot Prompting	39
Applications of Few-Shot Prompting	40
Zero-Shot Prompting: Pros and Cons	41
Few-Shot Prompting	43
Foundations of Few-Shot Prompting	43
Example of Few-Shot Prompting	43
Enhancing Few-Shot Prompting	44
Limitations of Few-Shot Prompting	44
Attempting to Improve Responses	44
Applications of Few-Shot Prompting	45
Pros of Few-Shot Prompting	46
Cons of Few-Shot Prompting	46
Chain-of-Thought (CoT) Prompting	47
Meta Prompting	49
Getting Started with the OpenAI Playground	51
Using ChatGPT As Your	56
Python Pair-Programmer	56
Option #1: Setting a System-Wide Environment Variable	56
For Mac OS	57
For Windows	57
For Linux	57
Option #2: Creating a .env File	58
Option #3: Hard-Coding the API Key Directly in Your Application (Take with Caution)	59
Setting Up the Prompt	63

import requests	63
Creating accuweather_forecaster.py with the AccuWeather API	64
Refining Our Prompt with Prompt Engineering	65
Creating a Project with Google Maps Platform API	71
Using ChatGPT to Take a cURL Command and Make It Useable in Python: The Flexible Programmer's Approach	72
Creating a Basic ChatGPT	77
Client in Python	77
Examining the Method Parameters	78
System Message (Dictionary)	80
User Message (Dictionary)	80
Assistant Message (Dictionary)	81
Tool Message (Dictionary)	82
Handling the Response (ChatCompletion)	84
ChatCompletionMessage	85
Using AI in the Enterprise! Creating a Text Summarizer for Slack Messages	87
Prompt #1: "tl;dr"	90
Prompt #2: "Explain This in Three Sentences or Less"	91
Prompt #3: "I'm a Manager. Explain to Me What Happened"	92
Prompt #4: "Give Me Suggestions on Next Steps"	93
Let's Talk About Real Prompt Engineering	94
Specifying What Your Bot Can (and Can't) Do by Setting the Scope	96
Confirming Your Settings	96
Viewing the OAuth & Permissions Page	96
Installing Your Slack Bot App to Your Workspace	96
Getting Your Slack Bot (Access) Token	97
Inviting Your Bot to Your Channel	97
Programmatically Reading Messages from Slack	97
Multimodal AI:	101
Creating a Podcast Visualizer with Whisper and DALL·E 3	101
Examining the Method Parameters	110
DALL·E Golden Rule #1: Get Familiar with the Types of Images That DALL·E Can Generate	110
DALL·E Golden Rule #2: Be Descriptive with What You Want in the Foreground and Background	111
Creating an Automated Community Manager Bot with Discord and	

Python	113
Understanding the Roles for the Bots	114
Handling Messages Sent to the Discord Server	119
Success! Running Your First Discord Bot: tech_support_bot_dumb.py	120
Registering a New Discord Bot App with Discord	120
Specifying General Info for the Bot	120
Specifying OAuth2 Parameters for the Bot	120
Getting the Discord ID Token for Your Bot and Setting the Gateway Intents	120
Handling Messages Sent to the Discord Server	122
Success Again! Running Your Second Discord Bot: content_moderator_bot_dumb.py	122
Adding Intelligence to Our Discord Bots,	123
Part 1: Using the Chat Model for Q&A	123
Updates to the on_message(message) Function	127
Running Our Intelligent Q&A Bot: tech_support_bot.py	130
Adding Intelligence to Our Discord Bots, Part 2: Using Chat and 134	
Moderations Models for Content Moderation	134
Prerequisites	143
1. Understanding the OpenSkills API	143
Key Endpoint	143
Example Request	143
Example Response	144
2. Setting Up the Application	144
Step 1: Import Necessary Libraries	144
Step 2: Create Function to Fetch Skills	144
Explanation	145
Step 3: Display Skills in a Readable Format	145
Explanation	146
Step 4: Putting It All Together	146
Explanation	146
3. Running the Application	146
4. Error Handling and Edge Cases	147
Handling Empty or Invalid Data	147
Handling Invalid Job Titles	147
5. Conclusion	147
APPENDIX 1	148

List of OpenAI Models	148
APPENDIX II	149
List of Open Source AI Tools	149
APPENDIX III	153
List of Popular No Code Development Tools	153
APPENDIX IV	156
List of Open Source LLMs	156

FOREWORD

In today's world of artificial intelligence, the ability to craft the right prompt is more important than ever. It is through effective prompts that we can communicate our ideas to machines and guide them to generate the outcomes we seek. In *From Prompt to Projects*, I aim to walk you through the essential skills and techniques of prompt engineering, demonstrating how the journey from an initial idea to a successful project can be both fascinating and rewarding.

Prompt engineering is not just about feeding instructions to an AI; it's about creating a meaningful interaction that enables machines to transform raw data into valuable insights, solutions, and even creative works. This book delves into the principles behind this process and offers a practical approach that will help you turn your ideas into real-world applications.

Throughout these pages, I have included everything from basic concepts to advanced strategies, ensuring that both beginners and more experienced professionals can grasp the core techniques and begin applying them to their own projects. Whether you are building an AI application, developing a business idea, or exploring new ways to use machine learning, this book will equip you with the knowledge to get the results you want.

I believe this book is more than just a guide to technical skills—it's an invitation to think creatively, to explore new possibilities, and to push the boundaries of what artificial intelligence can do. I hope that *From Prompt to Projects* will serve as both a resource and an inspiration for anyone looking to create and innovate in the field of AI.

N.S. Sai Karun

Asst. Professor, KBN College

PREFACE

The world of artificial intelligence has undergone rapid and transformative growth over the last few years, and with it, the field of prompt engineering has emerged as one of the most essential skills for leveraging AI's capabilities. As AI systems become more advanced and accessible, the need to effectively communicate with these systems has become critical. A well-crafted prompt is not just a command but the key to unlocking the vast potential of AI.

In *From Prompt to Projects*, my aim is to guide you through the fundamental concepts of prompt engineering and demonstrate how these concepts can be used to bring your ideas to life through AI. Whether you are a student, developer, entrepreneur, or enthusiast, this book will provide you with the tools and insights necessary to transform simple prompts into powerful projects.

The journey from prompt to project is an exciting one, as it combines creativity with technical knowledge. With this book, I hope to empower you to not only understand the process of prompt engineering but also to inspire you to think beyond the basics and apply these techniques to solve real-world problems.

Each chapter has been designed to provide clear explanations, practical examples, and hands-on exercises that will help you build a strong foundation in prompt engineering. While this book is suitable for beginners, I have also included advanced techniques to ensure that even those with prior experience can deepen their understanding and expand their skills.

As we move forward in a world increasingly shaped by AI, the ability to harness its power through well-constructed prompts will become an invaluable skill. I encourage you to approach this book with curiosity and creativity, and I hope it serves as a useful resource on your journey to mastering the art of prompt engineering.

Thank you for choosing *From Prompt to Projects*. I look forward to accompanying you on this exciting journey of discovery and innovation.

N.S. Sai Karun

Asst. Professor, KBN College

INTRODUCTION

Prompt engineering is an emerging discipline focused on the art and science of crafting prompts to optimize interactions with large language models (LLMs). As these models become increasingly complex and capable, understanding how to communicate effectively with them is essential for researchers, developers, and users alike. This discipline encompasses various techniques designed to improve the performance, reliability, and safety of LLMs across a broad range of applications, from simple question answering to complex reasoning tasks.

To engage in effective prompt engineering, it is crucial to grasp the foundational capabilities and limitations of LLMs. These models excel in natural language understanding and generation, allowing them to interpret and produce human-like text based on input. They are particularly adept at tasks like summarization, translation, and creative writing. However, their contextual awareness has its limits; while they can maintain coherence over short conversations, their understanding diminishes with longer or more complex exchanges. Additionally, while LLMs can provide accurate information, they also have a tendency to generate false or misleading content, underscoring the importance of critical evaluation. Finally, although LLMs are versatile, they may require specific prompt structures to perform optimally in specialized tasks such as coding or mathematical reasoning.

The significance of prompt engineering cannot be overstated. By crafting effective prompts, users can extract the best possible performance from LLMs, ensuring efficiency across diverse tasks. Moreover, thoughtful prompt design can enhance safety by mitigating the risks of generating undesirable outputs, such as biased or harmful content. Finally, well-engineered prompts lead to clearer and more accurate responses, greatly improving the overall user experience.

In summary, prompt engineering is a vital discipline that enhances interactions between users and large language models. By understanding LLM capabilities and applying effective prompt engineering techniques, users can maximize the utility of these powerful tools while ensuring safety and reliability. As LLMs continue to evolve, the strategies for effectively harnessing their potential will also advance, making prompt engineering an essential skill for anyone working with AI technologies.

KEY TERMINOLOGY

Here is the list of Key terms used in this book .

1. **Prompt:** The input text provided to a model.
2. **Instruction:** A directive telling the model what to do.
3. **Context:** Background information that informs the model's response.
4. **Input:** Data or text included in the prompt.
5. **Example:** A sample provided to guide the model's output.
6. **Few-shot Learning:** Providing a few examples in the prompt.
7. **Zero-shot Learning:** Asking the model to perform a task without examples.
8. **Prompt Structure:** The organization of the prompt elements.
9. **Clarity:** The quality of being clear and unambiguous.
10. **Relevance:** How closely the prompt elements relate to the task.
11. **Iterative Refinement:** Adjusting prompts based on output feedback.
12. **Task Specification:** Clearly defining the task in the prompt.
13. **Anchoring:** Providing specific context to ground responses.
14. **Output:** The text generated by the model in response to the prompt.
15. **Bias Mitigation:** Techniques to reduce biased outputs.
16. **Fine-tuning:** Adjusting a model's parameters for specific tasks.
17. **Model Behavior:** How the model responds to different prompts.
18. **Response Quality:** The accuracy and relevance of the model's output.
19. **Template:** A pre-structured prompt used for consistent results.
20. **Input Formatting:** The way input data is presented to the model.
21. **Prompt Engineering:** The practice of crafting effective prompts.
22. **Token:** A piece of text, such as a word or punctuation, used in processing.
23. **Training Data:** The data used to train the language model.
24. **Inference:** The process of generating responses from a model.
25. **Pre-training:** Initial training phase on a large dataset.
26. **Prompt Sensitivity:** How the model's output changes with prompt variations.
27. **Model Capacity:** The ability of the model to learn from data.
28. **Interactivity:** The model's ability to engage in dialogue.
29. **Prompt Chaining:** Using multiple prompts in sequence for complex tasks.
30. **Response Length:** The number of tokens in the model's output.
31. **Context Window:** The amount of text the model can consider at once.
32. **Summarization:** Reducing text to its main points.
33. **Translation:** Converting text from one language to another.
34. **Sentiment Analysis:** Determining the emotional tone of text.
35. **Classification:** Categorizing text into predefined groups.
36. **NLP (Natural Language Processing):** The field focused on interactions between computers and human language.
37. **Sparsity:** The presence of many zero values in data, impacting model performance.

38. **Generalization:** The model's ability to apply learned knowledge to new data.
39. **Prompt Variability:** The diversity of prompts used to elicit different responses.
40. **Output Diversity:** The range of different responses generated for a given prompt.
41. **Feedback Loop:** Using output results to inform future prompts.
42. **Error Analysis:** Examining incorrect outputs to improve prompting.
43. **Task Complexity:** The difficulty level of the task being prompted.
44. **Semantic Understanding:** The model's grasp of meaning in language.
45. **Pragmatic Understanding:** The model's ability to interpret language in context.
46. **Engagement:** The model's capacity to maintain an interactive dialogue.
47. **Data Augmentation:** Techniques to expand training datasets for better learning.
48. **Multimodal Input:** Combining different types of data, like text and images, in prompts.
49. **Hierarchical Prompting:** Organizing prompts in layers or levels of complexity.
50. **Predefined Templates:** Established formats for prompts to ensure consistency.
51. **Prompt Optimization:** The process of refining prompts for better results.
52. **Adaptive Learning:** The model's ability to adjust based on new information.
53. **Prompt Contextualization:** Providing context that influences the model's output.
54. **Language Model Architecture:** The underlying structure of the model (e.g., transformer).
55. **Model Tuning:** Adjusting parameters post-training for specific tasks.
56. **Prompt Analysis:** Evaluating the effectiveness of prompts used.
57. **Dialogue Management:** Techniques for managing interactions in conversation.
58. **Error Correction:** Adjusting prompts based on observed mistakes.
59. **Multi-turn Dialogue:** Engaging the model in a back-and-forth conversation.
60. **Task-Specific Training:** Training the model on specific types of tasks.
61. **Reinforcement Learning:** A learning paradigm where models are rewarded for good outputs.
62. **Predictive Text:** The model's ability to generate text based on patterns.
63. **Prompt Refinement:** Continually improving prompts for effectiveness.
64. **Attention Mechanism:** A component that allows the model to focus on specific input parts.
65. **Token Limit:** The maximum number of tokens the model can process in one go.

66. **Batch Processing:** Handling multiple prompts simultaneously for efficiency.
67. **Data Bias:** The influence of biased training data on model outputs.
68. **Interactive Feedback:** Real-time responses that guide prompt adjustments.
69. **Content Generation:** The creation of new text based on prompts.
70. **Query Expansion:** Enhancing prompts to elicit more detailed responses.
71. **Temporal Context:** Information relating to time that influences responses.
72. **Prompt Length:** The number of tokens or words in the prompt itself.
73. **Cohesion:** The logical flow and clarity of the model's response.
74. **Prompt Normalization:** Standardizing prompts to reduce variability.
75. **Model Interpretability:** Understanding how and why a model produces certain outputs.
76. **Knowledge Injection:** Integrating specific knowledge into the model's responses.
77. **Feedback Mechanism:** A system for providing insights on model performance.
78. **Contextual Bias:** The influence of context on model interpretation.
79. **Response Evaluation:** Assessing the quality of the model's output.
80. **Prompting Framework:** A structured approach to designing prompts.
81. **Task Automation:** Using prompts to automate repetitive tasks.
82. **Human-in-the-loop:** Involving human oversight in the prompting process.
83. **Error Rate:** The frequency of incorrect outputs from the model.
84. **Data Annotation:** Labeling data for training and evaluation.
85. **Probabilistic Output:** The model's response based on likelihood rather than certainty.
86. **Narrow AI:** AI focused on specific tasks rather than general intelligence.
87. **User Intent:** The goal or purpose behind a user's prompt.
88. **Knowledge Base:** A collection of information the model can reference.
89. **Semantic Similarity:** The degree to which different prompts convey the same meaning.
90. **Adaptive Prompting:** Adjusting prompts based on previous interactions.
91. **Cognitive Load:** The mental effort required to understand prompts and responses.
92. **Output Validation:** Checking the accuracy and relevance of generated responses.
93. **Data Privacy:** Ensuring user information is protected in prompts.
94. **Personalization:** Tailoring prompts and responses to individual users.
95. **Scalability:** The ability to maintain performance with increasing input size.
96. **Prompt Inversion:** Using outputs to inform and refine input prompts.
97. **Dynamic Prompting:** Adjusting prompts in real-time based on interactions.
98. **Model Robustness:** The model's ability to perform well under varied conditions.

99. **Knowledge Distillation:** Simplifying a complex model while retaining its performance.
100. **LLM (Large Language Model):** A type of AI model designed to understand and generate human-like text based on large datasets.

WHAT IS PROMPT?

In the realm of artificial intelligence, particularly with large language models (LLMs), the concept of a "prompt" is foundational. But what exactly constitutes a prompt, and why is it so pivotal in the interaction between humans and machines? A prompt serves as a communication bridge, enabling users to direct the model's attention and guide its responses. This chapter delves into the essence of prompts, exploring their structure, purpose, and the nuances that make them effective.

At its core, a prompt is any input text that is given to a language model to elicit a response. This input can range from a simple question, like "What is the capital of France?" to more complex instructions, such as "Write a short story about a dragon who learns to fly." The versatility of prompts allows users to harness the capabilities of LLMs across a myriad of applications, from casual inquiries to intricate tasks requiring detailed responses.

The structure of a prompt can vary significantly depending on the desired outcome. A well-crafted prompt typically includes several key elements: clarity, context, and specificity. Clarity is crucial; the more unambiguous the prompt, the easier it is for the model to understand what is being asked. For instance, instead of asking, "Tell me about dogs," which is vague, a more precise prompt like "List three common dog breeds and their characteristics" provides a clearer directive, guiding the model to generate a more focused response.

Context plays an equally important role in shaping the model's output. By providing relevant background information, users can enhance the model's understanding of the subject matter. For example, if a user wants to know about the effects of climate change, supplying context such as "In the context of rising sea levels" can significantly narrow the focus of the model's response. This additional information helps the model draw upon its training data in a manner that aligns with the user's intent, resulting in more relevant and accurate outputs.

Specificity is another critical aspect of prompt design. The more detailed a prompt, the better the model can tailor its response. This is particularly evident in tasks that require the model to adopt a specific style or format. For example, if a user wants a summary of a research paper, including the phrase "in bullet points" can help the model present the information in a clear and organized manner. Conversely, a prompt lacking specificity may lead to broad or generic answers, underscoring the importance of thoughtful prompt construction.

The power of prompting is further amplified by its adaptability. Different contexts may require different approaches to prompting. For instance, a conversational prompt, like "What are the main challenges in renewable energy?" may necessitate a different structure than an instructional prompt, such as "Explain the process of photosynthesis in simple terms."

Recognizing the type of interaction desired can significantly influence how prompts are formulated.

Moreover, prompts can be enriched with examples to facilitate better responses. This technique, known as few-shot learning, involves providing one or two examples within the prompt to illustrate the desired format or style. For instance, if a user asks the model to generate a poem, including a short example can guide the model to produce a similar piece, enhancing both creativity and relevance.

In contrast, zero-shot learning is a technique where users provide a prompt without any examples, relying solely on the model's training and context to generate the response. This method can be effective, but it requires that the prompt be exceptionally clear and well-defined to achieve optimal results. The choice between few-shot and zero-shot prompting often depends on the user's familiarity with the model's capabilities and the complexity of the task at hand.

While crafting prompts, it is essential to consider potential pitfalls. Vague or ambiguous prompts can lead to unexpected or irrelevant outputs. For instance, asking "Explain gravity" might yield a broad response that doesn't address the specific aspect the user is interested in. Therefore, users must refine their prompts through iterative testing, adjusting the wording and structure based on the outputs generated. This process of trial and error is a critical component of effective prompt engineering.

Safety is another crucial consideration in prompt design. LLMs can inadvertently produce biased or harmful content if prompts are not carefully crafted. Users should be mindful of the language they use and the potential implications of their prompts. Techniques for bias mitigation, such as explicitly framing sensitive topics or avoiding loaded language, can help in generating responses that are both responsible and accurate.

As users become more skilled at crafting prompts, they will find that the interaction with LLMs becomes increasingly rewarding. A well-structured prompt not only enhances the model's performance but also fosters a more engaging and productive dialogue. Understanding the intricacies of prompting empowers users to tap into the full potential of LLMs, unlocking new possibilities for creativity, problem-solving, and knowledge acquisition.

In conclusion, a prompt is more than just a string of text; it is a powerful tool that shapes the interaction between humans and language models. By carefully considering clarity, context, and specificity, users can craft prompts that lead to meaningful and relevant responses. As the field of AI continues to evolve, the art of prompting will remain a vital skill, enabling users to harness the capabilities of large language models effectively. Through thoughtful design and an understanding of the model's behavior, users can navigate this landscape with confidence, unlocking the true potential of artificial intelligence.

UNDERSTANDING PROMPT ENGINEERING

Prompt engineering is a critical area of inquiry within the field of artificial intelligence (AI), particularly in the context of natural language processing (NLP). It involves the systematic design, refinement, and optimization of input prompts provided to AI models, with the objective of eliciting responses that are not only accurate but also contextually relevant and coherent. The significance of prompt engineering is underscored by the fact that the outputs of AI systems are highly contingent upon the specificity and structure of the prompts they receive. In practical applications ranging from conversational agents to automated content generation, the efficacy of AI is significantly enhanced through thoughtful prompt engineering, which facilitates improved human-AI interaction and maximizes the utility of AI technologies in various domains.

How Prompts Influence AI Responses

The interaction between prompts and AI responses can be understood through several key dimensions that govern how information is processed and conveyed by AI models.

1. **Clarity and Specificity:** The clarity and specificity of a prompt are paramount in determining the quality of the response. Prompts that are ambiguous or overly broad often lead to vague, nonspecific answers. Conversely, well-defined prompts tend to elicit more precise and relevant information. This relationship underscores the importance of language precision in formulating prompts.
2. **Contextual Cues:** Contextual information provided within prompts significantly aids the AI's interpretative capabilities. By embedding relevant background information or situational context, prompts can guide the AI to produce responses that are more aligned with user intentions. For instance, incorporating thematic or situational elements into prompts enhances the model's ability to generate context-aware outputs.
3. **Format and Structure:** The structural composition of a prompt plays a crucial role in shaping the AI's response. Different prompt formats—such as direct inquiries, open-ended requests, or structured tasks—can lead to varied response styles. For example, direct questions typically elicit concise factual answers, while open-ended instructions may encourage elaborate and creative responses. This variability highlights the necessity of considering format as a strategic element in prompt engineering.

Introduction to Different Types of Prompts

The classification of prompts into distinct categories allows for a more nuanced understanding of their effects on AI responses. The primary types include:

1. **Questions:** Prompts framed as questions are inherently designed to seek specific information or clarification. For instance, a prompt such as "What are the benefits of regular exercise?" is structured to elicit a factual, informative response. The nature of the inquiry directs the AI to retrieve relevant knowledge and synthesize it into a coherent answer.
2. **Instructions:** Instructional prompts are directive in nature, commanding the AI to execute a particular task or generate content according to specified criteria. An example of this is the prompt, "Write a poem about autumn," which not only defines the topic but also implicitly suggests the desired creative form. The clarity of the instruction aids the AI in aligning its output with user expectations.
3. **Examples:** Providing illustrative examples within prompts serves to clarify the desired output format and content. For instance, the prompt "Generate a recipe for a dish similar to this one: [insert example]" effectively sets a clear benchmark for the AI. By modeling the expected response through examples, users can significantly enhance the AI's understanding of contextual requirements and stylistic preferences.

Traditional Engineering vs. Prompt Engineering

Traditional Engineering and **Prompt Engineering** represent two distinct domains of practice and application, each characterized by unique methodologies, objectives, and tools. Below, we explore the fundamental differences between these two fields.

1. Definition and Focus

- **Traditional Engineering:** This encompasses a broad range of disciplines, including civil, mechanical, electrical, and aerospace engineering. It focuses on the design, construction, and maintenance of structures, machines, and systems. Traditional engineering relies on well-established principles of physics, mathematics, and material science to solve complex problems and create functional artifacts.
- **Prompt Engineering:** This is a relatively new field within artificial intelligence and natural language processing. It concentrates on the formulation of input prompts to guide AI models, particularly language models, to generate desired outputs. Prompt engineering emphasizes the strategic use of language and context to optimize AI interactions and ensure relevance and coherence in responses.

2. Methodology

- **Traditional Engineering:** Engineers utilize a systematic approach that includes problem identification, requirements analysis, design, prototyping, testing, and implementation. This process often involves rigorous testing and validation of materials and designs, ensuring safety, functionality, and adherence to regulatory standards.
- **Prompt Engineering:** This practice involves iteratively designing and refining prompts to achieve specific outcomes from AI models. It incorporates elements of experimentation, where different prompts are tested to observe variations in AI responses. Feedback loops play a crucial role in this process, as engineers adjust prompts based on the quality and relevance of the AI outputs.

3. Tools and Technologies

- **Traditional Engineering:** Engineers employ a variety of tools and technologies, including computer-aided design (CAD) software, simulation tools, and project management software. They also utilize advanced manufacturing techniques, such as 3D printing and robotics, to create tangible products.
- **Prompt Engineering:** Practitioners use AI models, natural language processing tools, and programming interfaces (APIs) to test and refine prompts. They often rely on analytics and evaluation metrics to assess the effectiveness of prompts in eliciting desired responses from AI systems.

4. Objectives

- **Traditional Engineering:** The primary objective is to create efficient, safe, and sustainable solutions to real-world problems. This may involve the construction of infrastructure, the development of new materials, or the design of complex systems that serve specific functions.
- **Prompt Engineering:** The objective is to optimize communication between humans and AI, enhancing the accuracy, relevance, and creativity of AI-generated outputs. This can involve generating coherent text, answering questions accurately, or producing content that meets specific stylistic criteria.

5. Interdisciplinary Nature

- **Traditional Engineering:** While it often draws from multiple disciplines, traditional engineering typically adheres to established scientific and mathematical principles. It is grounded in empirical data and rigorous methodologies.
- **Prompt Engineering:** This field is inherently interdisciplinary, intersecting with linguistics, cognitive science, and computer science. It requires an understanding of language nuances, user intent, and AI behavior, necessitating a blend of skills that may not be traditionally associated with engineering.

THE BASICS OF AI AND LANGUAGE MODELS

Evolution of AI

The history of artificial intelligence can be traced back to the mid-20th century when the first AI concepts emerged. Over the years, AI has evolved through several stages, each marked by significant advancements in technology and methodology. One of the most notable transitions in AI has been the shift from rule-based systems to generative models. In the early days of AI, rule-based systems were the predominant approach. These systems relied on a set of predefined rules and decision trees to process input data and produce output. While rule-based systems were effective for simple tasks and scenarios with limited variability, they struggled to scale and adapt to more complex and dynamic situations. The rigidity of these systems made it challenging to account for the vast array of possibilities and nuances found in real-world problems. As AI research progressed, machine learning emerged as a more flexible and adaptive approach. Machine learning models learn patterns from training data and apply those patterns to make predictions or decisions. This data-driven approach enabled AI systems to tackle increasingly complex tasks and better generalize to new situations. Generative AI represents the next leap in AI's evolution, building upon machine learning's foundation. Unlike traditional machine learning models that focus on discriminative tasks—determining the most likely output given an input—generative models aim to generate new data based on the patterns and distributions observed in the training data. This ability to create novel data allows generative AI to excel in a wide range of applications, including natural language processing, image synthesis, and more.

Explanation of AI Language Models and Their Architecture

AI language models are advanced algorithms designed to understand, generate, and manipulate human language. These models leverage deep learning techniques, particularly neural networks, to process and produce text that mimics human language patterns.

Architecture

The architecture of language models typically involves several key components:

1. **Neural Network Structures:** Most modern language models are built upon architectures such as Transformers, which utilize mechanisms like self-attention to weigh the significance of different words in a sentence relative to one another. This enables the model to capture context and meaning effectively.
2. **Embedding Layers:** Words are transformed into high-dimensional vectors through embedding layers. These embeddings capture semantic relationships, allowing the model to understand nuances in meaning based on word usage in various contexts.
3. **Attention Mechanism:** The attention mechanism in Transformer models allows the model to focus on relevant parts of the input when generating outputs. This capability enhances contextual understanding and ensures that the generated text is coherent and contextually appropriate.
4. **Decoder and Encoder:** In many architectures, particularly in sequence-to-sequence models, an encoder processes the input data to create a context representation, while a decoder generates the output based on this representation. Some models, like GPT (Generative Pre-trained Transformer), are decoder-only architectures, primarily focused on text generation.
5. **Layers and Parameters:** Language models consist of multiple layers of neurons, each capable of capturing increasingly complex patterns in language. The number of layers and the parameters (weights and biases) within these layers greatly influence the model's capacity and performance.

The Role of Training Data in Shaping Model Responses

Training data is fundamental to the performance and behavior of AI language models. The model learns to generate text based on patterns and information gleaned from vast datasets. Key aspects of how training data shapes model responses include:

1. **Diversity and Quality:** The breadth and quality of the training data directly impact the model's ability to understand and generate diverse and nuanced language. High-quality, diverse datasets enable the model to learn various language styles, dialects, and topics, enhancing its versatility.
2. **Contextual Understanding:** Training data provides context for the model, allowing it to learn how words and phrases interact within different sentences and situations. This understanding is crucial for generating coherent and contextually relevant responses.
3. **Bias and Representation:** The training data can introduce biases present in the source material. If the dataset is skewed toward certain demographics, ideologies, or language uses, the model may replicate these biases in its outputs. This emphasizes the importance of curating training datasets to promote fairness and inclusivity.
4. **Task-Specific Training:** Fine-tuning language models on specific datasets allows them to perform particular tasks, such as sentiment

analysis, translation, or summarization. This process tailors the model's responses to meet specific user needs.

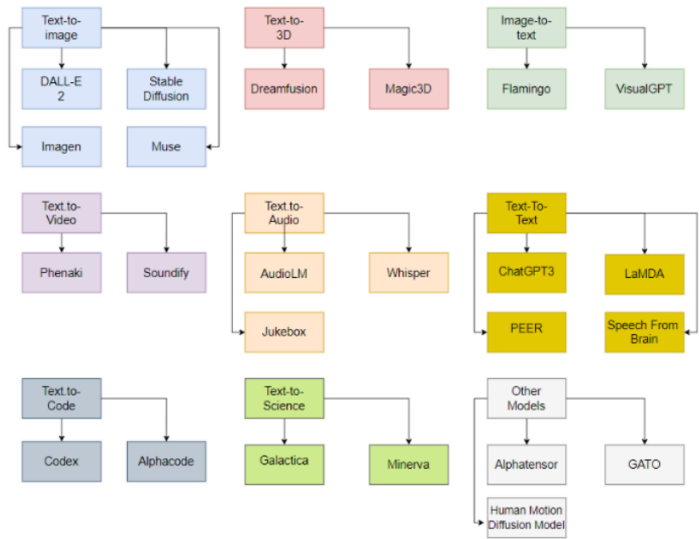
Examples of Popular Language Models

Several prominent language models exemplify the advancements in this field:

1. **GPT (Generative Pre-trained Transformer):** Developed by OpenAI, GPT models (including GPT-3 and the latest versions) are known for their capacity to generate human-like text across a wide range of topics. They are used in applications such as chatbots, content creation, and more.
2. **BERT (Bidirectional Encoder Representations from Transformers):** Created by Google, BERT focuses on understanding the context of words in a sentence by looking at both preceding and following words. This bidirectional approach enhances its performance in tasks like question answering and sentiment analysis.
3. **T5 (Text-to-Text Transfer Transformer):** Also developed by Google, T5 frames all NLP tasks as a text-to-text problem, allowing it to be versatile in handling various applications, from translation to summarization.
4. **XLNet:** A model that combines ideas from both BERT and autoregressive models, XLNet captures bidirectional contexts while maintaining the capability to predict the next word in a sequence, enhancing its performance on various language tasks.
5. **RoBERTa:** An optimized version of BERT, RoBERTa improves performance by utilizing larger training datasets and dynamic masking during training, leading to enhanced understanding and generation capabilities.

ADVANCEMENTS IN AI MODELS: A COMPREHENSIVE OVERVIEW

The field of artificial intelligence (AI) has seen remarkable advancements in recent years, particularly in the development of models that can process, generate, and understand various forms of data, including text, images, audio, and video. These models leverage sophisticated algorithms and vast datasets to produce outputs that closely align with human expectations and creativity. This overview explores a diverse range of AI models categorized by their specific functionalities, highlighting their architectures, applications, and significance in various domains.



Text-to-Image Models

1. **DALL-E 2:** Developed by OpenAI, DALL-E 2 is a cutting-edge model that generates highly detailed images from textual descriptions. Building on its predecessor, DALL-E, this model improves the quality and coherence of images while understanding nuanced prompts. It is particularly notable for its ability to create imaginative and diverse visual outputs, making it a powerful tool for artists, designers, and content creators.
2. **Imagen:** Created by Google Research, Imagen focuses on producing photorealistic images from text. Utilizing advanced diffusion techniques, this model excels in generating images that not only match the textual input closely but also exhibit remarkable detail and realism. Its emphasis on quality makes it a significant player in the field of text-to-image generation.
3. **Stable Diffusion:** This model employs a diffusion process to iteratively generate images based on text prompts. Stable Diffusion is noteworthy for its efficiency, allowing high-resolution outputs to be produced on

consumer-grade hardware. This accessibility democratizes advanced image generation capabilities, enabling a broader audience to engage with AI technology.

4. **Muse:** Muse is designed for creating artistic visuals from textual descriptions, incorporating styles from various art movements. Users can specify artistic parameters in their prompts, leading to unique and stylistically varied images. This model is particularly appealing for those in creative fields seeking inspiration or new artistic expressions.

Text-to-3D Models

1. **DreamFusion:** This innovative approach generates 3D models from text descriptions, employing neural rendering techniques to create three-dimensional representations. DreamFusion enhances the interaction between text and 3D content creation, proving useful in applications such as gaming, virtual reality, and product design.
2. **Magic3D:** Similar to DreamFusion, Magic3D focuses on producing high-quality 3D models based on textual descriptions. By integrating advanced geometry processing and rendering techniques, it ensures that the generated models are not only visually appealing but also functional in real-world applications.

Text-to-Video Models

1. **Phenaki:** This model generates video content from textual descriptions, enabling users to create dynamic and narrative-driven videos. By leveraging advancements in temporal coherence, Phenaki ensures that generated videos are fluid and logically structured, aligning closely with the provided text. This capability opens new avenues for storytelling and multimedia content creation.

Audio Processing Models

1. **Soundify:** Soundify transforms textual descriptions into soundscapes or musical compositions, enabling users to generate auditory experiences based on text inputs. By combining sound design with music generation, this model produces engaging audio content suitable for various applications, including advertising and immersive experiences.
2. **AudioLM:** This model generates audio from text prompts, particularly focusing on speech and music. AudioLM emphasizes maintaining high fidelity and contextual relevance, making it suitable for applications like virtual assistants and interactive media.
3. **Whisper:** Developed by OpenAI, Whisper is an advanced speech recognition model that translates spoken language into text. Supporting multiple languages and dialects, Whisper excels in understanding nuanced speech, providing valuable capabilities for transcription services, real-time translation, and accessibility tools.

Conversational and Text Generation Models

1. **LaMDA (Language Model for Dialogue Applications):** Created by Google, LaMDA is designed specifically for conversational AI. It enables more natural and open-ended dialogues, focusing on engaging and context-aware interactions that improve the user experience in chatbots and virtual assistants.
2. **Jukebox:** Jukebox generates music across various genres and styles from textual prompts, including lyrics and thematic suggestions. By capturing the intricacies of music composition, Jukebox offers users a unique platform for music creation, bridging the gap between AI and artistic expression.
3. **PEER:** This model enhances collaborative text generation, allowing multiple inputs to create cohesive narratives or documentation. PEER is particularly useful in team settings where diverse perspectives contribute to the final output, streamlining collaborative workflows.

Text-to-Text Models

1. **ChatGPT-3:** One of the most well-known conversational AI models, ChatGPT-3, developed by OpenAI, excels at generating coherent and contextually relevant text based on user prompts. It is capable of assisting with a wide range of tasks, from casual conversations to professional content creation, showcasing the versatility of AI in text generation.

Code Generation Models

1. **Codex:** An evolution of the GPT-3 model, Codex specializes in translating natural language into programming code. It supports multiple programming languages and can generate code snippets based on user queries, making it an invaluable tool for developers and programmers.
2. **AlphaCode:** Designed to solve competitive programming problems, AlphaCode analyzes problem statements and produces code solutions. It demonstrates a strong understanding of algorithms and data structures, positioning itself as a useful resource for coding challenges.
3. **Minerva:** This model focuses on solving mathematical problems and generating relevant code solutions. Minerva can tackle various mathematical tasks, from basic calculations to complex theorem proving, highlighting the integration of mathematics and AI.

Multimodal and Cross-Disciplinary Models

1. **Gato:** A versatile AI model capable of performing various tasks across different domains, Gato exemplifies the concept of a generalized agent that can learn and adapt to multiple tasks. Its ability to handle diverse inputs and outputs makes it a significant advancement in the quest for general AI.
2. **Galactica:** This model generates scientific content and research-related text based on user prompts, facilitating scientific communication and knowledge dissemination. By producing coherent and relevant academic content, Galactica enhances accessibility to scientific information.

Other Innovative Models

1. **AlphaTensor:** Focused on optimizing matrix multiplication algorithms, AlphaTensor represents a significant advancement in computational efficiency within mathematics and computer science. Its contributions can lead to improvements in various applications requiring matrix operations.
2. **Human Motion:** This model analyzes and generates human motion data, making it useful in fields such as animation, robotics, and biomechanics. By capturing intricate details of human movement, it can simulate realistic motion sequences for entertainment and research applications.

Analysis of Technologies Behind the Models

1. **Transformer Architecture:** Most of these models, including DALL-E 2, ChatGPT-3, and LaMDA, rely on transformer architectures, which utilize self-attention mechanisms to understand and generate complex data. This architecture has revolutionized NLP and is increasingly applied in other domains, such as image and audio generation.
2. **Generative Models:** Models like Jukebox and Soundify use generative techniques to create new content. These models learn from existing data and use this knowledge to produce unique outputs, whether that's music, soundscapes, or images.
3. **Diffusion Models:** Imagen and Stable Diffusion employ diffusion processes to generate images iteratively. These models start with random noise and gradually refine it into coherent images, allowing for high-resolution outputs.
4. **Neural Rendering:** For 3D models, technologies like DreamFusion and Magic3D utilize neural rendering techniques to translate text descriptions into detailed 3D representations, facilitating more interactive and immersive experiences.
5. **Sequence Modeling:** AudioLM and Whisper leverage sequence modeling techniques to process and generate audio. These models can understand context and maintain coherence over time, which is crucial for audio applications.
6. **Reinforcement Learning:** Models like AlphaTensor use reinforcement learning to optimize specific tasks, such as matrix multiplication, demonstrating the versatility of AI across various computational challenges.

ELEMENTS OF A PROMPT

Prompts are crucial in guiding AI models to generate relevant and accurate responses. Each element of a prompt plays a specific role in shaping the model's output. Below is an elaboration on the four primary elements of a prompt: instruction, context, input data, and output indicator.

1. Instruction

Definition: An instruction is a specific task or directive that guides the AI model on what it should do. It clearly defines the action the model is expected to perform.

Significance:

- **Clarity:** A well-defined instruction helps the model understand the user's expectations, leading to more relevant outputs. For instance, asking the model to "summarize the following article" directs it to focus on condensing information rather than generating new content.
- **Task Specification:** Instructions can vary from simple requests (like "define X") to complex tasks (such as "generate a poem in the style of Shakespeare"). This specificity ensures that the model is focused on the correct task.

Examples:

- "Translate the following sentence into French."
- "Write a short story about a dragon and a princess."
- "List the pros and cons of renewable energy."

2. Context

Definition: Context refers to any external information or background knowledge that provides additional clarity or relevance to the prompt. This can include previous interactions, relevant facts, or specific details related to the task.

Significance:

- **Enhances Relevance:** Context helps the model generate more tailored and meaningful responses. For example, providing background information about a specific topic can result in a more informed answer.
- **Avoids Ambiguity:** By including context, users can minimize misunderstandings. If a user asks for information about "bark," specifying that it refers to "tree bark" rather than "dog barking" avoids confusion.

Examples:

- "In the context of climate change, explain the importance of renewable energy."
- "As a historical figure, how would Albert Einstein view modern technology?"
- "Considering recent events in the tech industry, summarize the key trends."

3. Input Data

Definition: Input data comprises the specific information, question, or subject matter that the user wants the model to address or respond to. It forms the core of what the model will analyze or generate content about.

Significance:

- **Foundation for Response:** The input data directly influences the quality and relevance of the output. Precise and relevant input allows the model to provide accurate answers.
- **Flexibility:** Input data can vary widely—from single keywords to complex paragraphs. This flexibility enables users to ask a broad range of questions or request different types of information.

Examples:

- "What are the symptoms of influenza?"
- "Describe the main events of World War II."
- "Provide a recipe for chocolate chip cookies."

4. Output Indicator

Definition: An output indicator specifies the desired format, type, or structure of the response that the user expects from the model. This can include instructions on length, style, or particular formatting.

Significance:

- **Guides Presentation:** By indicating the output format, users can ensure that the responses are not only accurate but also suitable for their needs. For instance, a user may require a list, a paragraph, or a table, and the output indicator helps achieve that.
- **Facilitates Use:** Specifying an output style can make it easier for users to integrate the generated content into their work or applications. For instance, requesting a bullet-point summary is helpful for quick reference, while a detailed explanation might be needed for thorough understanding.

Examples:

- "Provide a bullet-point summary of the key features."
- "Write a formal letter of recommendation."
- "Generate a list of five tips for effective time management."

Comparison of Prompt Elements

When evaluating the four key elements of a prompt—instruction, context, input data, and output indicator—it's essential to understand their unique roles, interactions, and how they collectively influence the output of AI models. Here's a comparative analysis of these elements:

Element	Role	Focus	Impact on Output	Examples
Instruction	Directs the model on what task to perform	Action-oriented	Determines the nature of the task; shapes response quality	"Summarize the article."
Context	Provides background or additional information	Contextual understanding	Enhances relevance and specificity; reduces ambiguity	"In the context of climate change, explain..."
Input Data	Contains the specific information to analyze	Core subject matter	Forms the basis of the response; influences accuracy	"What are the symptoms of influenza?"
Output Indicator	Specifies the desired format or style of output	Presentation and format	Guides the structure of the response; aids usability	"Provide a bullet-point summary."

Analysis of Element Interactions

- 1. Instruction and Context:**
 - **Interaction:** While the instruction outlines the task, the context enriches it. A clear instruction with relevant context can lead to a more nuanced response.
 - **Example:** "Translate the following sentence into French, considering formal language use." Here, the instruction specifies the action, while the context shapes how the action is performed.
- 2. Input Data and Output Indicator:**

- **Interaction:** The input data defines what is being asked, and the output indicator clarifies how that information should be presented. The effectiveness of the output often relies on both being well-defined.
- **Example:** "List the pros and cons of renewable energy in bullet points." The input specifies the subject, while the output indicator dictates the format, enhancing readability.
- 3. **Context and Input Data:**
 - **Interaction:** Context can greatly influence how input data is interpreted. A well-contextualized question can lead to a more precise and relevant response.
 - **Example:** "Considering the latest research on climate change, what are the main benefits of renewable energy?" The context ensures the model understands the relevance of current information in responding to the input data.
- 4. **Instruction and Output Indicator:**
 - **Interaction:** The instruction provides the task, while the output indicator clarifies how to complete that task effectively. Together, they ensure the model produces output that meets user expectations.
 - **Example:** "Explain the significance of photosynthesis in a concise paragraph." The instruction specifies the task, while the output indicator sets the expectation for brevity.

Some Common Examples of Prompts:

The following instances demonstrate how thoughtfully constructed prompts can facilitate the execution of distinct tasks.

Topics:

1. **Text Summarization**

Example Prompt: "Summarize the following article on climate change in three sentences."

This prompt requests a concise summary, allowing for efficient information digestion while retaining the core message of the original text.

2. **Information Extraction**

Example Prompt: "Extract all dates mentioned in the following text."

Here, the prompt specifies the task of identifying dates, enabling focused data retrieval for analysis or record-keeping.

3. **Question Answering**

Example Prompt: "What are the main causes of World War II as described in the following text?"

This prompt encourages precise responses based on the provided content, facilitating targeted knowledge acquisition.

4. **Text Classification**

Example Prompt: "Classify the following news article into categories: sports, politics, or entertainment."

This task utilizes the prompt to categorize content, aiding in content management and retrieval strategies.

5. **Conversation**

Example Prompt: “You are a customer service agent. Respond to the following inquiry: ‘What is your return policy?’”

This example illustrates the use of prompts to simulate a realistic conversational exchange, enhancing user experience through interactive dialogue.

6. **Code Generation**

Example Prompt: “Write a Python function to calculate the factorial of a number.”

This prompt leads to the generation of executable code, showcasing the model's ability to produce programming solutions based on user requirements.

7. **Reasoning**

Example Prompt: “Given the premises that all humans are mortal and Socrates is a human, what can we conclude about Socrates?”

This task requires logical reasoning, demonstrating the model's capability to engage in deductive reasoning based on the information provided.

ADVANCED PROMPTING TECHNIQUES

In the realm of language models, the manner in which prompts are constructed significantly influences the quality and relevance of the generated output. This chapter explores advanced prompting techniques, detailing various strategies that can be employed to enhance model performance across diverse tasks. Each technique is illustrated with examples, providing a comprehensive understanding of their applications.

1. Zero-shot Prompting

Zero-shot prompting refers to the practice of instructing the model to perform a task without providing any prior examples. This approach relies on the model's inherent knowledge and understanding of the task at hand.

Example:

Prompt: "Translate the following sentence into French: 'The weather is beautiful today.'"

In this instance, the model must leverage its existing knowledge of language translation to provide an accurate response.

2. Few-shot Prompting

Few-shot prompting builds upon the zero-shot technique by supplying a limited number of examples within the prompt. This helps the model better infer the desired output format and style, thereby enhancing contextual understanding.

Example:

Prompt: "Translate these sentences into French: 1. 'Hello.' 2. 'Goodbye.'"

By presenting examples, the model can more effectively mimic the expected translation style.

3. Chain-of-Thought Prompting

Chain-of-thought prompting encourages the model to articulate its reasoning process step-by-step before arriving at a final answer. This technique is particularly beneficial for complex reasoning tasks, as it promotes clarity and transparency in the model's thought process.

Example:

Prompt: "Explain the process of photosynthesis step by step."

The model is prompted to outline the various stages involved, facilitating a deeper understanding of the concept.

4. Meta Prompting

Meta prompting involves instructing the model on how to approach a problem or task. This technique sets the contextual groundwork for the model's reasoning and response formulation.

Example:

Prompt: "Consider the implications of renewable energy. What are the potential benefits and challenges?"

By framing the inquiry in this manner, the model is guided to explore multiple facets of the topic.

5. Self-Consistency

Self-consistency entails generating multiple responses to the same prompt and selecting the most frequent or coherent answer. This method enhances reliability, especially in situations where uncertainty is present.

Example:

Prompt: “What are the benefits of meditation?”

The model generates several responses, allowing for the identification of common themes to produce a consolidated answer.

6. Generate Knowledge Prompting

Generate knowledge prompting focuses on eliciting fact-based or knowledge-rich responses from the model. This technique is particularly useful for tasks requiring accurate information retrieval.

Example:

Prompt: “What are the key findings from recent studies on climate change?”

The model draws on its knowledge to summarize relevant research findings effectively.

7. Prompt Chaining

Prompt chaining involves linking multiple prompts together, where the output from one serves as input for the next. This technique guides the model through a structured, multi-step process.

Example:

Prompt: “First, summarize the article. Then, provide three questions based on the summary.”

This method facilitates a comprehensive exploration of the content by guiding the model stepwise.

8. Tree of Thoughts

The tree of thoughts technique encourages branching reasoning, allowing the model to explore various paths of thought before converging on a final conclusion. This approach is particularly useful for complex issues requiring nuanced analysis.

Example:

Prompt: “What are the potential outcomes of implementing a universal basic income? List various perspectives.”

The model can generate a diverse array of viewpoints, enriching the discussion.

9. Retrieval Augmented Generation

Retrieval augmented generation combines generated text with information retrieved from external sources, enhancing both the richness and accuracy of responses. This technique is particularly advantageous for fact-based tasks.

Example:

Prompt: “Using the retrieved data, explain the impact of social media on youth mental health.”

Here, the model incorporates external information to provide a well-rounded analysis.

10. Automatic Reasoning and Tool-use

This technique allows the model to automatically invoke external tools or reasoning capabilities to address complex problems. It expands the model's functionality beyond mere text generation.

Example:

Prompt: "Use a calculator to find the square root of 256 and explain the process."

In this case, the model must utilize reasoning to provide both the calculation and an explanation.

11. Automatic Prompt Engineer

Automatic prompt engineering involves the use of algorithms to optimize and refine prompts autonomously. This enhances the model's performance across various tasks by continuously improving prompt quality.

Example:

Prompt: "Generate prompts that elicit detailed explanations of scientific concepts."

This technique enables the model to identify and implement the most effective prompts.

12. Active-Prompt

Active-prompting dynamically adjusts prompts based on the model's ongoing performance, continuously refining interactions to enhance output quality.

Example:

Prompt: "If the previous answer was unclear, rephrase the question to clarify expectations."

This approach encourages iterative refinement, leading to improved clarity in communication.

13. Directional Stimulus Prompting

Directional stimulus prompting guides the model toward a specific type of response by framing the prompt in a manner that suggests the desired focus.

Example:

Prompt: "Describe the environmental impacts of urbanization in a positive light."

By directing the model's focus, this technique yields responses that align with specific narrative angles.

14. Program-Aided Language Models

This technique integrates programming logic or specific coding tasks within the prompt, enhancing the model's capacity to handle technical queries.

Example:

Prompt: “Write a Python script that sorts a list of numbers.”

The model is tasked with producing executable code based on user specifications.

15. ReAct

ReAct encourages the model to respond dynamically to given situations, fostering an interactive and responsive environment conducive to natural dialogue.

Example:

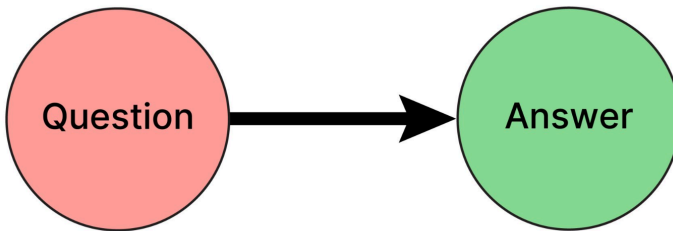
Prompt: “If a user expresses frustration, respond empathetically and offer solutions.”

This method promotes a conversational flow that enhances user engagement.

By employing these advanced prompting techniques, users can significantly enhance the capabilities of language models, enabling them to perform a wide array of tasks with improved accuracy and relevance. As these methods evolve, they open new avenues for interaction and application, reinforcing the potential of artificial intelligence in various domains.

ZERO-SHOT PROMPTING

Large language models (LLMs), including prominent iterations such as GPT-3.5 Turbo, GPT-4, and Claude 3, have been meticulously engineered to adhere to user instructions while leveraging extensive datasets for training. The expansive scale of this training endows these models with the capability to execute certain tasks in a "zero-shot" manner. Zero-shot prompting refers to a method of interaction in which the prompts issued to the model do not incorporate examples or demonstrations. Instead, these prompts directly instruct the model to perform a specific task without additional context or guidance, allowing for a more straightforward engagement.



Zero-shot

To illustrate this concept, consider the following example involving text classification:

Prompt:

Classify the text into neutral, negative, or positive.

Text: I think the vacation is okay.

Sentiment:

Output:

Neutral

In this instance, it is noteworthy that the prompt did not present the model with any prior examples of text alongside their corresponding classifications. The model's understanding of the term "sentiment" is indicative of its zero-shot capabilities.

Recent research, such as that conducted by Wei et al. (2022), has demonstrated that instruction tuning can significantly enhance zero-shot learning capabilities. Instruction tuning involves fine-tuning models on datasets that are articulated via explicit instructions, thereby aligning the model's output more closely with user expectations. Moreover, the application of reinforcement learning from human feedback (RLHF) has emerged as a vital methodology for scaling instruction tuning, allowing

models to be calibrated to better align with human preferences. This innovative approach underpins the functionality of contemporary models like ChatGPT.

As we explore the efficacy of zero-shot prompting, it is essential to acknowledge its limitations. In instances where zero-shot prompting yields suboptimal results, it is advisable to incorporate demonstrations or examples within the prompt, a practice known as few-shot prompting. The forthcoming section will delve into the mechanics and applications of few-shot prompting, further elucidating its role in enhancing model performance.

Applications of Zero-Shot Prompting

Text Classification:

Application: Automatically categorizing texts based on sentiment, topic, or intent.

Example:

Prompt: Classify the following review as positive, negative, or neutral.

Text: "The service was prompt, but the food was mediocre."

Output: Neutral

Question Answering:

Application: Providing answers to questions based on general knowledge or contextual understanding.

Example:

Prompt: What is the capital of France?

Output: Paris

Named Entity Recognition:

Application: Identifying and classifying key entities within a text, such as names, locations, and organizations.

Example:

Prompt: Identify the named entities in the following sentence.

Text: "Apple Inc. was founded by Steve Jobs in Cupertino."

Output: [Apple Inc. - Organization, Steve Jobs - Person, Cupertino - Location]

Summarization:

Application: Generating concise summaries of longer texts.

Example:

Prompt: Summarize the following article.

Text: "The climate crisis is affecting weather patterns globally, leading to more extreme weather events."

Output: "The climate crisis is causing more extreme weather events worldwide."

Applications of Few-Shot Prompting

Chatbot Development:

Application: Training conversational agents to respond appropriately in varied contexts by providing examples of desired interactions.

Example:

Prompt: Here are some examples of good responses:

User: "What's the weather like today?"

Assistant: "It's sunny with a high of 75°F."

User: "Can you recommend a book?"

Assistant: "Sure! I recommend 'To Kill a Mockingbird'."

Now, respond to this user: "How do I make a chocolate cake?"

Output: "To make a chocolate cake, you'll need flour, sugar, cocoa powder, baking powder, and eggs. Mix the ingredients and bake at 350°F for 30 minutes."

Content Generation:

Application: Producing creative content, such as stories, poetry, or marketing copy, by providing stylistic guidelines and examples.

Example:

Prompt: Here are examples of short marketing slogans:

"Just Do It"

"Think Different"

Generate a new slogan for a coffee brand.

Output: "Awaken Your Senses."

Programming Assistance:

Application: Helping developers write code snippets or solve coding problems based on a few provided examples.

Example:

Prompt: Here are some Python functions:

```
def add(a, b): return a + b
```

```
def subtract(a, b): return a - b
```

Write a function to multiply two numbers.

Output:

```
def multiply(a, b): return a * b
```

Translation:

Application: Translating phrases or sentences by providing examples of translation style and language nuances.

Example:

Prompt: Here are translations:

"Hello" → "Hola"

"Thank you" → "Gracias"

Translate: "Goodbye."

Output: "Adiós."

Zero-Shot Prompting: Pros and Cons

Zero-shot prompting, wherein a model is instructed to perform a task without prior examples, offers distinct advantages and challenges. Here's an overview of the pros and cons:

Pros

Efficiency:

Time-Saving: Eliminates the need for example generation, allowing users to quickly obtain responses without constructing elaborate prompts.

Scalability: Can be applied to numerous tasks simultaneously without tailoring prompts for each specific instance.

Flexibility:

Task Generalization: Models can adapt to a wide variety of tasks, leveraging their training data to infer the requirements based on the prompt alone.

Versatile Applications: Useful in scenarios where task specifications may vary significantly, enabling quick shifts in focus or topic.

Reduced Complexity:

Simplicity in Interaction: Users can engage with the model using straightforward commands, making it accessible for non-technical users.

Less Cognitive Load: By omitting examples, users can concentrate on the primary instruction without being distracted by formatting or structure.

Potential for Novelty:

Creative Outputs: Models may produce innovative or unexpected responses since they aren't constrained by predefined examples, allowing for creative problem-solving.

Cons

Variability in Accuracy:

Unpredictable Performance: The lack of specific examples can lead to inconsistent outputs, especially for complex tasks where nuanced understanding is critical.

Inherent Ambiguity: The model may misinterpret instructions, resulting in outputs that do not meet user expectations.

Limited Contextual Understanding:

Lack of Specificity: Without examples, the model may struggle to grasp context or particular requirements, leading to generic or irrelevant responses.

Difficulty with Nuanced Tasks: Tasks requiring a high degree of specificity or subtlety may be poorly executed due to the absence of guiding examples.

Dependency on Training Data:

Bias and Gaps: The effectiveness of zero-shot prompting relies heavily on the diversity and quality of the model's training data; biases present in the data can lead to skewed outputs.

Knowledge Limitations: The model's responses are confined to the information encoded during training, which can limit its ability to address emerging topics or domains not well-represented in the dataset.

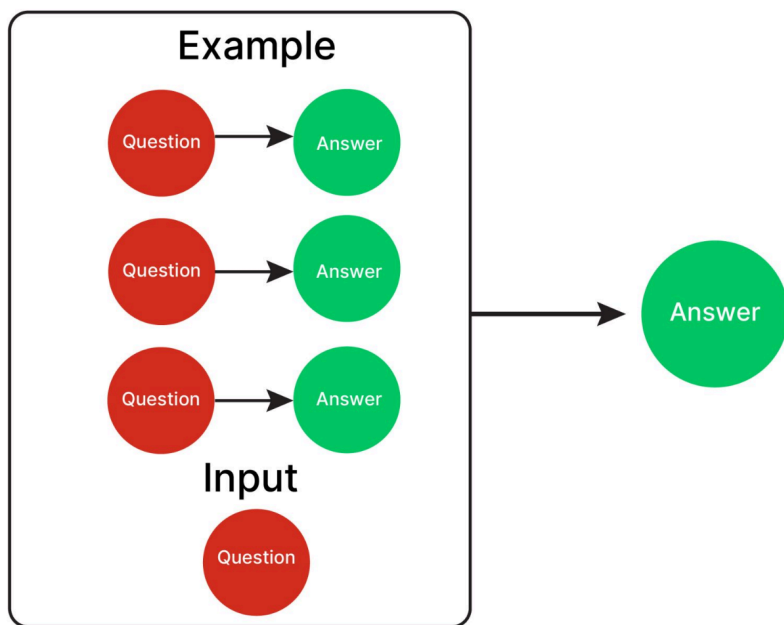
Challenges in Complex Tasks:

Difficulty with Multi-Step Instructions: Tasks that require sequential reasoning or multi-part instructions may not be effectively handled without clear examples to guide the process.

FEW-SHOT PROMPTING

Few-shot prompting is a technique that enhances the performance of large language models (LLMs) by providing a small number of examples within the prompt. This method enables in-context learning, where the model can leverage the demonstrations to better understand the task and produce more accurate responses. This technique is especially beneficial for complex tasks that may be beyond the capabilities of zero-shot prompting alone.

Foundations of Few-Shot Prompting



Research by Touvron et al. (2023) suggests that the properties of few-shot prompting became prominent as LLMs were scaled to sufficient sizes, as noted in Kaplan et al. (2020). The ability to perform few-shot learning effectively is increasingly recognized as a hallmark of advanced models, reflecting their capacity to generalize from limited input.

Example of Few-Shot Prompting

To illustrate few-shot prompting, we can reference an example from Brown et al. (2020) concerning the use of newly introduced words in sentences.

Prompt:

A "whatpu" is a small, furry animal native to Tanzania. An example of a sentence that uses the word "whatpu" is:

"We were traveling in Africa and we saw these very cute whatpus."

To do a "farduddle" means to jump up and down really fast. An example of a sentence that uses the word "farduddle" is:

Output:

"When we won the game, we all started to farduddle in celebration."

In this case, the model effectively learned to apply the new term "farduddle" based on just one provided example, showcasing the concept of one-shot learning.

Enhancing Few-Shot Prompting

When faced with more challenging tasks, increasing the number of demonstrations can lead to improved outcomes. For instance, research by Min et al. (2022) indicates that both the label space and the input text distribution specified by the demonstrations are critical for effective few-shot prompting. The formatting of the examples also plays a significant role in performance, even if the labels themselves are randomly assigned.

Example with Random Labels:

Prompt:

This is awesome! // Negative

This is bad! // Positive

Wow that movie was rad! // Positive

What a horrible show! //

Output:

Negative

This example demonstrates that even with randomly assigned labels, the model can produce the correct output, emphasizing the robustness of the formatting.

Limitations of Few-Shot Prompting

Despite its strengths, few-shot prompting is not without its limitations, particularly in tasks requiring complex reasoning. A previous example illustrates this challenge:

Prompt:

The odd numbers in this group add up to an even number: 15, 32, 5, 13, 82, 7, 1.

A:

The model's response indicated an incorrect understanding of the task, stating, "Yes, the odd numbers in this group add up to 107, which is an even number." This error underscores the limitations of few-shot prompting in cases where deeper reasoning or multiple steps are involved.

Attempting to Improve Responses

To assess whether few-shot prompting can yield better results, we can provide additional examples with clear answers to guide the model:

Prompt:

The odd numbers in this group add up to an even number: 4, 8, 9, 15, 12, 2, 1.

A: The answer is False.

The odd numbers in this group add up to an even number: 17, 10, 19, 4, 8, 12, 24.

A: The answer is True.

The odd numbers in this group add up to an even number: 16, 11, 14, 4, 8, 13, 24.

A: The answer is True.

The odd numbers in this group add up to an even number: 17, 9, 10, 12, 13, 4, 2.

A: The answer is False.

The odd numbers in this group add up to an even number: 15, 32, 5, 13, 82, 7, 1.

A:

Output:

The answer is True.

This example illustrates that while additional context and examples were provided, the model still produced an incorrect response. This limitation indicates that merely supplying examples may not suffice for tasks requiring advanced reasoning, highlighting the need for more sophisticated prompt engineering techniques.

Applications of Few-Shot Prompting

Few-shot prompting has a wide range of applications across various domains, leveraging the ability of large language models (LLMs) to learn from minimal examples. Here are some key applications:

Natural Language Processing (NLP):

Sentiment Analysis: Classifying text sentiment (positive, negative, neutral) with minimal examples.

Example:

Prompt: "This movie was fantastic!" // Positive

"I did not enjoy the meal." // Negative

"The service was mediocre." //

Output: Neutral

Text Generation:

Creative Writing: Assisting in generating stories, poetry, or marketing copy based on style examples.

Example:

Prompt: "Write a short poem about autumn."

"The leaves fall gently, golden and bright,

As daylight fades into the night."

"Now write a similar poem about winter."

Output: "Snowflakes dance in the chilly breeze,

As winter whispers through the trees."

Programming Assistance:

Code Generation: Helping developers with code snippets or solutions based on examples.

Example:

Prompt: “Here’s a function to add two numbers:

```
def add(a, b): return a + b
```

Now, write a function to subtract two numbers.”

Output: “def subtract(a, b): return a - b”

Translation:

Language Translation: Translating phrases by providing examples of translations in the desired style.

Example:

Prompt: “Translate the following: ‘Good morning’ in Spanish is ‘Buenos días’.

Now translate ‘Good night’.”

Output: “Buenas noches.”

Named Entity Recognition (NER):

Identifying Entities: Extracting and categorizing names, locations, and organizations from text.

Example:

Prompt: “In the sentence ‘Apple Inc. is based in Cupertino, California,’ identify the entities.”

Output: “Apple Inc. - Organization, Cupertino - Location, California - Location”

Pros of Few-Shot Prompting

Improved Accuracy:

Better Performance: By providing examples, the model can often produce more accurate and contextually relevant responses.

Flexibility:

Versatile Task Handling: Few-shot prompting allows for adaptation to a variety of tasks without extensive retraining.

Reduced Need for Extensive Datasets:

Efficiency: Users can effectively perform tasks without the need for large labeled datasets, saving time and resources.

Enhanced Understanding of Nuances:

Complex Tasks: Few-shot prompting can help the model understand specific nuances and contexts that are critical for certain tasks.

Cons of Few-Shot Prompting

Complexity in Reasoning Tasks:

Limitations in Depth: For tasks requiring multiple reasoning steps, few-shot prompting may still fall short, leading to incorrect outputs.

Dependence on Quality of Examples:

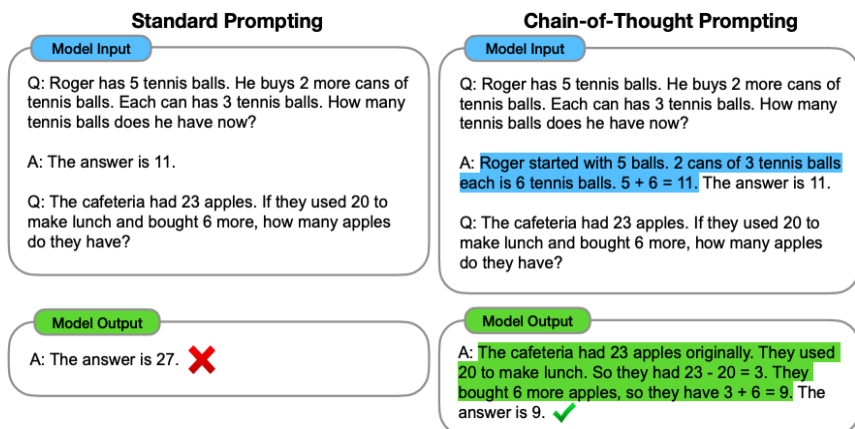
Inconsistent Results: The effectiveness of few-shot prompting heavily relies on the quality and relevance of the provided examples. Poorly chosen examples can mislead the model.

Potential for Bias:

Training Data Influence: If the examples provided are biased or not representative, the model may produce skewed or biased outputs.

CHAIN-OF-THOUGHT (CoT) PROMPTING

Chain-of-thought (CoT) prompting facilitates complex reasoning capabilities through intermediate reasoning steps. This technique can be effectively combined with few-shot prompting to enhance performance on intricate tasks that necessitate reasoning prior to delivering a response.



For example, consider the following problem: "The odd numbers in this group add up to an even number: 4, 8, 9, 15, 12, 2, 1." The response is as follows: "Adding all the odd numbers (9, 15, 1) gives 25. The answer is False." Another case is presented with the group "17, 10, 19, 4, 8, 12, 24," resulting in the answer: "Adding all the odd numbers (17, 19) gives 36. The answer is True." Continuing this method, for the group "16, 11, 14, 4, 8, 13, 24," we find: "Adding all the odd numbers (11, 13) gives 24. The answer is True." In contrast, the group "17, 9, 10, 12, 13, 4, 2" leads to: "Adding all the odd numbers (17, 9, 13) gives 39. The answer is False." Finally, for the group "15, 32, 5, 13, 82, 7, 1," the output is: "Adding all the odd numbers (15, 5, 13, 7, 1) gives 41. The answer is False."

The effectiveness of providing reasoning steps is evident, as it significantly enhances the accuracy of the results. Notably, the authors suggest that this capability emerges in sufficiently large language models, indicating a potential for improved performance with minimal examples.

Zero-Shot CoT Prompting

An emerging approach in the realm of prompt engineering is zero-shot CoT, introduced by Kojima et al. (2022). This method incorporates a directive such as "Let's think step by step" into the original prompt to stimulate logical reasoning. For instance, consider the problem: "I went to the market and bought 10 apples. I gave 2 apples to the neighbor and 2 to the repairman. I then went and bought 5 more apples and ate 1. How

many apples did I remain with?" The initial response yielded an incorrect answer of 11 apples. However, upon applying the zero-shot CoT prompt, the response unfolds as follows: "First, you started with 10 apples. You gave away 2 apples to the neighbor and 2 to the repairman, leaving you with 6 apples. Then you bought 5 more apples, totaling 11 apples. Finally, after eating 1 apple, you would remain with 10 apples."

This methodology demonstrates significant promise, particularly in scenarios where few examples are available for prompting.

Automatic Chain-of-Thought (Auto-CoT)

To further streamline the process of employing chain-of-thought prompting with demonstrations, Zhang et al. (2022) propose an automatic approach known as Auto-CoT. This method aims to minimize the manual effort typically required to craft effective and diverse examples, which may lead to suboptimal solutions. Auto-CoT consists of two primary stages:

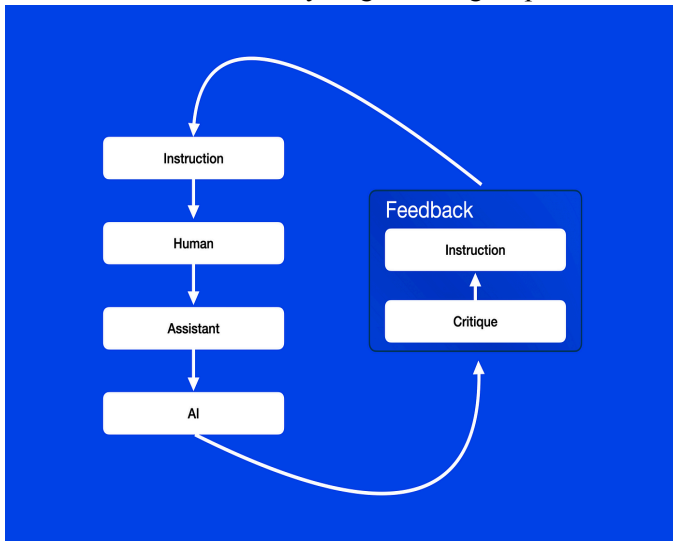
1. **Question Clustering:** This stage involves partitioning questions from a given dataset into several clusters based on similarities.
2. **Demonstration Sampling:** In this stage, a representative question from each cluster is selected, and its reasoning chain is generated using the Zero-Shot CoT approach along with simple heuristics. These heuristics may involve criteria such as the length of questions (e.g., approximately 60 tokens) and the number of steps in the rationale (e.g., around five reasoning steps). This method encourages the model to produce simple yet accurate demonstrations.

The Auto-CoT methodology thus facilitates a more efficient and effective approach to generating reasoning chains for a variety of tasks, leveraging the inherent capabilities of large language models.

META PROMPTING

Introduction

Meta prompting is an advanced technique that shifts the focus from the specific content of tasks to their underlying structure and format. This approach aims to enhance interactions with large language models (LLMs) by emphasizing how information is organized and presented, rather than getting bogged down in the particulars of the content itself. By prioritizing the structural aspects, meta prompting seeks to create a more efficient and flexible way of generating responses.



Working of Meta Prompting

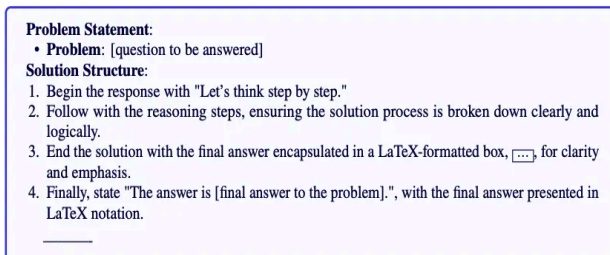


Figure 1: A structure meta prompt presented in markdown format for solving MATH [17] problems.

Key Characteristics

Structure-oriented:

This aspect of meta prompting emphasizes the importance of how problems and solutions are structured. Instead of concentrating on the details of a specific task, it looks at the overarching format. For example, when generating a response, the focus might be on whether the answer is

best presented as a list, a summary, or a comparison, depending on the structure that best fits the query.

Syntax-focused:

Here, the technique leverages the rules and patterns of language to guide responses. By providing a specific syntactical framework, meta prompting helps to ensure that the output aligns with expected linguistic forms. This might involve using particular sentence structures or grammatical cues that signal how the information should be arranged, making the response clearer and more coherent.

Abstract examples:

Instead of using specific, detailed examples, meta prompting employs more generalized or abstract scenarios. These examples illustrate the underlying structure of problems and solutions without getting into the nitty-gritty. For instance, instead of discussing a particular case study, it might present a generic framework for problem-solving that can apply to many situations, helping users see the common patterns.

Versatile:

One of the strengths of meta prompting is its versatility. It can be adapted for a wide range of domains and applications, from education and research to business and creative writing. This broad applicability means that users can utilize the same structural techniques across different contexts, making it a powerful tool for generating structured and relevant responses in various fields.

Categorical approach:

Drawing from type theory, this characteristic emphasizes the importance of categorizing and logically arranging the elements within a prompt. By organizing components into clear categories, users can create prompts that are more effective and easier for LLMs to understand. This systematic approach helps ensure that the model's responses are not only relevant but also logically coherent, as the relationships between different parts of the prompt are made explicit.

GETTING STARTED WITH THE OPENAI PLAYGROUND

Now it's time to take the concepts that we've learned so far and start to put them to good use! However, we need to do first things first, and therefore, you will need to have a developer account with OpenAI and create an API key.

Head over to the following URL to create your dev account and API key:

<https://platform.openai.com/account/api-keys>

As you can see from Figure 1-3, you can name your API key anything that you want.

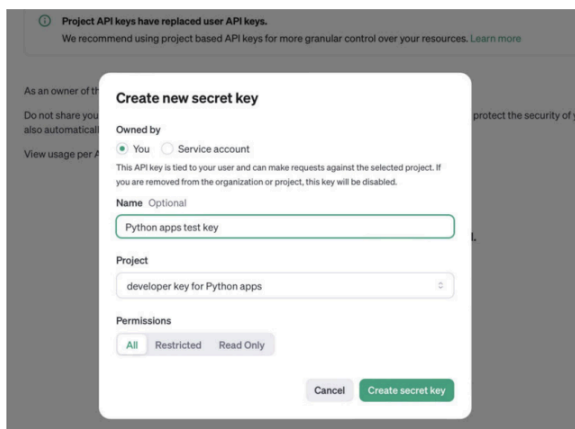


Figure 1-3. Before You Can Access the Playground or Make API Calls, You Need to Have an API Key

You should be aware that as a requirement to create an API key, you will need to provide to OpenAI a credit card so that you can be billed for usage of their models.

Now that you've got your API key, let's go straight to the Chat Playground at the following URL:

<https://platform.openai.com/playground>

Upon entering the Playground, click on the combobox at the top and select the Chat option to start the Chat Playground, as shown in Figure 1-4.

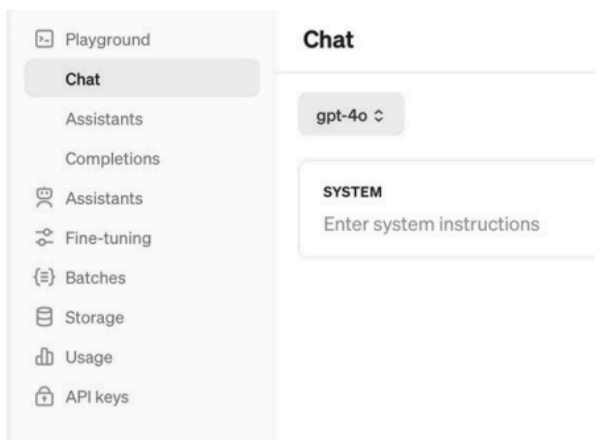


Figure 1-4. After Entering the Playground, Select the Chat Option

Figure 1-5 depicts the Chat Playground, with certain parts numbered so that they can be easily identified.

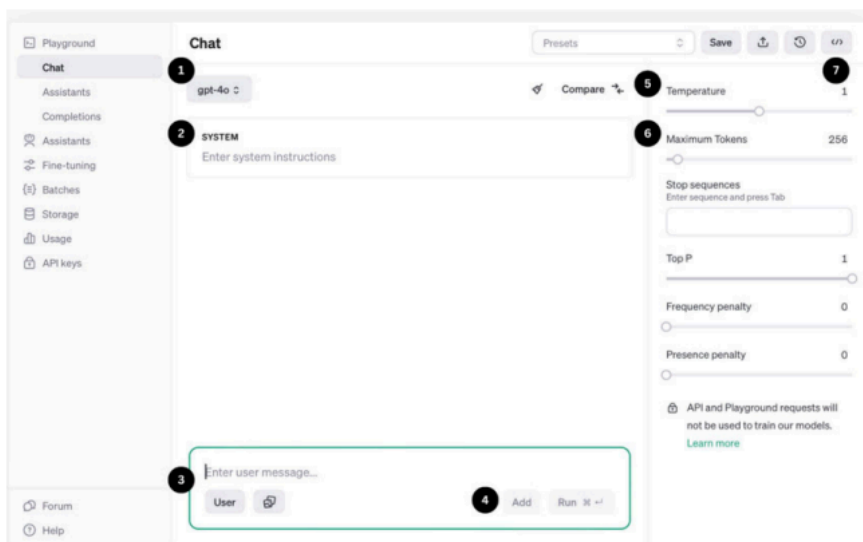


Figure 1-5. The Chat Playground Can Be a Little Daunting at First Glance

Earlier in this chapter, we talked about the various models that are available for developers. Click the Model field in order to see a list of models that are available.

You may also see that some models have a month and day associated with their name, which is simply a snapshot of that model. Programmatically selecting a snapshot enables developers to have some sort of predictability in the responses that they will receive from ChatGPT, because the current models are always updated.

As you can see, the user interface for the Chat Playground is vastly more complex than the ChatGPT website that everyone else uses. So, let's talk about the System field (see Figure 1-5, item 2).

In my opinion, ChatGPT can be described as "a vastly powerful form of artificial intelligence...with amnesia." Therefore, when you're using ChatGPT programmatically, you need to inform the system who it is in the conversation!

Figure 1-6, shown below, gives you a glimpse of the thousands of different roles that ChatGPT can play in a conversation.

1. System

2. Model



Figure 1-6. The System Field in the Chat Playground Allows You to Set the Role That ChatGPT Will Play in the Conversation

3. User/Assistant

The User field (Figure 1-5, item 3) in the Chat Playground is where you type your

prompt to ChatGPT, which can be anything that you want, for example, "describe how telemedicine will affect the medical industry."

When you initially load the Chat Playground, the Assistant field is not visible. In

order to make it appear, you need to click "User" button to switch to the Assistant field.

Now, you may be asking yourself, "Why is this field needed at all?"

Well, that's a fine question. If you want ChatGPT to remember something that it has already told you in a previous conversation, then you need to type into the Assistant field anything that it has already told you that you think is relevant in order to continue with the conversation. Remember, it is a vastly powerful AI, but it has amnesia!

4. Add (Optional)

Add (Figure 1-5, item 4) is where you'd click in order to add either an Assistant message to the conversation or another User message. Now, you may ask, "What's the point of adding yet another User message to

the conversation, when I can type what I want in the original User field above?” Good question.

If you wanted to separate your command from your data, then you would use a

separate User message for that. Do you remember in Listing 1-4 earlier in this chapter, where we had to use the “###” to separate the command to ChatGPT from the data that we wanted it to analyze? Well, this is now no longer needed because the command would be the first User message;

the data would be the second User message.

5. Temperature (Optional)

As noted earlier in this chapter, the temperature selector ranges between 0 and 2 and allows you to select the “randomness” of the response.

6. Maximum Tokens (Optional)

Do you remember the discussion earlier in this chapter about tokens? By selecting anything in the range for this item, you can adjust the number of tokens (which directly affects the number of words) in the response.

7. Code (Optional)

After you have submitted your prompt using the Playground, you can click on the code button (Figure 1-4, item 7) in order to see the code necessary to send the same prompt using any of the languages that they support.

Try It Now!

Now that we’re familiar with several of the features of the Chat Playground, let’s send our first prompt using the settings discussed above. Listings 1-7 and 1-8 below use the same prompt asking ChatGPT to give a few paragraphs on telemedicine, but the role of the system is vastly different from each other.

Listing 1-7. PROMPT. The Pros and Cons of Telemedicine as a Researcher

System: You are a strictly factual researcher

User: Write 3 paragraphs on pros and cons of telemedicine

Listing 1-8. PROMPT. The Pros and Cons of Telemedicine as an Opinionated Health Blogger

System: You are a highly opinionated health blogger who always has stories with firsthand experience

User: Write 3 paragraphs on pros and cons of telemedicine

You are encouraged to try these two prompts yourself and see what the responses are. Adjust the settings for the temperature and token length to get familiar with how those parameters affect the outcome.

Conclusion

You just learned more about how ChatGPT can be used by developers. We covered some of the basics of the Chat Playground, which is a web interface for developers to interact with the ChatGPT API.

We talked about how to set the system, user, and assistant roles in the Chat Playground and how to adjust settings such as the temperature and maximum length of output.

You learned about some of the parameters and terminology necessary to use the Chat Playground, such as the model, temperature, and tokens. Getting familiar with the parameters of the Chat Playground is essential to knowing how to use the REST API since the Playground is a subset of capabilities offered by the REST API.

In the next chapter, we'll see how to use ChatGPT as your "pair-programmer" and create a productivity app that gives us weather and arrival time to work.

USING CHATGPT AS YOUR PYTHON PAIR-PROGRAMMER

I'm a big fan of some of the practices of XP (eXtreme Programming) and especially pairprogramming. No matter what flavor of pair-programming that you prefer, it involves two engineers sitting down at the same screen and solving the same problem together. One of the biggest benefits that you get is a fresh set of eyes on a problem, and of course, you now have two engineers who have “touched” the codebase instead of one. Sometimes you can have one engineer write the code and the other write the tests and the comments. No matter how you slice it, it's all good stuff.

Overview

This chapter will walk you through obtaining and testing out your API key, get you comfortable making calls to the OpenAI Python API for ChatGPT, and introduce you to how it's done for other models as well. Furthermore, we'll be using ChatGPT as a pairprogrammer to get an application that will be able to receive the name of a city and the time you'd like to arrive to work, then give you the weather and an estimated arrival time based on traffic! Sounds exciting? Then, let's jump right in.

Installing (or Updating) the OpenAI Library with pip

In order to use the OpenAI library with Python, you will need at minimum Python 3.7.1 installed on your machine. In order to check the version of Python that you have, just open a terminal window, and execute: `python3 -v`

If Python 3 is installed, then your terminal will display the version that you have.

Now, of course, if your terminal is informing you that you have a noncompatible version (i.e., older than 3.7.1), then you should update Python on your machine. Additionally, if you don't have Python 3 installed at all, then you should install it before proceeding.

Now that you have all the prerequisites, it's time to install the OpenAI Python library itself. Go back to your terminal window, and execute the following command: `pip install --upgrade openai`

The command above will install the OpenAI library for you if it doesn't already exist and will update the library to the most recent version if it's already there.

Three Ways to Set Your API Key

When setting up your API key for the OpenAI API, there are different methods to choose from, each designed for specific project needs and security concerns.

Option #1: Setting a System-Wide Environment Variable

The environment variable approach establishes a system-wide environment variable for API key storage. This provides a centralized

point for key management, simplifying deployment across diverse projects.

Let's look at the steps on how we'd go about doing it.

For Mac OS

First off, open Terminal. You can find it in the applications folder or use spotlight (Command + Space) to search for it.

Next, edit your bash profile. For older MacOS versions, you'd use the command `nano ~/.bash_profile`. Users of newer MacOS versions will need to use `nano ~/.zshrc`. This will open the profile file in a text editor.

Now, let's add your environment variable. In the editor, add the line below, replacing 'your-api-key-here' with your actual API key without the single quotation marks.

```
export OPENAI_API_KEY='your-api-key-here'
```

Let's save and exit by pressing Ctrl+O to write the changes, then pressing Ctrl+X to close the editor.

Now, you're going to load your profile by using `source ~/.bash_profile` for older Mac OS versions and `source ~/.zshrc` for the newer Mac OS versions. This will load the updated profile.

Finally, we're going to verify that we've done everything correctly. In the terminal, type `echo $OPENAI_API_KEY`. If everything went well, it should display your API key, confirming the setup. `echo $OPENAI_API_KEY`

For Windows

Start by opening the command prompt. You can find it by searching "cmd" in the start menu.

Now, we're going to set the environment variable in the current session by using the command below, replacing 'your-api-key-here' with your actual API key. This command sets the `OPENAI_API_KEY` for the current session. `setx OPENAI_API_KEY 'your-api-key-here'`

You can make the setup permanent by adding the variable through system properties:

- Right-click "This PC" or "My Computer" and select "Properties."
- Click "Advanced system settings."
- Click the "Environment Variables" button.
- In the "System variables" section, click "New..." and enter `OPENAI_API_KEY` as the variable name and your API key as the variable value.

To make sure everything is working properly, reopen the command prompt and type the command below to verify the setup. It should display your API key. `echo %OPENAI_API_KEY%`

For Linux

To set the environment variable for the current session, open a terminal window and use the export command. Replace 'your-api-key-here' with your actual API key.

```
export OPENAI_API_KEY='your-api-key-here'
```

To make the environment variable persistent across sessions, you can add it to your shell's configuration file, such as `~/.bashrc` for Bash. Here's how you can do it: Open the configuration file in a text editor. For example:

```
nano ~/.bashrc
```

Add the following line at the end of the file: `export OPENAI_API_KEY='your-api-key-here'`

Save the file and exit the text editor.

To apply the changes immediately, you can either close and reopen the terminal or run:

```
source ~/.bashrc
```

To verify that the environment variable is set correctly, you can echo its value in the terminal. This command should display your API key: `echo $OPENAI_API_KEY`

Option #2: Creating a .env File

Using a system-wide environment variable is great for making the API key accessible by any application or script running on the machine. However, if your use case is a little more simplistic, we can simply create a local variable accessible within just the scope of a particular program or script. It's also useful for situations in which different projects necessitate different keys, so you can prevent conflicts in key usage. Let's dive right in!

We're going to start by creating a local `.env` file. This file will house your API key, ensuring it's only utilized by the designated project. Navigate to the project folder where you intend to create the `.env` file.

Note to prevent your `.env` file from being unintentionally shared via version control, create a `.gitignore` file in your project's root directory. add a line with `.env` to ensure the confidentiality of your api key and other sensitive information.

Next, use the terminal or an IDE to create both the `.gitignore` and `.env` files. Copy your API key, and replace 'your-api-key-here' with your actual API key without the single quotation marks.

At this point, your `.env` file should look like this:

```
OPENAI_API_KEY='your-api-key-here'
```

Finally, you can import the API key into your Python code using the following snippet.

Listing 2-1. Importing Your `.env` File into Your Python Application

```
import os
from dotenv
import
load_dotenv
from openai
import
OpenAI
# Load environment variables from
.env load_dotenv()
```

```
# Use the API key from the
environment variable api_key
=
```

```
os.getenv("OPENAI_API_KEY")
client = OpenAI()
```

Option #3: Hard-Coding the API Key Directly in Your Application (Take with Caution)

This last method isn't recommended for long-term use because of security reasons. But, for the sake of knowing how it works, we're going to cover how you can hard-code your API key into your application if you want to quickly test out your API key to make sure it's working.

To begin, you'll assign the API key to a variable within the Python code. Replace "YOUR_API_KEY" with the actual API key you received from OpenAI. Ensure that this API key is kept secure and not shared publicly.

Next, you'll initialize the OpenAI client within your Python script. This is done by instantiating the OpenAI class with the `api_key` parameter set to the API key. By providing the API key during initialization, you enable the OpenAI client to access the services offered by the OpenAI API. This step ensures that your Python script can communicate with the OpenAI API using the specified API key.

Listing 2-2. Coding Your API Key Directly into Your Application from `openai` import `OpenAI`

```
# API key
API_KEY = "your-api-key-here"
# Initialize OpenAI client with
hardcoded API key
client = OpenAI(api_key=API_KEY)
```

Now, let's make our first application with the OpenAI API, and test out the key at the same time by getting a list of models useable with the OpenAI API.

Note From this point on, the code examples will be accessing our api key with a local `.env` file.

Creating Your First Python ChatGPT App:

`model_list.py`

We're actually going to accomplish two tasks at once here. We're going to create a basic application in Python using the OpenAI APIs, and in the process, we're going to verify that we've properly obtained an API key. So, needless to say, in case you haven't done so already, follow the instructions in Chapter 1 to create your OpenAI developer account and obtain your API key. Going forward, all the code samples in this book require a valid API key.

Using `OpenAI.models.list()` to Get a List of Available Models

One of the most basic (but also essential) capabilities that we can invoke is in the Model class. Why, you may ask? The Model class allows you to get a list of all the AI models that are currently available for use by developers via the Python API.

After successfully getting an instance of the OpenAI and Model objects, the API will provide what is referred to within the OpenAI API as a “SyncPage,” which is essentially a dictionary with the structure shown in Table 2-1.

Handling the Response

Table 2-1. *The Structure of the Model Object*

Field	Type	Description
object data	string	this always returns
↳ id	Dictionary	the literal “list.” a
↳ object	string	dictionary of ai
↳ created	string	models offered by
	integer	openai.
↳ owned_by	string	the unique iD of the ai model, which is essentially the full name of the model.
		this always returns the literal “model.” the creation date for the model. the name of the organization that owns the model.

Note since objects can contain dictionaries (which can be hard to represent in a table), we’re using the following notation “↳” to indicate the elements of the dictionary. as you can see from table 2-1, “id,” “object,” “created,” and “owned_by” are all elements of the “data” dictionary in the response.

Now that we have the details of the Model object, let’s talk about how we can test the API key we obtained in the first chapter. There are actually a few ways to do this.

Using Your API Key to Get a List of Available Models with the OpenAI API

With our API key set up in a local .env file, we’re going to use the following code to get a list of models available within the OpenAI API, which will then be printed into our terminal.

Listing 2-3. Getting a List of Models Available with the OpenAI API by Calling the models.list() method with model_list.py

```
"""Module for interacting with OpenAI API to list the models available"""
```

```
import os from
dotenv import
load_dotenv
```

```

from openai
import OpenAI
# Load environment variables from .env
load_dotenv()
# Use the API key from the
environment variable api_key =
os.getenv("OPENAI_API_KEY")
client = OpenAI() models_list =
client.models.list()
print(models_list)

```

After running the code in Listing 2-3, Listing 2-4 is a truncated response that you should expect to see.

Listing 2-4. An Ugly-Looking Response After Running Our model_lister.py Application

```

SyncPage[Model](data=[Model(id='dall-e-3', created=1698785189,
object='model', owned_by='system'), Model(id='dall-e-2',
created=1698798177, object='model', owned_by='system'),
Model(id='gpt-3.5turbo- 0125', created=1706048358, object='model',
owned_by='system'), Model(id='text-embedding-ada-002',
created=1671217299, object='model', owned_by='openai-internal'),
Model(id='tts-1-hd-1106', created=1699053533, object='model',
owned_by='system'), Model(id='tts-1-hd', created=1699046015,
object='model', owned_by='system')]. . .

```

Getting a Prettier List of Models

Listing 2-4 is a partial list due to the sheer size of the number of models available for developers to use! The good news, however, is that the full response is provided as a table in Appendix 1.

So, first off, this is great. We know that our API key works! However, if you hadn't noticed, we're getting a fairly ugly-looking response. For the sake of brevity, Listing 2-4 has been shortened, but it's quite obvious no one is going to want to read that big ol' chunk of text when we just want to know all of the models within the OpenAI API.

It would be nice if we could get a more cleaned up response after running our code.

So, let's add some code that will format the response.

Listing 2-5. Cleaning Up the Response We Get from the OpenAI API with model_lister_pretty.py

```

"""Module for interacting with OpenAI API to list the models
available"""
import os
from
dotenv
import
load_doten
v from

```

```

openai
import
OpenAI
# Load environment variables from
.env load_dotenv()
# Use the API key from the
environment variable api_key
=
os.getenv("OPENAI_API_K
EY") client = OpenAI()
models_list =
client.models.list()
# Iterate through the list and print
model information for model in
models_list.data:
    print(f"Model ID: {model.id}")    print(f"Created:
{model.created}")    print(f"Object:    {model.object}")
    print(f"Owned    By:    {model.owned_by}")
print("\n=====\\n")    #
Separator for better
                                readability

```

Listing 2-6 has the response from running model_lister_pretty.py.

Listing 2-6. The Cleaned Up Response from OpenAI with model_lister_pretty.py

Model ID: text-embedding-ada-002

Created: 1671217299

Object: model

Owned By: openai-internal

Model ID: gpt-3.5-turbo-instruct

Created: 1692901427

Object: model

Owned By: system

Model ID: dall-e-3

Created:

169878518

9 Object:

model

Owned By: system

Wait, How Many Tokens Are in My Prompt?

At a certain point, you're going to start thinking about the prompts that you plan to send to ChatGPT and give considerable thought to the token limitations (and the costs) regarding the model that you want to use. In case you forgot, be sure to refer back to Table 1-1 for a list of models and the price of the tokens. Additionally, OpenAI created a simple-to-use

website that allows you to see how many tokens are in your prompt, as shown in the figure below.

ChatGPT Token Counter

<https://platform.openai.com/tokenizer>

Figure 2-1. *The ChatGPT Tokenizer Can Give You a Quick Count of the Tokens in Your Prompt*

Using ChatGPT As a Pair-Programmer to Build a Weather Application

Now that we've made sure that our API key works, it's time to start writing our first project with ChatGPT. This is the exciting part! We're going to use ChatGPT to help us create an application in Python that will give us the weather and the time it'll take us to get to work from the location of our home to work. Sounds impressive, doesn't it?

Setting Up the Prompt

Let's ask ChatGPT how we should get started. Now is a good time to practice prompt engineering. We're going to explain to ChatGPT what we want and see what it comes up with. This is the prompt we used.

Listing 2-7. Prompting ChatGPT to Help Us Make a Weather App For Productivity

How can I make a simple application in Python that allows me to type in my city and then use AccuWeather's API to give me the weather?

This is the code we got in the response.

Listing 2-8. Getting a Response Back from ChatGPT to Build Our Weather App, `accuweather_forecaster.py`

To create a simple application in Python that allows you to type in your city and retrieve the weather using AccuWeather's API, you can follow these steps:

- Sign up for an AccuWeather API key on their developer portal: <https://developer.accuweather.com/>.
- Install the requests library if you haven't already. You can install it via pip:
pip install requests
- Write a Python script to interact with the AccuWeather API. Here's a basic example:
import requests

```
def get_weather(accuweather_api_key, city):
    # Set up the parameters for the location request
    params = {
        "apikey": accuweather_api_key,
        "q": city,
    }

    # Make a request to get location data
```

```

                                location_url      =
"http://dataservice.accuweather.com/locations/v1/cities/search"
    response = requests.get(location_url, params=params)

    if response.status_code == 200:
        location_data = response.json()

        # Check if the location data is available
        if location_data:
            location_key = location_data[0]["Key"]

                                weather_url      =
f"http://dataservice.accuweather.com/currentconditions/v1/{location_key}"
}

        # Request weather data for the location
        response = requests.get(weather_url, params={"apikey":
accuweather_api_key})

        if response.status_code == 200:
            weather_data = response.json()

            # Check if the weather data is available
            if weather_data:
                weather_text = weather_data[0]["WeatherText"]

                                temperature      =
weather_data[0]["Temperature"]["Metric"]["Value"]
                print(f"The current weather in {city} is {weather_text}
with a temperature of {temperature} °C.")
            else:
                print("Weather data not found.")
            else:
                print("Failed to retrieve weather data.")
            else:
                print("Location not found.")
            else:
                print("Failed to retrieve location data.")

    if __name__ == "__main__":
        accuweather_api_key = "your-api-key" # Replace with your
actual API key
        city = input("Enter your city: ")
        get_weather(accuweather_api_key, city)

```

Creating accuweather_forecaster.py with the AccuWeather API

So, let's follow the steps we got from ChatGPT. First, we need to set up an account with AccuWeather. Go to <https://developer.accuweather.com/> and hit register to sign up.

Figure 2-2. AccuWeather Home Page for Developers

After signing in, you'll notice that the menu changes, and you can now navigate over to **My Apps**.

Figure 2-3. Navigating to the My Apps Tab on the AccuWeather Developer Portal

Once you're on the **My Apps** tab, you're going to want to **Add a new App** in order to get an API key to use in our application.

Figure 2-4. Creating a New App with AccuWeather

Figure 2-5. Adding Specifications for Our AccuWeather App

As you set things up, you'll need to name your app and answer benign questions like where the API will be used and what you plan to be creating with the API. As you can see in Figure 2-5, we've called our AccuWeather app "Weather Tracker"; however, this is not the name of our actual Python application, `accuweather_forecaster.py`.

The names don't have to match. You can name your app in AccuWeather whatever you like. The most important configuration to enable here is where you're being asked to specify the product you're intending to use. **Be sure to enable the Core Weather Limited Trial.** The difference between choosing Core Weather and MinuteCast is an application that works and an application that doesn't and takes a long time to figure out why it doesn't (can you tell this was a memorable experience?).

It may take some time for your application to be approved, but usually, this is a very quick process. When it's done, you'll see your new application on the **My Apps** page, which will include your API key! Mission accomplished.

Figure 2-6. Seeing Your Newly Created App on the AccuWeather Developer Portal

Refining Our Prompt with Prompt Engineering

Back to our code example provided by ChatGPT, we can provide our API key where it's required in the code. For clarity's sake, we've renamed the API key variable to "`accuweather_api_key`." If you need a refresher, this is the line of code where we need to input our API key from AccuWeather.

Listing 2-9. Inputting Our API Key from AccuWeather into Our `accuweather_forecaster.py` app if `__name__ == "__main__"`:

```
accuweather_api_key =
"your-accuweather-api-key-here"    city =
input("Enter your city: ")
get_weather(accuweather_api_key, city)
```

Now, with everything all together, when you run the code, you should see a prompt that asks you to input the name of a city.

Listing 2-10. After Running the Script, Enter a Valid City Name Enter your city:

Once you provide a valid city name, you should find that everything runs smoothly and, you'll get a response that looks something like this

(obviously, the forecast will change depending on what city you decide to enter and, well, what the weather is like the day you run it).

Listing 2-11. It Was a Little Chilly in New York City on the Day We Ran the `accuweather_forecaster.py` Script

The current weather in New York City is Sunny with a temperature of 5.6°C.

We get a nice, basic description of the weather in New York City from running the code, but there are a couple of things we can improve on.

AccuWeather has icons for various weather conditions. Right now, we can see that the weather in New York is currently sunny, but we could also get a longer description that describes the weather for the rest of the day. Americans might also prefer to see the temperature in Fahrenheit (as American authors, that's what we're going to choose as well). So, let's get ChatGPT to help us make those changes.

So that ChatGPT can retain the context, we're going to prompt ChatGPT in the same Chat conversation as follows.

Listing 2-12. Prompting ChatGPT to Help Change the Code in Our `accuweather_forecaster.py` Application to Get a More Detailed Response from the AccuWeather API

This code works, however, I'd like to make the following changes:

- get the temperature in Fahrenheit
- make an API call to get an icon number and an icon url to print in the response
- make an API call to get a weather description in the response

This is the answer we got from ChatGPT.

Listing 2-13. Response from ChatGPT to Modify the Code in Our `accuweather_forecaster.py` Application

Sure, here's the modified version of the code to accommodate your requirements: ```python
import requests

```
def get_weather(accuweather_api_key, city):
    # Base URL for location search
    base_url = "http://dataservice.accuweather.com/locations/v1/cities/search"
    params = {
        "apikey": accuweather_api_key,
        "q": city,
    }

    # Request location data
    response = requests.get(base_url, params=params)

    # Check if the location data is valid
    if response.status_code == 200:
        location_data = response.json()
```

```

if location_data:
    location_key = location_data[0]["Key"]

    weather_url =
f"http://dataservice.accuweather.com/currentconditions/v1/{location_key}"
}

    # Request weather data for the location
    response = requests.get(weather_url, params={"apikey":
accuweather_api_key})

    if response.status_code == 200:
        weather_data = response.json()

        if weather_data:
            # Extract weather information
            weather_text = weather_data[0]["WeatherText"]
            temperature_imperial =
weather_data[0]["Temperature"]["Imperial"]["Value"]

            # Fetch weather icon
            icon_number = weather_data[0]["WeatherIcon"]
            icon_url =
f"http://developer.accuweather.com/sites/default/files/{icon_number:02d}
-s.png"

            # Request weather headline
            headline_url =
f"http://dataservice.accuweather.com/forecasts/v1/daily/1day/{location_k
ey}"

            response = requests.get(headline_url, params={"apikey":
accuweather_api_key})

            if response.status_code == 200:
                headline_data = response.json()

                if headline_data and "Headline" in headline_data:
                    headline_text = headline_data["Headline"]["Text"]
                    print(f"The current weather in {city} is {weather_text}
with a temperature of {temperature_imperial}°F.")
                    print(f"Weather Headline: {headline_text}")
                    print(f"Weather Icon: {icon_url}")
                else:
                    print("Weather headline not found.")
            else:

```

```

        print("Failed to retrieve weather
headline.")
    else:
        print("Weather data not found.")
    else:
        print("Failed to retrieve weather data.")
    else:
        print("Location not found.")
    else:
        print("Failed to retrieve location data.")

if __name__ == "__main__":
    accuweather_api_key =
"your-accuweather-api-key-here" # Replace with your
actual API key
    city = input("Enter your city: ")
    get_weather(accuweather_api_key, city)

```

This modified version of the code:

- Converts temperature from Celsius to Fahrenheit.
- Retrieves weather icon number and description.
- Prints the weather icon number and constructs the icon URL.
- Make sure to replace "your-accuweather-api-key-here" with your actual AccuWeather API key.

When we run this code, we get exactly what we were looking for. We'll enter a valid city name again (New York City, in our case), and this is the response we got.

Listing 2-14. Response from the AccuWeather API After Running the Modified Code in Our accuweather_forecaster API Key

Weather in New York City: Temperature: 45.0°F Weather Text: Sunny
Weather Headline: Expect rainy weather late tomorrow night through late Thursday night

Icon	Number:	1
------	---------	---

Icon	URL:
------	------

<http://developer.accuweather.com/sites/default/files/01-s.png>

Now, our code works exactly the way we want it to, but there's even more room for improvement. If you're using an IDE with any kind of linting, you might see quite a few linting errors from the code we're running now. These are some of the linting errors you might see with the code as it is now.

Listing 2-15. The Linting Errors Pointed Out by Our IDE

Redefining name 'city' from outer scope (line 45) Redefining name 'city' from outer scope (line 45) Missing timeout argument for method 'requests.get' can cause your program to hang indefinitely
Missing timeout argument for method 'requests.get' can cause your program to hang indefinitely Missing timeout argument for method 'requests.get' can cause your program to hang indefinitely Missing

module docstring Missing function or method docstring Line too long (102/100) Line too long (103/100) Line too long (121/100) Constant name "accuweather_api_key" doesn't conform to UPPER_CASE naming style

Note so, what exactly is linting? Linting is the process of analyzing code for

errors, but it's mostly for stylistic inconsistencies. it helps maintain code quality, enforces coding standards, and helps make code more readable by using common conventions.

To clean this up, we're just going to run this by ChatGPT, again, in the same Chat conversation.

Listing 2-16. Prompting ChatGPT to Help Us Resolve Our Linting Errors

My IDE is making me aware of the following linting errors. Help me modify my code to comply to the styling standards.

###

Redefining name 'city' from outer scope (line 45)

Redefining name 'city' from outer scope (line 45)

Missing timeout argument for method 'requests.get' can cause your program to hang indefinitely

Missing timeout argument for method 'requests.get' can cause your program to hang indefinitely

Missing timeout argument for method 'requests.get' can cause your program to hang indefinitely

Missing module docstring
Missing function or method docstring Line too long (102/100) Line too long (103/100) Line too long (121/100) Constant name "accuweather_api_key" doesn't conform to UPPER_CASE naming style

This was the response we got from ChatGPT.

Listing 2-17. Getting a Response from ChatGPT That Resolves All of Our Linting Errors in `accuweather_forecaster_cleaned_up.py`

To address the linting errors and improve the code's compliance with styling standards, you can make the following modifications:

- Add module docstring to describe the purpose of the script.
- Add function docstring to describe the purpose of the `'get_weather'` function.
- Rename the `'accuweather_api_key'` variable to `'ACCUWEATHER_API_KEY'` to comply with the UPPER_CASE naming convention for constants.
- Remove redefining `'city'` from the outer scope by renaming the input parameter of the `'get_weather'` function.
- Add a timeout argument to the `'requests.get'` calls to prevent hanging indefinitely.

- Split long lines to comply with the maximum line length. Here's the modified code:

```
```python
"""
```

Script to fetch current weather information using AccuWeather API.

```
import requests def get_weather(accuweather_api_key, city):
```

```
 """
```

Get current weather information for a given city using AccuWeather API.

Args: accuweather\_api\_key (str):

AccuWeather API key.

city (str): Name of the city for which weather information is requested.

Returns:

None

```
 """
```

```
base_url = "http://dataservice.accuweather.com/locations/v1/cities/search"
```

```
 params = {
 "apikey": accuweather_api_key,
 "q": city,
 }
```

```
 response = requests.get(base_url, params=params,
 timeout=10)
 location_data = response.json()
```

```
 if location_data:
```

```
 location_key = location_data[0]["Key"]
```

```
weather_url = f"http://dataservice.accuweather.com/currentconditions/v1/{location_key}"
```

```
 params = {
 "apikey": accuweather_api_key,
 }
```

```
 response = requests.get(weather_url, params=params,
 timeout=10)
 weather_data = response.json()
 if weather_data:
```

```
 weather_text = weather_data[0]["WeatherText"]
```

```
temperature_imperial = weather_data[0]["Temperature"]
["Imperial"]["Value"]
```

```
Fetching icon number and icon URL
```

```
icon_number = weather_data[0]["WeatherIcon"]
```

```
icon_url = (f"http://developer.accuweather.com/sites/default/files/"
 f"{icon_number:02d}-s.png")
```

```
 # Making API call to get weather headline
```

```
 headline_url = (
```

```
f"http://dataservice.accuweather.com/forecasts/v1/daily/1day/"
```

```
 f"{location_key}"
```

```
)
```

```

 response = requests.get(headline_url, params=params,
 timeout=10)
 headline_data = response.json()
 if headline_data and "Headline" in
headline_data:
 headline_text =
headline_data["Headline"]["Text"]
 print(f"The
current weather in {city} is {weather_text} "
f"with a temperature of {temperature_imperial}°F.")
 print(f"Weather Headline: {headline_text}")
 print(f"Weather Icon: {icon_url}")
 else:
 print("Weather headline not
found.")
 else:
 print("Weather data not
found.")
 else:
 print("Location not found.")
if __name__ == "__main__":
 ACCUWEATHER_API_KEY =
"your-accuweather-api-key"
 user_city = input("Enter your city: ")

```

```

get_weather(ACCUWEATHER_API_KEY,
user_city) ``

```

These modifications should address the linting errors and improve the code's readability and compliance with styling standards. Be sure to replace "your-accuweather-apikey-here" with your actual API key.

Using ChatGPT As a Pair-Programmer to Build an Application That Estimates Distance and Arrival Time

The next thing we're going to want to do is start making an application that can give us an arrival time and distance from one place to another, for example, from our house to our workplace. We're going to use a classic here and dive into the Google Maps API.

## Creating a Project with Google Maps Platform API

Most people already have a Gmail account, but on the off chance you don't, be sure to create one before proceeding.

Google has a mountain of APIs for various applications, so we're going to need to go directly to the API for Google Maps, in specific, at <https://developers.google.com/maps/documentation>.

**Figure 2-7.** The Google Maps Platform Home Page

On the Google Maps Platform page, click **Getting Started** to set up your account for using the APIs. After following the steps Google presents you with, you'll be taken to the page on Figure 2-8, where you can see the different APIs available with the Google Maps Platform. But what'll probably catch your attention first is the fact that you still have to

### **Finish Account Setup.**

#### **Figure 2-8. Finishing Your Google Maps Platform Account Setup**

Finishing your account setup will compose of entering credit card information so you can start the free trial that will allow you \$200 worth of credit, which is more than enough for the purposes of our testing.

#### **Figure 2-9. Navigating to the APIs & Services Tab on the Google Maps Platform**

After properly setting up your account, you'll be greeted by a welcome page. On the left, you'll find a menu icon you can click on to reveal a list of services you have access to.

You want to navigate to **Apps & Services** and then click **Library**.

#### **Figure 2-10. The API Library Page**

You're going to want to click on **Maps JavaScript API** and then **Enable** it.

#### **Figure 2-11. Enabling the JavaScript API**

After enabling the JavaScript API, go back twice and open the menu again, and then click on the **Google Maps Platform** to see the dashboard for Google Maps.

From here, we're going to be looking at another side menu that looks similar to the one from before, but here we're going to click on **Apps & Services** again to see a different page this time. From here, you can click on **Routes** to enable it.

#### **Figure 2-12. Enabling the Routes API**

Once you've enabled the API we need, navigate to the **Keys & Credentials** tab and **Create a new API key**.

#### **Figure 2-13. The Keys and Credentials Page on the Google Maps Platform**

Now that we have our Google Maps API key, we can make use of our pair programmer. There are actually two ways we can use ChatGPT to make it easier to run code that successfully retrieves the information we want: how far away is the destination and how long will it take to get there (and here's a fun fact – the Google Maps Routes API factors in traffic trends in the estimated travel time, making our application perfect for productivity!).

### **Using ChatGPT to Take a cURL Command and Make It Useable in Python: The Flexible Programmer's Approach**

To some, using cURL can be a bit intimidating, which is why we've laid out two methods for using ChatGPT to help us write code that makes use of the Google Maps Platform API; one way that takes cURL code and "translates" it to something we can use in Python and another way that surpasses the need to look at any scary cURL code at all.

In this first method, instead of navigating the Google Maps Platform documentation, we're actually going to cut to the chase and give you the cURL code you need and show you how to use ChatGPT to make it useable to you as a Python programmer. You're welcome.

Here's the cURL code from the Google Maps Platform documentation.



**Listing 2-18.** cURL Code for Using the Google Maps Routes

```
API curl -X POST -d '{ "origin":{
 "location":{
 "latLng":{
 "latitude": 37.419734,
 "longitude": -122.0827784
 }
 }
},
"destination":{
 "location":{
 "latLng":{
 "latitude": 37.417670,
 "longitude": -122.079595
 }
 }
},
"travelMode": "DRIVE",
"routingPreference": "TRAFFIC_AWARE",
"departureTime": "2023-10-15T15:01:23.045123456Z",
"computeAlternativeRoutes": false,
"routeModifiers": {
 "avoidTolls": false,
 "avoidHighways": false,
 "avoidFerries": false
},
"languageCode": "en-US",
"units": "IMPERIAL"
}' \
-H 'Content-Type: application/json' -H 'X-Goog-API-Key:
YOUR_API_KEY' \
-H 'X-Goog-FieldMask:
routes.duration,routes.distanceMeters,routes.polyline.encodedPolyline' \
'https://routes.googleapis.com/directions/v2:computeRoutes'
```

Remember, when practicing prompt engineering, the best practice is to separate the instructions from the content with three hashes (###) to get the best results. To that end, here's the prompt we gave ChatGPT so that we could use this cURL code and produce the results we desired using three hashes to separate the instructions from the content, the content being the cURL code presented in Listing 2-18:

**Listing 2-19.** Prompting ChatGPT to Provide Python Code Using the cURL Command As Context

I'm coding in Python and trying to build an application that makes a request to the Google Maps API and pulls down the amount of time it'll take to get from one location to the other. This is the cURL code that would accomplish that goal. Help me convert this

to something I can use in Python. ### [cURL code from listing 218]

This is the code we got in the response from ChatGPT.

**Listing 2-20.** Getting a Response Back from ChatGPT with Useable Python Code

```
import requests import json
Define the payload
payload= {
 "origin": {
 "location": {
 "latLng": {
 "latitude": 37.419734,
 "longitude": -122.0827784
 }
 }
 },
 "destination": {
 "location": {
 "latLng": {
 "latitude": 37.417670,
 "longitude": -122.079595
 }
 }
 },
 "travelMode": "DRIVE",
 "routingPreference": "TRAFFIC_AWARE",
 "departureTime": "2025-02-17T17:00:00Z",
 "computeAlternativeRoutes": False,
 "routeModifiers": {
 "avoidTolls": False,
 "avoidHighways": False,
 "avoidFerries": False
 },
 "languageCode": "en-US",
 "units": "IMPERIAL"
}
Define headers
headers = {
 "Content-Type": "application/json",
 "X-Goog-API-Key": "YOUR_API_KEY",
 "X-Goog-FieldMask":
"routes.duration,routes.distanceMeters,routes.
polyline.encodedPolyline"
}
```

```
Make the request response =
requests.post("https://routes.googleapis.com/directions/
v2:computeRoutes", json=payload, headers=headers)
Print the response
print(response.json())
```

Note in order for this code to run, you must set a date and time in the “DepartUre-time” parameter that is beyond the time at which you run the code. in other words, the departure time cannot be set in the past. in this example, the date is set to a specific time in 2025, but if you’re running this code beyond that year, you need to adjust the code in the “DepartUre-time” parameter in order for your code to run as expected. After running this code, this is the response you should get.

**Listing 2-21.** The Successful Response After Running the Code from ChatGPT

```
{'routes': [{'distanceMeters': 773, 'duration': '158s', 'polyline':
{'encodedPolyline': 'ipkcFjichVzQ@d@gU{E?'}]}}
```

Using ChatGPT to Avoid Reading a Lot of Documentation and Having to Decipher Through Any cURL Code

As mentioned before, there is a *lot* of documentation to go through in the Google Maps

Platform. If you even took a cursory glance at the other APIs we passed by to get to the Routes API, in specific, you would have seen different APIs for pulling vast varieties of data; air quality, pollen, aerial view (like you’d see when you use Google Maps), and many other features of Google Maps you may normally take for granted all have individual APIs. Do you know what geocoding is? Well, you’d have to find out the hard way if you went through the documentation on your own.

Reading through documentation is annoying. Especially when we only want a few features in any given API or service. In a perfect world, we wouldn’t even *need* to know what geocoding is in order to get our code working in different scenarios other than what’s provided in the example code from Google.

We may not live in a perfect world, but we *don’t* have to know what geocoding is, at least not anymore. We can simply use ChatGPT to get through the documentation we don’t want to read.

Here’s what we asked ChatGPT so you don’t have to read any of the Google Maps Route API to get working code.

**Listing 2-22.** Prompting ChatGPT to Avoid Reading Documentation

I need to use the Google Maps Platform's Routes API gives me to pull down information pertaining to how much time it'll take to get from one destination to the other, as well as how much distance is between the two points. How can I make an HTTP request in Python to get the information I'm looking for?

This is the code we got in our response from ChatGPT.

**Listing 2-23.** Getting a Response Back from ChatGPT to Avoid Reading

## Documentation

```
import requests
def get_route_info(google_maps_api_key, origin, destination):
 url = f"https://maps.googleapis.com/maps/api/directions/json?origin={origin}&destination={destination}&key={google_maps_api_key}"
 response = requests.get(url)
 if response.status_code == 200:
 route_data = response.json()
 if route_data["status"] == "OK":
 route_info = route_data["routes"][0]["legs"][0]
 distance = route_info["distance"]["text"]
 duration = route_info["duration"]["text"]
 print(f'Distance: {distance}, Duration: {duration}')
 else:
 print("Error:", route_data["status"])
 else:
 print("Error:", response.status_code)
Replace 'YOUR_GOOGLE_MAPS_API_KEY' with your actual Google Maps API key
google_maps_api_key = 'YOUR_GOOGLE_MAPS_API_KEY'
origin = 'New York, NY'
Example origin destination = 'Los Angeles, CA'
Example destination
get_route_info(google_maps_api_key, origin, destination)
If you run this code, you should get this very clean, very simple response: Distance: 2,789 mi, Duration: 1 day 17 hours
```

## Conclusion

Let's debunk a common myth and set the record straight: ChatGPT isn't some mindreading wizard! As an AI tool, it's not here to replace the expertise of developers and architects – it's here to complement it and save a lot of time and hassle in the day-to-day frustrations the average programmer faces on a regular bases. Sure, it can help translate your natural language requests into code, but let's not forget the crucial role of human judgment. Ultimately, it's up to developers to decide whether to use, tweak, or toss out the resulting code.

## CREATING A BASIC CHATGPT CLIENT IN PYTHON

The purpose of this chapter is plain and simple. We're going to build the most powerful ChatGPT client available using only a few lines of code in python. This client will do a lot more than what you're able to do using the ChatGPT website and will provide you more options than what is available using the Chat Playground that we saw in Chapter 1.

Creating Our ChatGPT Chat Completion Application,

chatgpt\_client.py

Listing 3-1 is the code for our ChatGPT client, chatgpt\_client.py.

**Listing 3-1.** chatgpt\_client.py

```
"""
```

Script to demonstrate API call using OpenAI's GPT-4 for chat completions.

```
"""
```

```
from dotenv import load_dotenv from openai import OpenAI
Load environment variables from .env load_dotenv()
instantiate the OpenAI object client = OpenAI()
response = client.chat.completions.create(model="gpt-4o",
messages=[
 {
 "role": "system",
 "content": "You are a Python developer"
 },
 {
 "role": "user",
 "content": "Why is Python typically used for data
science?"
 }
],
temperature=0.85, max_tokens=1921, top_p=1,
frequency_penalty=0,
presence_penalty=0
)
print(response)
```

As you analyze the code in Listing 3-1, you're going to see several things that are quite familiar from the Chat Playground such as the model, messages, temperature, and tokens.

Note In this chapter, we'll use the Python datatypes, so you'll see alist where the official OpenAPI documentation specifies an Array.  
Using OpenAI.chat.completions.create() to Send Messages to ChatGPT  
The OpenAI.chat.completions.create() method is a method that's basically a one-toone representation of what you can do in the Chat Playground; therefore, this method should feel like second nature to you.

Table 3-1 describes the format of the parameter necessary to call the OpenAI.chat.completions.create() method. Although the table is lengthy, after a quick glance, you should see that only a few fields are actually required in order to successfully invoke the method.  
The response to the method is called a ChatCompletion.

**Examining the Method Parameters**

***Table 3-1. The Structure of the Create ChatCompletion Object***

Field	Type	Required ?	Description
model	String	Required	The ID of the model you want to use for the ChatCompletion. Compatible models include gpt-4 gpt-4-0613 gpt-4-32k gpt-4-32k-0613 gpt-4o gpt-4o-mini o1 o1-mini There are four types of messages, each with their own requirements: System message (see Table 3-2) User message (see Table 3-3) Assistant message (see Table 3-4) Tool message (see Table 3-5)
messages	List	Required	

Default: false  
~~By specifying a seed, the system will make an attempt to generate repeatable results.~~  
In theory, this means that if you make repeated requests with the same seed and parameters, you should expect to receive the same result.  
In order to get the seed value to put in the subsequent request, copy the system\_ fingerprint from your last response.

You can provide up to four sequences where the API should stop generating further tokens.

This can be useful for controlling the length or content of responses.

If "stream" is set to "true," partial message updates will be sent as server-sent events.

This means tokens will be sent as data-only events as they become available, and the stream will end with a "data:

[DONE]" message.

temperature	Number or null Default: 1		Valid values range between 0 and 2. Controls the randomness of the model's output. The best practice is to adjust the top_p or temperature, but not both. This parameter controls which
tool_choice	String or JSON object	Optional	(if any) function is called by the model. You have two options: "none" or "auto."

Use "none" if you don't want the model to call a function.

Use "auto" if you want the model to pick between generating a message or calling a function.

Specifying a particular function via {"type": "function", "function": {"name": "my\_function"}} forces the model to call that function.

Please note that "none" is the default when no functions are present and "auto" is the default if functions are present.

Currently, only functions are supported as a tool. Use this to provide a list of functions the model may generate JSON inputs for. represent your end-user.

This will help OpenAI monitor and detect abuse.

OK, Table 3-1 appears to be a little daunting! However, as mentioned earlier, only the model and messages are required parameters.

Additionally, we also have the code in Listing 3-1 above in order to show how the parameters are actually used within a real application.

So, as you can see, as a Python developer, we have several options and parameters available to use that ordinary people can't do using the ChatGPT website or using the Chat Playground.

Now, the one parameter that needs the most detailed explanation is the messages parameter, so let's analyze that further.

**There Are Four Types of Messages**

When invoking the ChatGPT API programmatically, there are four types of messages that you can provide to the API:

- System message
- User message
- Assistant message
- Tool message

The good news is that if you refer back to Chapter 1 where we explained how to use the Chat Playground, you can see that we've already have encountered the first three message types! The only new message type that we're not currently familiar with is the tool message."

**System Message (Dictionary)**

*Table 3-2. The Structure of the System Message*

Field	Type	Required ?	Description
role	String	Required	This must be set to the string, "system"
content	String	Required	These are the instructions that you want the system to perform in the conversation.
name	String	Optional	This is an optional name that you can provide the system.

Listing 3-2 is a snippet from Listing 3-1 that shows how the system message is formatted.

**Listing 3-2.** Formatting the System Message

```
messages=[
 {
 "role": "system",
 "content": "You are a Python developer"
 },
 ...
]
```

**User Message (Dictionary)**

*Table 3-3. The Structure of the User Message*

Field	Type	Required ?	Description
-------	------	------------	-------------



role	Require	This must be set to the string, "user"
String	d	This string contains the actual message or
content	Require	question that you want to send to
String	Optionald	ChatGPT.
name	String	This is an optional name that you can provide for yourself in the conversation.

Listing 3-3 is a snippet from Listing 3-1 that shows how the user message is formatted.

**Listing 3-3.** Formatting the User Message

```
messages=[
 ...
 {
 "role": "user",
 "content": "Why is Python typically
used for data science?"
 }
 ...
]
```

**Assistant Message (Dictionary)**

Note In case you forgot, the assistant message is used to “remind” ChatgPt what it told you in a previous response. Ideally, this can allow you to continue a conversation that you had with it weeks or months in the past.

**Table 3-4.** The Structure of the Assistant Message

Field	Type	Required ?	Description
role	String	Required	This must be set to the string, "assistant"
content	String	Required	This string contains the response from ChatGPT from a previous conversation.

			This is an optional name that you can provide for ChatGPT in the conversation. If ChatGPT used a tool in a previous response, then include the tool that it specified here.
			This is the ID of the tool called by ChatGPT.
name	Optional		This is the type of the tool called by ChatGPT. Only the literal, "function," is a valid tool.
tool_calls	Optional		
↳ id	String	Required	This is the function that the model called.
↳ type	List		
↳	String	Required	
function	Object	Required	

Listing 3-4 is a snippet from Listing 3-1 that shows how the assistant message is formatted.

**Listing 3-4.** Formatting the Assistant Message

```
messages=[
 ...
 {
 "role": "assistant",
 "content": "Python is typically used for data science
for several reasons..."
 }
 ...
]
```

**Tool Message (Dictionary)**

A tool message is an advanced type of message for a very specific use case. You can't use them on the ChatGPT website nor in the Chat Playground. Using a tool message and the tool parameter from Table 3-1, you can enable ChatGPT to "call a function" for you. At first glance, you may be thinking, "Wow! ChatGPT will load my code and execute it for me in the Cloud? That's awesome!" Unfortunately, that's not the case.

By providing a function name and the parameters necessary to invoke it, ChatGPT will let you know whether to invoke the function and the parameters to put in the function. You will then need to call the function *yourself* within your Python code.

**Table 3-5.** The Structure of the Tool Message

Field	Type	Required?	Description
role	String	Required	This must be set to the string, "tool"
content	String	d	This string contains the content of
tool_call_id	List	Required	the tool message.
		Optional	This is the ID of the tool call.

Running chatgpt\_client.py

So, after running the code we created in Listing 3-1, we can expect a response that will look something like this in Listing 3-5.

**Listing 3-5.** ChatGPT Explains Why Python Is So Awesome

```
ChatCompletion(id='chatcmpl-9ACnRg1bk54jYeIFbxJ3yDnomQmij',
choices=[Choice(finish_reason='stop', index=0,
message=ChatCompletion Message(content="Python is typically used
for data science for several reasons:\n\n1. **Simplicity and
Readability**: Python's syntax is clear and straightforward, which makes
the code easy to read and write. This simplicity allows data scientists to
quickly understand Python's syntax and start coding faster.\n\n2.
Versatile Library Ecosystem: Python has a wide range of libraries
and tools designed specifically for data science tasks. Libraries like
NumPy and SciPy are great for scientific computation, Pandas is
excellent for data manipulation and analysis, Matplotlib and Seaborn for
data visualization, and libraries like Scikit-learn for machine
learning.\n\n3. **Support for Different Data Formats**: Python supports
a variety of data formats which are commonly used in data science. You
can easily load and handle data in different formats such as CSV, Excel
spreadsheets, SQL databases, and more.\n\n4. **Community Support**:
Python has a large and active community of developers who constantly
contribute to improving the language and its tools. This also means that
when problems arise, it's usually easier to find solutions and examples in
Python than in less popular languages.\n\n5. **Integration Capability**:
Python can easily integrate with other languages like C, C++, Java, and it
can also be run on almost all operating systems. This makes it a
convenient choice for data scientists who need to work with different
software and systems.\n\n6. **Support for Advanced Data Analysis**:
Python supports various types of advanced data analysis, including
machine learning, artificial intelligence, and deep learning, with libraries
like TensorFlow, PyTorch, and Keras. \n\n7. **Ideal for Prototyping**:
Python's simplicity and speed make it ideal for prototyping. Data
scientists can use Python to build a model, see how it works, and then
use Python or another language to build a more permanent version if
necessary.", role='assistant', function_call=None, tool_calls=None),
logprobs=None)], created=1712219497, model='gpt-4-0613',
object='chat.completion', system_fingerprint=None,
usage=CompletionUsage(completion_tokens=368, prompt_tokens=25,
total_tokens=393))
```

So, as we take a brief look at Listing 3-5, we see that a bulk of the response (called a ChatCompletion) is the answer to our question that we posed to ChatGPT in Listing 3-1 at the very beginning of the chapter. However, there's a lot of metadata around our response, and let's analyze the ChatCompletion object in more detail.

Handling the Response (ChatCompletion)

Table 3-6. The Structure of the ChatCompletion Object Response

Field	Type	Description
id	string	A distinct identifier for the ChatCompletion.
choices	list	A list of ChatCompletion options. there can be multiple options in the response if “n” is greater than 1 in table 3-1.
↳ finish_reason	string	every response will include a finish_reason. the possible values for finish_reason are
↳ index	Integer	stop: the API returned complete message or a message terminated by one of the stop sequences provided via the stop parameter.
↳ message	Object	length: the model output was incomplete due to the max_tokens parameter in the request or token limit of the model itself. tool_call: the model called a tool, such as a function.
↳	null	content_filter: the response was terminated due to a violation of the content filters.
logprobs	string	null: the API response still in progress or incomplete. the index of the choice in the list of choices.
model	string	A ChatCompletionMessage generated by the model. this is explained in further detail in table 3-7. log probability information for the choice. the model used for the ChatCompletion.
system_fingerprint	string	Use this parameter as the “seed” in a subsequent request if you want to reproduce results in from a previous conversation.
object		this always returns the literal, or “chat.completion.”

(continued)

Table 3-6. (continued)

Field	Type	Description
-------	------	-------------

usage	Object	Usage statistics for the completion request.
↳ completion_tokens	Integer	number of tokens in the generated completion.
↳ prompt_tokens	Integer	number of tokens in the prompt.
↳ total_tokens	Integer	the total count of tokens utilized in the request, including both the prompt and the completion.

The most important item in the `ChatCompletion` object is the `ChatCompletionMessage` which is explained in more detail in Table 3-7.

**ChatCompletionMessage**

*Table 3-7. The Structure of the ChatCompletionMessage*

Field	Type	Description
role	String	This will always be the literal, "assistant."
content	String or null	This is a string that contains the response from ChatGPT to our request.
tool_calls	List	If you indicated in Table 3-1 that you want ChatGPT to call a tool (which is currently a function), then this list will exist in the <code>ChatCompletionMessage</code> .
↳ id	String	This is the ID of the tool called by ChatGPT.
↳ type	String	This is the type of the tool called by ChatGPT. Only the literal, "function," is a valid tool.
↳ function	Object	This is the function and the parameters that the model called.

**Conclusion**

In this chapter, we took our experiences from Chapters 1 and 2 and created a fully functioning ChatGPT client in Python. In the code for our ChatGPT client, we saw some terms that we were already introduced to from the Chat Playground such as the model, messages, temperature, and tokens.

We also saw that, as Python developers, OpenAI gives us a **ton of additional options** to invoke ChatGPT that aren't available to average everyday users or even to technical people who use the Chat Playground. In this chapter, we took the time to explain these options, with a focus on the messages that we can send.

Now that we have a working ChatGPT client in Python, let's see how to leverage it for the rest of the examples in the book!

## USING AI IN THE ENTERPRISE! CREATING A TEXT SUMMARIZER FOR SLACK MESSAGES

In today's corporate world, it's extremely common for companies to have an instance of Slack (or Microsoft Teams) to organize themselves and use it as a central place of communication to everyone in the company. Now, if you've ever used Slack before, I think you know how easily a channel can become flooded with a ton of messages because **some** important thing happened **somewhere** in the company or

the world. Of course, the more responsibility that you have within the company (i.e., manager, team leader, architect, etc.), the more channels you're expected to participate in. In my opinion, Slack is a double-edged sword. You need to use it to do your job, but as a developer, you definitely can't attend a daily standup meeting and say, "Yesterday, uh, I spent all day reading Slack. No roadblocks."

Additionally, if you work for a company with clients in various time zones (which is quite common nowadays), it's pretty daunting to open Slack in the morning and see a ton of messages that were posted while you were away from the keyboard. So, in this chapter, we're going to apply AI in the enterprise to make Slack more useful. We'll leverage the code in the previous chapter and create a Slack bot in Python that will summarize the important conversations in a Slack channel. We're going to be utilizing ChatGPT's capabilities for text summarization and focus a bit more on **prompt engineering**.

**Using ai in the enterprise! Creating a text sUmmarizer for slaCk messages**

### **So, What Is Prompt Engineering?**

Simply stated, prompt engineering is the process of carefully crafting and refining prompts and input parameters to instruct and guide the behavior of ChatGPT and other AI models. It's basically the industry-wide term for creating the right input in order to get the result that you're looking for.

### **ChatGPT Is Here to Take Away Everyone's Jobs (Not Really)**

It is my humble opinion that every company in the world is sitting on a gold mine of untapped information. If you are using any system that keeps a log of exchanges between employees, a database of support requests from your customers, or any large repository of text (yes this includes your email, Microsoft Exchange, and corporate Gmail), then, you have a large repository of unstructured text that is waiting to be utilized.

Therefore, the best use of ChatGPT is not to eliminate anyone's jobs. It should be used in order to augment and extend what team members in your company are already doing. As we saw in the previous chapter, as a programmer, ChatGPT can work as a very effective pair-programmer. It

is also very good at performing certain difficult tasks very efficiently and quickly. Therefore, the project of this chapter involves tackling a practical example of what can be done in order to make useful a large source of unstructured text.

You can use the ChatGPT client you created in Chapter 3 for the prompt engineering examples listed further on in this chapter, or you can use the playground mode feature we talked about in Chapter 1. Either way, let's dive right in.

### **Examining a Real-World Problem: Customer Support for a Software Company**

Let's look at one of the most grueling tasks in software development: providing tech support. Imagine the joys of fielding calls and messages all day from people who might be frustrated, confused, or just in need of a solution while using your software. Here's some of the reasons why customer support is a tough nut to crack:

- Your end-users and your customers are notoriously bad at explaining problems with your software.
- Level 1 technicians, often the first line of defense, typically handle the most basic issues or user errors. But when problems get more complex, users are escalated to Level 2.
- The mid-tier is a tricky place, because they have more knowledge and experience than the tech support staff at Level 1; however, they don't have the opportunity to directly get answers from the end-user.
- Really bad problems get escalated to Level 3; however, these are the most expensive tech support staff because they have the most knowledge and experience. They have hands-on experience with the code as well as the servers and the infrastructure.

So, let's work with a real-world example of a typical conversation within a typical tech support channel within Slack. Below is a list of the team members and their roles within a fictional company:

- Fatima (customer service representative)
- John (software engineer)
- Dave (PM)
- Keith (CTO)

The listing below provides an example of a conversation between the team members at a software startup. Fatima, the customer service representative, lets the team know that their app is crashing immediately after launching (not a good problem to have). Keith, the CTO, steps in immediately to escalate the issue.

#### **Listing 4-1.** Team Members Within a Slack Channel Trying to Analyze a Customer's Problem

Fatima [16:00 | 02/08/2019]: Hey everyone, I have an urgent issue to discuss. I just got off a call with a client who's experiencing app crashes as soon as they load it. They're really frustrated. Can we get this sorted ASAP?





Using ai in the enterprise! Creating a text sUmmarizer for slaCk messages

Keith [16:01 | 02/08/2019]: Thanks for bringing this to our attention, Fatima. Let's jump on this right away. @John, can you take the lead in investigating the issue since our architect is out sick today?

John [16:02 | 02/08/2019]: Sure thing, Keith. I'll dive into the codebase and see if I can find any potential culprits for the crashes.

John [16:02 | 02/08/2019]: Fatima, could you gather some additional information from the client? Ask them about the specific device, operating system, and any recent updates they might have installed.

Fatima [16:03 | 02/08/2019]: Absolutely, John. I'll reach out to the client immediately and gather those details. Will update you all once I have them. Dave [16:04 | 02/08/2019]: I understand the urgency here. Let's make sure

we keep the client informed about our progress Fatima. We don't want them feeling left in the dark during this troubleshooting process.

Fatima [16:04 | 02/08/2019]: Definitely, Dave. I'll keep the client updated at regular intervals, providing them with any relevant information we uncover.

John [16:20 | 02/08/2019]: I've checked the codebase, and so far, I haven't found any obvious issues. It's strange that the app is crashing on load. Could it be a memory-related issue? Keith, do we have any recent reports of memory leaks or high memory usage?

Keith [16:22 | 02/08/2019]: I'll pull up the monitoring logs, John, and check if there have been any memory-related anomalies in recent releases. Let me get back to you on that.

Fatima [17:01 | 02/08/2019]: Quick update, everyone. The client is using an iPhone X running iOS 15.1. They mentioned that the issue started after updating their app a few days ago

Keith [17:05 | 02/08/2019]: Thanks for the update, Fatima. That's helpful information. John, let's focus on testing the latest app update on an iPhone X simulator with iOS 15.1 to see if we can replicate the issue.

John [17:06 | 02/08/2019]: Good idea, Keith. I'll set up the emulator and run some tests right away.

Keith [17:30 | 02/08/2019]: John, any progress on replicating the issue on the emulator?

John [17:32 | 02/08/2019]: Yes, Keith. I managed to reproduce the crash on the emulator. It seems to be related to a compatibility issue with iOS 15.1

. I suspect it's due to a deprecated method call. I'll fix it and run more tests to confirm.

John [18:03 | 02/08/2019]: Fixed the deprecated method issue, and the app is no longer crashing on load. It looks like we've identified

and resolved the problem. I'll prepare a patch and send it to you, Keith, for review and deployment.

Keith [18:04 | 02/08/2019]:

Thank you, please provide me with the patch as soon as possible. Once I review it, we'll deploy the fix to the app store.

Dave [18:06 | 02/08/2019]: Great job, team! John, please keep the client informed about the progress and let them know we have a fix ready for them on the next app update. Can someone make sure the release notes reflect this?

John [18:07 | 02/08/2019]: Will do, Dave. I'll update the client and ensure they're aware of the upcoming fix.

Keith [18:27 | 02/08/2019]: Patch reviewed and approved, John. Please proceed with updating the app in the store. Let's aim to have it done within the next hour.

John [18:26 | 02/08/2019]: Understood, Keith. I'm in the process of uploading it now.

Fatima [18:38 | 02/08/2019]: I just informed the client about the fix. They're relieved and grateful for our prompt response. Thanks, everyone, for your collaboration and quick action. It's a pleasure working with such a competent team!

Using ai in the enterprise! Creating a text sUmmarizer for slaCk messages

Dave [18:40 | 02/08/2019]: Well done, team! Your efforts are greatly appreciated. We managed to turn this urgent problem around in record time. Let's keep up the good work!

Prompt Engineering 101: Text Summarization

So, needless to say, no one wants to spend their day constantly scrolling through Slack channels reading about issues and problems that are on fire. We're going to utilize the capabilities of ChatGPT for text summarization. To keep things simple, let's try a few prompts to send the entire listing of the chat messages to ChatGPT in order for it to give us a usable summary of all the events that have happened.

### **Prompt #1: “tl;dr”**

This prompt is short, sweet, and to the point. In case you're not familiar with it, it's a term used online frequently to briefly summarize a long article. It literally stands for “too long; didn't read,” which is how I feel about a lot of conversations on Slack when I'm swamped with work. Conveniently, ChatGPT can understand this simple term (I wouldn't necessarily call it a word) as a prompt, and the results are pretty impressive.

**Listing 4-2.** PROMPT. Asking for a Summarized Version of a Longer Conversation

System: You are a helpful assistant

User: Fatima [16:00 | 02/08/2019]: Hey everyone, I have an urgent ...

User: Tl;dr

Model: gpt-4o

Temperature: 1

Maximum length: 360

Top P: 1

Frequency penalty: 0

Presence penalty: 0

Notice how the number of tokens is kept small, at 360, which will ultimately help

ChatGPT to create a short response. Also, see that the temperature is left at 1 (the default value), but the Top P is increased all the way to its maximum (which is 1). Therefore, it's important to play around with temperature and other settings to control the tone and variety of ChatGPT's responses.

Listing 4-3 below shows ChatGPT's summarization of the conversation from Listing 4-1.

**Listing 4-3.** RESPONSE. Getting a Summarized Version of a Very Long Conversation

The team quickly responds to a client's app crashing issue.

John investigates the codebase while Fatima gathers information from the client. They discover a compatibility issue with iOS 15.1 and John fixes it. The patch is reviewed and approved, and the app is updated in the store. The client is informed and grateful for the quick resolution. The team is praised for their collaboration and prompt action.

That's fantastic! It reduced that extremely long conversation down to a simple and brief summary of only 68 words. As you can see, ChatGPT is not replacing a job here; however, it will definitely make someone more productive. Now, let's try a few more prompts, shall we?

**Prompt #2: "Explain This in Three Sentences or Less"**

So, If "tl;dr" doesn't provide a brief enough explanation for you, then we can tell ChatGPT to make things even simpler. ChatGPT does well with parameters, so let's specify that the explanation shouldn't be any longer than three sentences.

**Listing 4-4.** PROMPT. Asking for a Shorter Version of a Longer Conversation

System: You are a helpful assistant

User: Fatima [16:00 | 02/08/2019]: Hey everyone, I have an urgent ...

User: Explain this in 3 sentences or less.

Model: gpt-4o

Temperature: 1

Maximum length: 250

Top P: 0

Frequency penalty: 0

Presence penalty: 0

As you may have noticed, the temperature is set to 0.82 in this example. We really want ChatGPT to give us a concise answer here, so the “drier,” the better.

**Listing 4-5.** RESPONSE. Getting a Three-Sentence Summary

A customer reported an urgent issue of app crashes to the team. John investigates the issue and asks Fatima to gather additional information from the client. They discover that the issue is related to a compatibility problem with iOS 15.1. John fixes the problem and Keith approves the deployment of the fix to the client's production environment.

**Prompt #3: “I’m a Manager. Explain to Me What Happened”**

When you don’t want to sound like the one person that doesn’t understand, ChatGPT has the capability to rephrase what has been said and break it down until it’s very simple.

**Listing 4-6.** PROMPT. Asking for a Simplified Version of a More Complex Conversation

System: You are a helpful assistant

User: Fatima [16:00 | 02/08/2019]: Hey everyone, I have an urgent ... User: Summarize this conversation and explain it to me like I'm a manager with little technical experience.

Model: gpt-4o

Temperature:0.82

Maximum length: 750

Top P: 1

Frequency penalty: 0

Presence penalty: 0

Again, the temperature is set below 1.0 so that ChatGPT can keep the answer brief and easy to understand. We increased the token length so that ChatGPT can take the time to explain some potentially technical concepts.

**Listing 4-7.** RESPONSE. Getting a Simplified Version of a More Complex Conversation

In this conversation, Fatima brings up an urgent issue with a client who is experiencing app crashes as soon as they load it. Keith assigns John to investigate the issue since their architect is out sick. John asks Fatima to gather additional information from the client about their device, operating system, and recent updates. Dave emphasizes the importance of keeping the client informed throughout the troubleshooting process. John checks the codebase but doesn't find any obvious issues, so he suspects it could be a memory-related problem. Keith agrees to check for memory-related anomalies in recent releases. Fatima provides an update that the client is using an iPhone X running iOS 15.1 and mentioned that the issue started after updating the app. Keith suggests testing the latest app update on an iPhone X simulator

with iOS 15.1 to replicate the issue. John successfully reproduces the crash on the emulator and identifies a compatibility issue with iOS 15.1, possibly due to a deprecated method call. He fixes the issue and prepares a patch for review and deployment.

Keith approves the patch and asks John to update the app in the store. Dave requests that the client be kept informed and that the release notes reflect the upcoming fix.

John uploads the patch and informs the client about the fix.

Keith reviews and approves the patch, and John proceeds with updating the app in the store. Fatima informs the client about the fix, and they express relief and gratitude for the prompt response. Dave commends the team for their collaboration and quick action, emphasizing the importance of maintaining good work.

In summary, the team quickly addressed an urgent issue where a client's app was crashing on load. They investigated the issue, identified a compatibility problem with iOS 15.1, fixed it, and prepared a patch for deployment. The client was kept informed throughout the process, and the fix was successfully implemented, resulting in a relieved and grateful client.

#### **Prompt #4: “Give Me Suggestions on Next Steps”**

In addition to making a summary that's easier to understand, ChatGPT can explain in different styles and “tones.” We're going to increase the Top P to 0.75 because we want ChatGPT to think of more possibilities than what is the default. We are also going to increase the presence penalty to 0.5 because we want to encourage a versatile response.

**Listing 4-8.** PROMPT. Asking to ChatGPT for Advice on the Best Next Steps

System: You are a helpful assistant

User: Fatima [16:00 | 02/08/2019]: Hey everyone, I have an urgent ...

User: I'm a QA engineer at the company. Give me suggestions on next steps. Model: gpt-4o

Temperature: 1

Maximum length: 1200

Top P: 0.75

Frequency penalty: 0

Presence penalty: 0.5

**Listing 4-9.** RESPONSE. Suggestions from ChatGPT on Possible Next Steps

As a QA engineer, here are some suggestions for next steps:

1. Test the patch: Once the patch has been deployed to the app store, thoroughly test it on various devices and operating systems to ensure that the issue has been resolved and there are no new bugs or regressions introduced.

2. Conduct regression testing: In addition to testing the specific issue, perform a comprehensive regression test to ensure that the fix does not have any unintended side effects on other functionalities of the app.

3. Communicate the fix to the client: Keep the client updated on the progress and let them know that the fix is available in the latest app update. Provide them with clear instructions on how to update their app and encourage them to test it on their end as well.

4. Monitor user feedback: Keep an eye on user feedback after the fix has been deployed. Encourage users to report any issues they encounter, and be proactive in addressing them promptly.

5. Document the issue and resolution: Create a detailed report documenting the issue, the steps taken to reproduce and resolve it, and any other relevant information. This will serve as a reference for future troubleshooting and help improve the development process.

6. Conduct a post-mortem analysis: After the issue has been resolved and the fix has been deployed, schedule a post-mortem meeting with the team to discuss the root cause, lessons learned, and any process improvements that can be implemented to prevent similar issues in the future.

7. Continuous improvement: Use this incident as an opportunity to identify areas of improvement in the development and testing processes. Evaluate if there are any changes that can be made to prevent similar issues from occurring in the future.

Remember, open communication, collaboration, and continuous learning are key to ensuring the highest quality of your app and maintaining a strong relationship with your clients.

Now, of course, ChatGPT (like every other form of artificial intelligence on the planet) is not entirely perfect. For instance, recommendation number three is a valid item to do, except for the fact that it's not usually the responsibility of a QA person to communicate anything directly to the client or customer. That communication can be done either through tech support or with a product manager who has those communication channels (especially if they are a significant customer). So, the suggestion is fine, but it's not appropriate for a person with that role within a company.

### **Let's Talk About Real Prompt Engineering**

If you Google the term "prompt engineering," you'll find a ton of examples, blogs, and even full websites with subscription plans that will try to convince you that the perfect prompts can be created by simply only using text. As you have seen from the examples above, prompt engineering cannot be accomplished solely by merely crafting textual inputs.

In reality, the process is quite similar to cooking a sophisticated meal. Imagine trying to cook, for example, beef bourguignon using only salt as the seasoning and neglecting all other ingredients and spices! Honestly, the result would pale in comparison to the actual dish.

Similarly, try assembling an entire orchestra but only use one instrument and one musician. That's an embarrassing "one-man band." Therefore, simply adjusting the text to the prompt isn't enough in order to truly perform prompt engineering. The parameters such as the model's temperature, which controls randomness; the Top P, impacting token probability; the specific model used; the number of tokens; and the other parameters all play highly pivotal roles in getting a great response.

This book is not about prompt engineering, since (as you can see from the explanation above) it truly involves several factors that don't have anything to do with Python. However, you are highly encouraged to experiment with *all* the parameters to the models provided by OpenAI to find what works best for your use case.

#### Registering a Slack Bot App

Now that we know the various ways for ChatGPT to summarize a large body of text for us, let's see what's necessary in order to create a simple bot in Python that will programmatically grab all the messages from a channel within a Slack instance.

Note in order to accomplish these steps, you will need to have administrative access to a slack workspace. most developers will not have these levels of permissions; therefore, in order to fully experiment, i recommend that you create your own personal slack workspace for testing purposes. this way, you will have all the rights and privileges to install your slack bot.

But, one step at a time. First, we're going to make our Slack bot app, so head over the Slack API website: <https://api.slack.com/>

**Figure 4-1.** *In Order to Create a Slack Bot, Go To the Slack API Website*

Of course, you'll need to have a Slack account in order for this to work, so if you don't have one, then you need to create one first.

After you have logged in, go to the top-right of the page and navigate to "Your apps > Create your first app," as shown in Figure 4-1 above. In Slack terminology, a "bot" is an "app," and bots are not allowed to run on a Slack instance unless they have been registered with Slack first.

**Figure 4-2.** *Creating a New Bot App for Slack*

As shown in Figure 4-2, above, you'll be taken to the **Your Apps** page where you can manage your Slack apps. Immediately, you'll see a pop-up to **Create an App** button in the middle of the screen.

Select the option to create your app **from scratch**. This is because we want to be able to manipulate all of the details of the application

ourselves without overcomplicating things with a bunch of default settings.

Afterwards, you'll be prompted to specify a name for your bot and to select the workspace that you want your bot to have access to, as shown in Figure 4-3 below.

Click the **Create App** button to proceed.

***Figure 4-3.** Creating a New Bot App for Slack*

## **Specifying What Your Bot Can (and Can't) Do by Setting the Scope**

Now, you'll be presented with a screen that has a ton of options for bots for Slack workspaces. The first thing you need to do, however, is from the sidebar on the left, click **OAuth & Permissions**.

Our bot is going to be pretty simple; all it needs to do is read the messages from a channel in order to give us a summary of what was said. In addition to reading the messages, we also need to know the names of the people in the Slack workspace, otherwise, we'll get a UUID representation of the person instead of their name, which is meaningless to us.

So, scroll down to add the following OAuth Scope to your Slack bot, as shown in Figure 4-4, below.

- channels:history
- channels:read
- users:read

***Figure 4-4.** Adding Scopes for the Slack Bot App*

## **Confirming Your Settings**

After you've added the appropriate scopes for your bot, scroll back up and click on **Basic Information** from the left side bar.

On the page that follows, you'll see that there's now a green checkmark beside "Add features and functionality," which confirms that you've added your scopes correctly, as shown in Figure 4-5.

***Figure 4-5.** Confirm Your Settings*

## **Viewing the OAuth & Permissions Page**

As shown in Figure 4-6, below, navigate to the **OAuth & Permissions** page, and click on the "Install to Workspace" button.

***Figure 4-6.** The OAuth & Permissions Screen*

## **Installing Your Slack Bot App to Your Workspace**

Now that all the permissions have been requested, it's time to install your bot to your workspace. During the installation process, you should see a screen as shown in Figure 4-7 below.

***Figure 4-7.** "Installing" a New Slack Bot App*

Click the **Allow** button to authorize the bot and allow the permissions you allotted in the previous step.

Note it's important to understand what "installing" means here. in a traditional python sense, installing an app means to copy your files and



dependencies to another machine and have it execute. that's not what's happening here.

here, when you “install” a bot app, you're enabling your slack workspace to allow an app to join the workspace – that's all. the code for your bot will run on your own machine and not on slack's servers.

### **Getting Your Slack Bot (Access) Token**

This time, “token” actually means access token! In order to connect to the Slack API and access messages and user information programmatically, you need a specific OAuth token generated for your Slack bot.

#### ***Figure 4-8. Copy Your OAuth Token for Your Slack Bot App***

Back on the **OAuth & Permissions** page, be sure to copy the bot token (which usually starts with "xoxb-") from the page here, as shown in Figure 4-8.

### **Inviting Your Bot to Your Channel**

Next, you're going to go to the channel you'd like to use to test your bot and type in the following command in the channel itself.

/invite

Select the option “Add apps to this channel,” and then select the name of the Slack bot that you specified earlier when you registered the bot with Slack.

#### ***Figure 4-9. Adding Your Slack Bot to a Channel***

Congratulations! You now have successfully registered a Slack bot app with Slack, enabled it to read messages in your workspace, and added the Slack bot to a channel. Before we can write the Python code to access the channel in our workspace, we need to know what is the internal ID that Slack uses for our channel.

#### **Finding the Channel ID of Your Channel**

Ok, this is an easy step to do. In Slack, right-click on the name of your channel and select the option “View Channel details.” At the bottom of the pop-up window is the ID of your channel. Copy that number and save it for later. Your Python app will need this in order to join the right channel in your Slack workspace.

#### **Using Your Slack Bot App to Automatically Grab Messages from a Channel**

Alright, now that we have done all the prerequisites and we know the ID of our channel, let's get to the code in Python that accesses all the messages from a particular Slack channel.

### **Programmatically Reading Messages from Slack**

with slack\_chat\_reader\_bot.py

First things first, you need to install the official Python Slack library, slack\_sdk . You can use pip install like this:

pip install slack\_sdk

Listing 4-10 is a simple Python Slack bot that obtains the user name, timestamp, and message content of each posting in the channel within a designated time period.

**Listing 4-10.** slack\_chat\_reader\_bot.py

```
from slack_sdk import WebClient from slack_sdk.errors import
SlackApiError from datetime import datetime, timezone
Define your Slack API token
SLACK_BOT_TOKEN = "YOUR_SLACK_API_TOKEN"
Initialize a WebClient instance client =
WebClient(token=SLACK_BOT_TOKEN)
Define your channel ID channel_id = "YOUR_CHANNEL_ID"
Define start and end time in UTC start_time_utc = datetime(2024, 8, 3,
10, 0, tzinfo=timezone.utc) end_time_utc = datetime(2024, 8, 12, 15, 0,
tzinfo=timezone.utc) # Convert start and end time to Unix timestamps
start_time_unix = int(start_time_utc.timestamp()) end_time_unix =
int(end_time_utc.timestamp()) try:
 # Call the conversations.history method using WebClient
 response = client.conversations_history(
channel=channel_id, oldest=start_time_unix, latest=end_time_unix,
)
 # Check if the API call was successful if response["ok"]:
 # Reverse the messages list to get them in chronological order
 messages = reversed(response["messages"]) for message in
messages:
 user_id = message.get("user") timestamp =
datetime.fromtimestamp(float(message.get("ts")), tz=timezone.utc)
 user_info_response = client.users_info(user=user_id) if
user_info_response["ok"]: user_name =
user_info_response["user"]["name"] print("User:", user_name)
 print("Timestamp:", timestamp) print("Message:",
message.get("text"))
 print() else:
 print("Failed to fetch user info:", user_info_response["error"])
 else:
 print("Failed to fetch messages:",
response["error"])
except SlackApiError as e:
 print(f"Error: {e.response['error']}")
```

Let's walk through this code together. First, we're importing essential modules for interfacing with the Slack API and managing date/time data. We then define the Slack API token (SLACK\_BOT\_TOKEN) needed for authentication, along with the ID of the target Slack channel (channel\_id).

Next, we establish the start and end times for the message retrieval window, specifying them in coordinated universal time (UTC). Our script converts these time values into Unix timestamps since the Slack API expects timestamps in this format for message retrieval.

If the API call is successful, our script proceeds to extract and process the messages.

Messages are presented in chronological order so that the oldest messages appear first. To keep things clean, for each message, the script retrieves the user ID and message timestamp. It then uses the `client.users_info` method to fetch additional information about the user who sent the message.

In case of any errors during the API request process (`SlackApiError`), the script catches and handles them by printing an error message.

Listing 4-11 shows the output after executing `slack_chat_reader_bot.py`, which is truncated here since you already have the full text in Listing 4-1 earlier in this chapter.

**Listing 4-11.** The Output from Executing `slack_chat_reader_bot.py`  
 Fatima [2023-08-11T09:04:20] : Hey everyone, I have an urgent issue to discuss. I just got off a call with a client who's experiencing app crashes as soon as they load it. They're really frustrated. Can we get this sorted ASAP? :tired\_face:

Keith [2023-08-11T09:04:35] : Thanks for bringing this to our attention, Fatima. Let's jump on this right away. John, can you take the lead in investigating the issue since our architect is out sick today?

John [2023-08-11T09:04:52] : Sure thing, Keith. I'll dive into the codebase and see if I can find any potential culprits for the crashes.

John [2023-08-11T09:05:30] : Fatima, could you gather some additional information from the client? Ask them about the specific device, operating system, and any recent updates they might have installed.

...

### Exercises Left for the Reader

So, there are obviously a few additional things we can do here, and these steps will be left for you (the reader) to accomplish, for example:

- Connecting the code in `slack_chat_reader_bot.py` that reads the messages from Slack to the `chatgpt_client.py` so that grabbing the messages and getting a summary is a single step process.
- Adding more capabilities to the Slack bot itself such as adding commands so that anyone in the channel can request a summary. In its current state, the bot doesn't post anything in the channel. However, the "user interface" to the bot is the channel itself; therefore, someone should be able to interact with the Slack bot by typing a command (such as requesting a summary)
- Making sure that the bot doesn't make a bad situation worse. Whenever the bot provides a summary, it should not post in the channel itself because that could add a lot of noise to an already noisy situation. The best practice is to have the bot send a private message to the person asking for a summary (or whatever new command that you create).

### Conclusion

In this chapter, we talked about one of the various ways artificial intelligence can be put to practical use within the enterprise today. We discussed what is truly "prompt engineering," by discussing that prompt engineering cannot be accomplished by simply textual input to ChatGPT alone. You definitely need to understand the ramifications of all the input parameters to the ChatGPT API, in order to properly, and effectively, perform prompt engineering.

Using what we learned about prompt engineering, we were able to successfully obtain summarizations of any large body of text provided to us. Finally, we saw the code necessary in order to run an automated bot that will grab messages from any Slack channel programmatically, if we specify a valid date range.

In this chapter (as well as the previous chapter), we were working exclusively with the Chat Completions model of the OpenAI APIs. In the next chapter, we're going to push the boundaries of what's possible by experimenting with the Whisper and DALL·E models.

## MULTIMODAL AI: CREATING A PODCAST VISUALIZER WITH WHISPER AND DALL·E

### 3

Now let's introduce a new term: **multimodal AI**. In the simplest of terms, generative AI models can create content in one of four formats:

- Text
- Audio
- Images
- Video

Each of those formats are a **mode**. Multimodal AI is the process of using multiple AI models together to generate (or to understand) content where the input is one type of mode and the output is a different type of mode.

Take, for example, OpenAI's Whisper model. If you provide it audio, it is able to create a transcription of everything said into text. The same thing applies to DALL·E. If you supply it with a textual prompt, then it is able to generate an image of what you described.

In this chapter, we're going to take multimodal AI to the next level! As an avid podcast listener, I've often wondered what the scenery, the imagery, the characters, the subject, or the background looked like while listening to a very immersive story in audio format.

© Lydia Evelyn, Bruce Hopkins 2024

L. Evelyn and B. Hopkins, *Beginning ChatGPT for Python*,  
[https://doi.org/10.1007/979-8-8688-0929-3\\_5](https://doi.org/10.1007/979-8-8688-0929-3_5)

MultiModal ai: Creating a podCast Visualizer with whisper and dall·e 3

So, we're going to create a Podcast Visualizer using multiple models from

OpenAI. There are a few steps involved, but the final results are stunning. While listening to a podcast about a guy cooking some amazing things with tofu (don't knock it until you try it), the Podcast Visualizer came up with the image in Figure 5-1.

**Figure 5-1.** *The AI-Generated Result of Visualizing a Podcast About Tofu Using the GPT-4, Whisper, and DALL·E Models*

In order to make the code for the Podcast Visualizer easy to follow along, we'll do things separately in the following three steps:

Step 1: Take a podcast episode, and use the Whisper model to get a transcript.

Step 2: Take the resulting transcript, and use the GPT-4 model to describe the visual aspects of what's being discussed in the podcast episode.

Step 3: Take the resulting description, and use the DALL·E model to generate an image.

The code presented here in this chapter has tons of practical uses, for example:

- If you're just curious about the things in a podcast episode could look like (which is always the case for us), you can get a simple representative visual image to associate with what you're listening to.
- For people who are hearing impaired, you can easily turn a podcast or radio program into a slideshow of images. This greatly enhances the accessibility of the content
- For podcasters, you can now have a simple way to add a visual/hero image to each of your episodes. This is useful since podcast players such as Apple Podcasts and Spotify allow podcasters to display a single image to associate with an individual episode. This can help with engagement for your listeners.

Introducing the Whisper Model by OpenAI

Now let's introduce another new term: **automatic speech recognition** (ASR). The average everyday consumer is very familiar with this technology due to its integration into mobile phones (e.g., Siri for the iPhone) and smart speakers (e.g., any Alexa device). At its core, ASR technology converts spoken language into text.

Whisper is OpenAI's model for speech recognition, and the accuracy is astonishingly high. The listing below is a transcript of an episode of the very popular Duolingo Spanish podcast, which makes the Spanish language easy to be understood by English listeners by combining both English and Spanish together in a woven narrative story. In

Listing 5-1, you can see the transcript that was generated using the Whisper model.

MultiModal ai: Creating a podCast Visualizer with whisper and dall·e 3  
**Listing 5-1.** The Whisper Model Performs Speech Recognition to Convert Audio into Text

...I'm Martina Castro. Every episode we bring you fascinating, true stories to help you improve your Spanish listening and gain new perspectives on the world. The storyteller will be using intermediate Spanish and I'll be chiming in for context in English. If you miss something, you can always skip back and listen again. We also offer full transcripts at podcast.duolingo.com.

Growing up, Linda was fascinated with her grandmother, Erlinda. Erlinda was a healer or curandera, someone who administers remedies for mental, emotional, physical, or spiritual illnesses.

In Guatemala, this is a practice passed down orally through generations in the same family. Mal de ojo, or the evil eye, is considered an illness by many Guatemalans who believe humans have the power to transfer bad energy to others. Neighbors would bring their babies to Linda's grandmother when they suspected an energy imbalance. Su madre lo llevaba a nuestra casa para curarlo...

If you've ever worked with a speech recognition system before (even with sophisticated technologies like Siri and Alexa), you will know that it has problems, for instance:

- **Speech recognition has problems with punctuation**
- Have you noticed that nobody speaks with punctuation? For the English language, we use changes in tone or volume to ask a question or give an exclamation. We also use short and long pauses for commas and periods.
- **Speech recognition has problems with foreign words and accents**
- Depending on who you ask, there are at least 170k words in the English language. However, in conversational English we are always using foreign words like:
  - Tsunami (Japanese origin): A large sea wave often caused by an earthquake
  - Hors d'oeuvre (French origin): An appetizer
  - Lingerie (French origin): Women's underwear or nightclothes
  - Aficionado (Spanish origin): Someone who is very passionate about a specific activity or subject
  - Piñata (Spanish origin): A brightly colored box of candy for kids to beat relentlessly
- **Speech recognition has problems with names**
- Certain names of people, businesses, and websites can often be hard to spell and understand
- **Speech recognition has problems with homophones**
- Do you remember those words that sound the same but have different spellings and meanings? The fantastic editor of this book knows all of them!
  - Would / Wood
  - Flour / Flower
  - Two / Too / To
  - They're / There / Their
  - Pair / Pare / Pear
  - Break / Brake
  - Allowed / Aloud

As you can see from Listing 5-1, above, Whisper was able to understand all the punctuation in the audio, identify all the foreign words (of which there were several), understand the names, as well as the company name ("Duolingo") within a URL! Of course, if you noticed, it could also understand the difference between "wood" and "would."

### Features and Limitations of the Whisper Model

The Whisper model is able to convert spoken audio from the following languages into text:

- Afrikaans
- Arabic

- Armenian
- Azerbaijani
- Belarusian
- Bosnian
- Bulgarian
- Catalan
- Chinese
- Croatian
- Czech
- Danish
- Dutch
- **English (of course!)**
- Estonian
- Finnish
- French
- Galician
- German
- Greek
- Hebrew
- Hindi
- Hungarian
- Icelandic
- Indonesian
- Italian
- Japanese
- Kannada
- Kazakh
- Korean
- Latvian
- Lithuanian
- Macedonian
- Malay
- Marathi
- Maori
- Nepali
- Norwegian
- Persian
- Polish
- Portuguese
- Romanian
- Russian
- Serbian
- Slovak
- Slovenian
- Spanish
- Swahili
- Swedish



- Tagalog
- Tamil
- Thai
- Turkish
- Ukrainian
- Urdu
- Vietnamese
- Welsh

So, at the end of the day, it will be able to understand audio spoken by yourself and probably any language spoken by your friends and colleagues.

Developers are limited to sending no more than 50 requests per minute to the API, so this constraint needs to be taken into consideration if you want to transcribe vast amounts of audio.

Whisper supports audio in flac, mp3, mp4, mpeg, mpga, m4a, ogg, wav, or webm formats. Regardless of the format that you use, the maximum file size to send to the API is 25MB.

Now, If you haven't worked extensively with audio files, please be aware that some formats create *really huge* files (e.g., wav format) and others create really small files (e.g., m4a format). So, converting your file to a different format can help you with the 25MB limitation. However, later in this chapter, we'll see the code for a tool that takes a single large audio file and splits it into multiple, smaller files.

Using `OpenAI.audio.transcriptions.create()` to Transcribe Audio

The `OpenAI.audio.transcriptions.create()` method converts audio into text and is only compatible with the Whisper model. Let's take a look at Table 5-1 to find out what needs to be in the method parameters in order to call it successfully.

### **Creating a Utility App to Split Audio Files:**

`audio_splitter.py`

So, we're almost at the point where we are able to programmatically invoke the Whisper model using the `OpenAI.audio.transcriptions.create()` method. However, the Whisper model has a limitation of 25MB per file.

Now, this is not a problem if you're listening to (for example) the *StarDate* podcast from the University of Texas at Austin. This podcast gives you a great glimpse of what to look for in the nighttime sky in about two minutes of audio. However, that's not the case for other audio programs which tend to last for up to an hour (or even more). In cases like that, you're going to definitely exceed the 25MB file limitation.

Therefore, let's pair-program with ChatGPT and use our human intelligence to create our own utility that will take a single audio file and split it into multiple smaller files.

Note in this section, i'm presenting one of many possibilities available on how to segment a large audio file into smaller pieces. For instance, you can use popular audio editing apps (such as the open source tool, audacity, or the licensed tool, adobe audition) to manually cut up a large file into smaller files.

FFmpeg is one of the most reliable ways to programmatically manipulate audio files, and pydub is an open source library recommended in OpenAI's documentation on using Whisper with Python.

Listing 5-2 is the prompt that is sent to ChatGPT in order to get a basic app that splits audio files.

**Listing 5-2.** PROMPT. Using ChatGPT to Create the audio\_splitter.py App

System: You are a Python developer User: Write an application that takes as input a single MP3 file and splits the file into contiguous segments no longer than 10 minutes using the pydub library and ffmpeg

After a little back and forth, we were able to create (as shown in Listing 5-3) audio\_the splitter.py application that includes our edits to what was generated by ChatGPT. **Listing 5-3.** RESPONSE.

```
audio_splitter.py
from pydub import
AudioSegment
import os
Path to the input audio file
input_audio_path = "audio-example.mp3"
Load the audio file audio =
AudioSegment.from_mp3(input_audio_path) # Define segment length
(in milliseconds) - 10 minutes segment_length_ms = 10 * 60 * 1000 # 10
minutes in milliseconds # Split the audio into segments of ten minutes
each segments = [audio[i:i+segment_length_ms] for i in range(0,
len(audio), segment_length_ms)]
Output directory for saving segments output_directory =
"/PATH/TO/OUTPUT"
Ensure output directory exists, create it if necessary
os.makedirs(output_directory, exist_ok=True)
Process each segment (for example, you can save them to
separate files) for i, segment in enumerate(segments):
segment.export(os.path.join(output_directory, f"segment_{i}.mp3"),
format="mp3")
```

Our goal is simple: split an MP3 file into contiguous segments no longer than ten minutes using the Python language. In this process, we follow these steps:

- Specify the input file path, output directory, and the desired segment duration in milliseconds (ten minutes).
  - Load the MP3 file using the pydub library's AudioSegment class, which provides functionalities for audio processing.
- Iterate through the input MP3 file, segmenting it into smaller parts of

- the specified duration (ten minutes or less) using slicing.  
Save each segment as a separate MP3 file in the specified output
- directory.

In order for this to work, you need to have the pydub library **and** FFmpeg properly installed and configured in your project. FFmpeg is an extremely versatile media converter which not only handles MP3 audio files, but various other audio file formats (including m4a, ogg, and wav). It is able to convert video formats as well as static images like png, jpeg, and gif.

After running the script, you'll have a folder full of segmented audio files that are ten minutes long or less. Using the audio\_splitter.py utility, you have a simple to use Python script to modify the settings that work best for you. For our purposes, the goal here is to have audio files that are <25MB, so if you're transcribing eight-hour legal proceedings (for example) in WAV format, you may need to adjust the duration to be shorter, like six minutes in length.

When using the audio splitter, the best practice is to have the output folder to be a different folder from the input, and you'll see why when we start to invoke the OpenAI audio.transcriptions.create() method using the Whisper model.

Creating the Audio Transcriber: whisper\_transcriber.py

Now, let's build our next Python app, whisper\_transcriber.py. Again, we're going to pair-program with ChatGPT to get a basis to work with. Listing 5-4 is the prompt to put in the Chat Playground to get things started.

**Listing 5-4.** PROMPT. Asking ChatGPT to Use OpenAI's Python Library and Send MP3 Files to Whisper's API

System: You are a Python developer.

User: Using Python, write a script that iterates over all of the mp3 files in a single folder on my local computer and send all the files in the folder to be transcribed by OpenAI's Whisper model, using the OpenAI's python library. Model: gpt-4o

Temperature: 1

Maximum Length: 1150

After some back and forth, Listing 5-5 shows the response ChatGPT gave us that worked.

**Listing 5-5.** RESPONSE. whisper\_transcriber.py

```
import os
import
openai from
dotenv
import
```

```

load_dotenv
def
transcribe_m
p3_files(ope
nai_api_key,
mp3_folder_
path):
 # Set OpenAI API key
 openai.api_key = openai_api_key
 # Model used for transcription
 model = "whisper-1"
 # Desired format for the transcription response
 response_format = "text"
 # Iterate over each MP3 file in the folder
 for filename in sorted(os.listdir(mp3_folder_path)):
 if filename.endswith(".mp3"):
 file_path = os.path.join(mp3_folder_path, filename)
 # Read the content of the MP3 file
 with open(file_path, "rb") as f:
 file_content = f.read()

 # Transcribe the MP3 file
 response = openai.audio.transcriptions.create(
 file=(filename, file_content),
 model=model,
 response_format=response_format
)
 # Print the entire response object
 print(response)
def main():
 except Exception as e:
 print(f"Transcription error for file {filename}: {e}")
 # Load environment variables from .env file
 load_dotenv()
 # API key for OpenAI
 openai_api_key = os.getenv("OPENAI_API_KEY")
 # Folder containing the MP3 files to be transcribed
 mp3_folder_path = "/PATH/TO/OUTPUT" # Replace with your MP3 folder path
 # Transcribe MP3 files
 transcribe_mp3_files(openai_api_key, mp3_folder_path)
if __name__ == "__main__":
 main()

```

As you can see from the code listing above, one of the major benefits of keeping the input file in a separate folder from the output files is that you can give the script above the path to the output folder and you don't have to worry about anything else. This script will grab all the files in the

output folder regardless of the number of files in that folder, so it could be one or 100 files – it doesn't matter.

Having a Little Fun and Trying Things Out with a Podcast

Ok, so let's run a test using the code that we have presented so far. *This American Life* is a weekly public radio program (and also a podcast) that's hosted by Ira Glass and produced in collaboration with WBEZ Chicago. The icon for this podcast is shown in Figure 5-2.

**Figure 5-2.** *If You're Looking for a Good Podcast with Compelling Stories, I*

*Recommend Listening to "This American Life" (Image credit: WBEZ Chicago)*

Each episode weaves together a series of stories centered around a specific theme or topic. Some stories are investigative journalism, and others are simply interviews with ordinary people with captivating stories. Episode 811 is entitled, "The one place I can't go." and the file is 56MB in MP3 format. Since we already know that 56MB is way too big to send to Whisper to get transcribed, we need the `audio_splitter.py` to break up the file for us.

Listing 5-6 shows an excerpt from the full transcript of the episode.

**Listing 5-6.** The Partial Transcript of Episode 811 of "This American Life"

"...My younger cousin Camille is not really a dog person, but there is one dog she adored. Her name was Foxy, because she looked exactly like a fox, except she was black. She was the neighbor's dog, but she and Camille seemed to have a real kinship, maybe because they both weren't very far from the ground. Camille was around four or five years old back then, and she had a little lisp, so Foxy came out as Fozzie. I thought it was one of the cutest things I'd ever heard.

The way Camille remembers Foxy, it's almost like a movie. Her memories feel like endless summer, hazy and perfect, like a scene shot on crackly film. I just remembered like the feeling of being excited to go and see Foxy. I have an image in my head of like coming to the house, and I could see Foxy was like outside. I can see Foxy through the door that leads to the garden.

There's a story about Camille and Foxy that I think about fairly often. I've talked about it with my sister for years, but never with Camille. And it's this. Once when they were playing..."

For brevity, we're only showing an excerpt of the transcript. The full transcript itself is over 8000 words due to the fact that the episode is nearly one hour in length.

## Going Meta: Prompt Engineering GPT-4 to Write a Prompt for DALL·E

Since the full text transcript of the podcast episode that we want to visualize is thousands of words, we're going to use GPT-4 to automatically create the prompt needed for the DALL·E model. DALL·E is able to take a textual description in a prompt and create an image, but it's best to keep the prompt as short as possible. Listing 5-7 is the prompt for GPT-4 to generate a prompt for DALL·E.

**Listing 5-7.** The Prompt for GPT-4 to Create a Prompt for DALL·E

System: You are a service that helps to visualize podcasts. User: Read the following transcript from a podcast. Describe for a visually impaired person the background and subject that best represents the overall theme of the episode. Start with any of the following phrases: - "A photo of" - "A painting of" - "A macro 35mm photo of" - "Digital art of"

User: Support for This American Life comes from

Squarespace... Model: gpt-4o

Temperature: 1.47

Maximum length: 150

Top P: 0

Frequency penalty: 0.33

Presence penalty: 0

DALL·E needs to know the type of image to generate so that's why we need to specify that the image should be a photo, painting, digital art, etc. We need to ensure that the resulting text generated by GPT-4 is short, so we want to have a maximum length of 150 tokens. Also, in order to prevent GPT-4 from repeating some phrases multiple times, we introduced a frequency penalty of 0.33.

Listing 5-8 shows the results from GPT-4 after reading the transcript of Episode 811 of *This American Life*.

**Listing 5-8.** The Prompt for DALL·E Created by GPT-4

Digital art of a young girl sitting in a garden with a black dog that looks like a fox. The girl is smiling and the dog is wagging its tail. The image has a hazy, dream-like quality, with crackly film effects to evoke nostalgia.

Using `OpenAI.images.generate()`  
to Create Images

In order to use the DALL·E model to dynamically create an image from a text prompt, you need to call the `OpenAI.images.generate()` method.

### Examining the Method Parameters

Table 5-2 describes the format of the parameter necessary for the request body for the `OpenAI.images.generate()` method. For obvious reasons, the prompt parameter is the only required parameter in order to successfully invoke the model.

### DALL·E Golden Rule #1: Get Familiar with the Types of Images That DALL·E Can Generate

First and foremost, one of the most important things that DALL·E needs to understand is the type of image that needs to be generated. Here's a list of several of the most common types of images that DALL·E is able to create:

- 3D render
- Painting
- Abstract painting
- Expressive oil painting
- Oil painting (in the style of any deceased artist)
- Oil pastel
- Digital art
- Photo
- Photorealistic
- Hyperrealistic
- Neon photo
- 35-mm macro photo
- High-quality photo
- Silhouette
- Vaporware
- Cartoon
- Plush object
- Marble sculpture
- Hand sketch
- Poster
- Pencil and watercolor
- Synth wave
- Comic book style
- Hand drawn

### **DALL·E Golden Rule #2: Be Descriptive with What You Want in the Foreground and Background**

We cannot emphasize enough that you need to be descriptive with DALL·E in order to get consistent, desirable results. It may sound weird, but the best way to describe your image to DALL·E is to act like you're describing a dream to another person.

So, as a mental exercise between us, try to describe your last dream. As you describe the people, places, and things in your dream, you have in your mind the most important things that you remember, as well as the experience that you felt. As you describe things to another person, tiny details start to emerge such as

- How many people were present (if any)?
- What position were the people or animals in? Standing, sitting, or laying down?
- What things were in the scenery and the background?
- What items stood out to you? Sounds? Smells? Colors?
- How did you feel? Happy, eerie, excited?

- What was the perceived time of day? Morning, midday, night?  
If you can describe a dream to another person, then you should have no problem describing what you want to DALL·E.

**Let's Play Around with Prompts to DALL·E**

To give you an idea of what kind of things you can do with DALL·E, we've created a couple prompts for images in a variety of styles for inspiration to get you started. Figures 5-5 through 5-10 show several different examples.

**Figure 5-5.** *Photo-Realistic: Penguin at an Office Desk*

**Figure 5-6.** *Vector Art: A Man Wearing Sunglasses with a Beard of Flowers*

**Figure 5-7.** *Vaporwave: Greek Statues in a Temple*

**Figure 5-8.** *Digital Painting: A Female Videogame Character in a Jungle with a Spear and a Shield*

**Figure 5-9.** *3D Art: A Baby Panda Working at a Typewriter*

**Figure 5-10.** *Oil Painting: Ship on Stormy Water with Dynamic Lighting and Lightning in the Background*

**Conclusion**

In this chapter, we accomplished a lot! With a few scripts, we created a Podcast Visualizer.

- We created and used `audio_splitter.py`, which works as a utility for us. If you have an audio file that's larger than the limitations of the Whisper model, this script will give you a folder of smaller audio files to send to Whisper.
- We created and used `whisper_transcriber.py` to get a transcription of a folder of audio files. The folder can contain a single audio file or several files. Your only limitation is the number of requests that you can send to the Whisper model.
- We did a little prompt engineering with GPT-4 in order to get a descriptive prompt of the imagery in a podcast based upon the transcript.
- Finally, we created and used `dalle_client.py` to take the prompt generated from calling the GPT-4 model and getting an image that represents the podcast episode visually.



## CREATING AN AUTOMATED COMMUNITY MANAGER BOT WITH DISCORD AND PYTHON

When you're launching an app or a service, it's important to build and maintain your own community. Below are the telltale signs of a healthy user community:

- Members engage in meaningful discussions, sharing insights, feedback, and support.
- Disagreements or debates occur, but they are approached constructively without resorting to personal attacks or derogatory language.  
There's an atmosphere of respect, where members listen to each other and acknowledge differing opinions.  
A mix of old and new members actively participate, ensuring the community remains vibrant and doesn't stagnate.
- Users contribute diverse content, from answering questions to sharing resources, which enriches the community's knowledge base. There's a balance between giving and taking; members who seek help or information also offer it to others.
- New members frequently join, often referred by existing members, indicating that the community is seen positively and worth  
• Users often become advocates for the community or platform, promoting it outside of the direct community space, such as on social media or other forums.
- The community helps to shape the app or service by providing new ideas for features and functionality

No matter what type of app or service that I create, I would love for my user community to exemplify the items listed above!

### Choosing Discord as Your Community Platform

Over the past few years, Discord has surged in popularity as a useful tool for community management for people who are passionate about their communities. This is partially due to its cross-platform compatibility, allowing members to stay connected whether they're on a desktop, mobile device, or web browser. However, one of its standout features is the invitation-based community system, which helps community managers to control growth and prevents spam. This model not only ensures a tailored experience for members but also enhances security, since community managers have the discretion to grant or deny access.

Discord not only supports text messaging but also supports voice chats and streaming video. Very similar to Slack, Discord allows community managers to separate content into channels to organize

discussions, streamline information flow, and help users see the content that they're interested in.

#### Creating a More Advanced Bot Than Our Slack Bot

Now, If you successfully went through the steps in Chapter 4 where we worked with a Slack bot, then the steps in this chapter will feel familiar to you. In Chapter 4, we created a Slack bot to read a single channel during a time period and get a summary of the content discussed. The Slack bot was not a community manager but was more like a helpful assistant.

For the remainder of this book, we're going to perform all the steps necessary to make powerful bots for Discord that will use AI to help actually manage the community.

#### Creating a More Advanced Bot Than Any Typical Discord Bot

If you've ever had any experience using a Discord bot, then you're probably aware that the most common way in order to interact with them is with what's called a `/` command." This enables typical bots (read: nonintelligent bots) to essentially work only when they have received a very specific operation or command. If the `/command` is not provided, then the bot will be silent and not do anything. Essentially, it exemplifies the phrase "speak only when you are spoken to."

However, we are creating a Discord bot that will be artificially intelligent, and therefore, it will be much more advanced than any typical Discord bot. We're going to create bots that will be able to read and see all messages in the Discord server and be intelligent enough to respond correctly.

#### Understanding the Roles for the Bots

So, let's explore a scenario in order to make things real. We're creating a public Discord server to interact with the users of a mobile banking app. Our end goal is to have bots written in Python to handle the following scenarios:

- **Q&A:** Monitor a specific channel and automatically answer questions from users about how to use the banking app. For this to work, the bot will need to be trained on how the app works
- **No solicitations:** For any business community, it's important that the participants of the community are not being targeted by unscrupulous individuals. For example, if you're creating a banking app, do you want your customers contacted by anyone whose username is "B4nk Admin"?
- **No harmful content:** For any community, it's important for the members to be protected from harmful content such as hate language

#### Our Example Bank: Crook's Bank

For the purposes of this example, I decided upon a fictional name of a fictional bank that would have an extremely low likelihood of coinciding with the name of a real bank. Therefore, for this example, "Crook's Bank" is launching a new mobile app for customers of their bank. They want to have a channel that will be monitored by a bot to answer

questions from users of the app, and they also want to ensure that no one is soliciting users of their app or posting hurtful or harmful content in their Discord servers. Figure 6-1 shows a fun illustration of Crook's Bank.

**Figure 6-1.** *This Fake App from a Fake Bank Has Real Problems*

**First Things First: Create Your Own Discord Server**

Before we can make an AI Discord bot, we're obviously going to need a Discord server already in place for the bot to interact with. Use either the Discord App or go to the

Discord website (log in first of course), and start the process to Add/Create a new server. After you have started the process, select the option labeled "Create My Own" as shown in Figure 6-2.

**Figure 6-2.** *Creating Your Own Discord Server*

Next, you'll be prompted to specify additional information about your server. Continue to proceed through the creation process until you are prompted to provide a name and icon for your server, as shown in Figure 6-3.

Specify the name of your server and provide an optional server icon (if you have one).

**Figure 6-3.** *Providing a Name for Your Own Discord Server*

**Create the Q&A Channel**

By default, every Discord server has a "general" channel, but we want a dedicated channel especially for questions and answers. Depending upon how you created your server, Figure 6-4 will be presented to you to create your new channel.

**Figure 6-4.** *Creating a Channel Using the Web Interface*

**Registering a New Discord Bot App with Discord**

Now that we have our Discord server with the appropriate channels created, it's time to register the bot itself – or rather, in our case, the bots themselves. In order to keep the code clean and manageable, we'll actually have multiple bots for our Discord server. The first bot will be used exclusively to answer questions in the "q-and-a" channel. The second bot will monitor all channels for unwanted content, such as harmful content or solicitations.

In order to create our bot, head over to the Discord Developers website, which is also shown in Figure 6-5:

<https://discord.com/developers>

**Figure 6-5.** *In Order to Create a Discord Bot, Go To the Discord Developers Website*

At the top-right of the page, click on the button "New Application," as shown in Figure 6-5.

In both Discord and Slack terminology, a "bot" is an "app," and bots are not allowed to run on Discord servers unless they have been registered with Discord first.

Specify a name for the bot and click the "Create" button, as shown in Figure 6-6.

**Figure 6-6. Creating/Registering a Bot for Discord**

**Specifying General Info for the Bot**

Afterward, you will be taken to a page where you can specify general information about your bot, as shown in Figure 6-7.

**Figure 6-7. I Decided to Give My Bot a Cute Little Robot Icon**

Be sure to familiarize yourself with the navigation menu on the left side of the page. As you can see, we have several categories of settings to configure for our bot. By default, we have landed on the "General Information" page, where we specify basic info about our bot. If you have an icon ready for your bot, you can upload it here.

**Specifying OAuth2 Parameters for the Bot**

Now, it's time to specify the scopes and permissions for our bot. If you followed the steps in creating a Slack bot in Chapter 4, then (as stated before) this procedure will feel familiar to you. Bots **cannot** and **should not** have the ability to do anything and everything – they should be only allowed to perform a list of operations that they were designed to perform.

On the settings navigation menu on the left, navigate to "OAuth2 > OAuth2 URL Generator" to continue.

Below are the scopes that we want:

- Scopes
- Bot

This is reflected in Figure 6-8.

**Figure 6-8. Selecting the Scopes**

After we select the bot's scope, we get to see all the permissions that are only applicable to bots (be sure to scroll down to see them).

Bots can be pretty powerful depending on the permissions you give it. There are permissions that allow the bot to act in the capacity of a normal human moderator, such as managing the server, roles, and channels. Bots with these permissions can also kick and ban members.

What we're going to enable for our bot right now are the ones that allow the bot to send and receive messages in text channels, and voice permissions allow the bot to participate in voice channels. Simple enough, right?

Select the following permissions for the bot:

- Bot permissions
- Read messages/view channels
- Send messages
- Read message history

This is reflected in Figure 6-9.

**Figure 6-9. Specifying the Bot Permissions**

Although you haven't written any Python code yet, now it's time to invite your bot to your server.

Invite Your Bot to Your Server

After you have selected the appropriate permissions, Discord will give you a dynamically generated URL that will enable you to invite your bot to your server (scroll down on the page in case you don't see it).

Copy the URL and paste it into a web browser where you're already authenticated into Discord. The result is shown in Figure 6-10.

**Figure 6-10.** *If You Read the Screen Carefully Here, You Can See That Discord Has a Sense of Humor*

Click the "Continue" button to add the bot to your server.

Next, you will see a page that looks quite similar to the previous one, but the main difference is that it will give you a summary of all the permissions and capabilities of the bot. Typically, this is quite useful if you are adding a bot to a server that you *did not create*. However, since we created this bot ourselves, this is just a confirmation of the settings that we have already specified earlier. Figure 6-11 shows the confirmation screen for authorizing our bot with the selected permissions.

**Figure 6-11.** *Confirming the Capabilities of the Bot*

Click on the "Authorize" button to give the bot the permission to run on your server.

If everything went smoothly, then you should see an automated message in the general channel of your server that indicates that the process has been successful.

Getting the Discord ID Token for Your Bot and Setting the Gateway Intents

Now, it's time to get the Discord ID token for your bot, which you'll use in your code to authenticate your bot programmatically.

Note For obvious reasons, using the word "token" here makes me nervous because this word has two distinct meanings in this book due to the context, but here's a quick refresher on the meanings:

- when using discord and slack apis, a "token" is an authentication token.
- when using openai apis, a "token" as a part of a word.

Go back to the Discord Developers website, and click the "Bot" category in the settings navigation menu to continue.

Although you haven't seen your token yet, you need to click the "Reset Token" button, as shown in Figure 6-12.

Be sure to copy and save the ID token to someplace safe. You will need this token in the Python code that's presented later in this chapter.

**Figure 6-12.** *Click on the "Reset Token" Button to See Your ID Token*

Scroll down the page to the section named "Privileged Gateway Intents" and enable the option named "MESSAGE CONTENT INTENT."

Note so, let's slow things down a bit and talk about intents. what exactly is an "intent" and why is it needed? For the purposes of the discord api, you need to specify explicitly every type of information

that you want to be notified by discord programmatically. otherwise, discord will constantly bombard you with events that are not relevant to you or your bot. For example, for our purposes, we don't care when people join or leave the server. however, if you want to send a list of server rules to anyone who joins your server for the first time, then you definitely would want to enable the "serVer memBers intent." when we deep dive into the code, you'll see more information about intents.

Be sure to click on the green button, "Save Changes" to save your changes. The result is shown in Figure 6-13.

**Figure 6-13.** Enable the option named "MESSAGE CONTENT INTENT"

### Creating a Q&A Bot App in Python to Answer

#### Questions from a Channel:

tech\_support\_bot\_dumb.py

Of course, now that we've done all the prerequisites necessary and we know the name of the channel that we want to monitor for questions from our users, let's get to the code in Python that joins our server and accesses all the messages from a specific Discord channel.

This is the first of two Discord bots that we're creating in this chapter. This bot, tech\_support\_bot\_dumb.py, will be responsible for watching the messages in the "q-and-a" channel in our Discord server.

Later on, in this chapter, we'll create another bot that will be responsible for moderating *all content* in the Discord server for unwanted content, including the "q-and-a" channel. The goal here is to follow the architectural pattern of "separation of concerns." Rather than creating a gigantic Python Discord bot that performs all the moderation needs for the Discord server, we're going to separate the functionality into two different apps.

We're also going to take things step by step and focus this chapter on getting past the learning curve for the Discord capabilities in Python. In the final chapters of this book, we'll enhance both bots and make them artificially intelligent using the Open AI APIs.

Listing 6-1 is the code that we need to create a basic Discord bot that watches all the messages posted in a single channel and provides an answer.

#### Listing 6-1.

tech\_support\_bot\_dumb.py

```
import discord
Bot's token for
authentication
discord_token =
'YOUR_TOKEN_HERE'
Name of the channel the bot should monitor
and interact with channel_to_watch = 'q-and-a'
Initialize the Discord client discord_client =
discord.Client()
```

```

Event handler for when the bot is ready
@discord_client.event async def on_ready():
 print('Logged in as',
 client.user)
 print('-----')
Event handler for when a message is received
@discord_client.event async def on_message(message):
 # Ignore messages sent by the bot to
 # prevent self-responses
 if message.author
 == client.user:
 return
 # Ignore messages not in the specified "q-and-a" channel
 if isinstance(message.channel, discord.TextChannel) and message.
 channel.name !=
 channel_to_watch:
 return
 # Send a greeting response to the user who sent the message
 reply = f'hi {message.author.mention}, I can help you with that!'
 await message.channel.send(reply)
Run the bot with the provided token
discord_client.run(discord_token)

```

At the beginning of the script, you should notice that we specify the channel that we are interested in monitoring with the "channel\_to\_watch" variable.

Note For some reason, discord's own terminology sometimes refers to discord servers as "guilds." however, from our perspective, a guild is simply a discord server.

### **Handling Messages Sent to the Discord Server**

As we wrap things up for this first Python Discord bot, we need to talk about the on\_message(message) function. This function is called asynchronously for every single message sent to the Discord server, as well as for messages from users sent directly to the bot itself as a DM. Note did you know that when the bot sends an answer to a person's question in the discord server, discord will invoke the on\_message(message) function of the bot to give the bot the message that the bot just sent. this sounds like a recipe for an infinite loop, doesn't it? therefore, we have logic in place for the bot to ignore messages sent from itself.

In the final lines of the on\_message(message) function, we make sure that we give a friendly reply to the original sender of the message by "tagging" them in the response. As we mentioned before, this first

version of the Q&A bot is dumb. It will respond to your question when posted in the Discord server, but the response won't actually answer your question.

### **Success! Running Your First Discord Bot:**

#### **tech\_support\_bot\_dumb.py**

Now, let's run our Python Discord bot. After executing the app, be sure to return back to your Discord server and try to type a question in the channel that you set up for Q&A. Figure 6-14 shows the response to my question, "Is this bot going to answer my questions about the app?"

#### **Figure 6-14. Success Running the Q&A Bot in Discord**

As you carefully inspect Figure 6-14, you'll see some key features such as

- On the right side, you'll see that the bot is online with a green status indicator.
  - After asking a question in the channel, the bot will tag you directly.
- Streamlining the Process of Registering Our Next Discord Bot App with Discord

Now that we have successfully performed all the steps in order to get a functioning Discord bot, creating the second bot will be a piece of cake! So, let's briefly reiterate all the steps from above in order to create our second Discord bot. I'll be sure to point out the items that need to be changed or enhanced due to the fact that this second bot will work as a moderator, instead of providing answers to questions from the users of our Discord server.

### **Registering a New Discord Bot App with Discord**

Perform the same steps as above; however, it would be wise to give the bot a different name. For me, this second bot will be named "Mod Bot."

#### **Specifying General Info for the Bot**

For me, I have a different icon for the content moderator bot, so I specified it here, as you can see in Figure 6-15.

#### **Figure 6-15. Providing a Name and Icon for the Second Bot**

#### **Specifying OAuth2 Parameters for the Bot**

This second bot needs more permissions in order to perform more tasks. Below are the scopes that we want:

- Bot
- Kick members
- Ban members
- Send messages
- Manage messages
- Read message history

Invite Your Bot to Your Server

Repeat the same steps as above for the first bot.

### **Getting the Discord ID Token for Your Bot and Setting the Gateway Intents**

Again, follow the steps above in order to get the Discord ID token. Then, scroll down the page to the section named "Privileged Gateway Intents,"



and enable the options named “SERVER MEMBERS INTENT” and “MESSAGE CONTENT INTENT.” Creating The Next Discord Bot: `content_moderator_bot_dumb.py`

The role of the content moderator is to make sure that unwanted content is not posted in the Discord server. Just like the previous bot that we created earlier in this chapter, this bot will not (yet) be artificially intelligent. In its current state, the bot will indiscriminately delete any message posted anywhere in the server that contains the word “puppies.”

It’s not because puppies are inherently evil. However, they do have a tendency to destroy your favorite pair of shoes when left alone. In all honesty, we simply need something to test our code in Discord when we run our bot.

Listing 6-2 is the code for `content_moderator_bot_dumb.py`.

**Listing 6-2.** `content_moderator_bot_dumb.py`

```
import discord
Bot's token for authentication discord_token = "
Banned word to monitor in messages banned_word = 'puppies'
Initialize the Discord client discord_client = discord.Client() # Event
handler for when the bot is ready @discord_client.event async def
on_ready():
 print('Logged in as', discord_client.user)
print('-----')
Event handler for when a message is received
@discord_client.event async def on_message(message):
 # Ignore messages sent by the bot to prevent self-responses

 if message.author == discord_client.user:
 return
 # Check if the message was sent in a guild (server)
 if message.guild is not None:
 # Check if the banned word is in the message content
 #if banned_word in
 message.content:
 # Delete the message
 await
 message.delete()
 # Mention the user who sent the inappropriate message
 author_mention = message.author.mention
 # Send a warning message mentioning the user
 await message.channel.send(f'{author_mention} This comment was
 deemed inappropriate for this channel. '
 'If you believe this to be in error,please contact one of the human server
 moderators.')
Run the bot with the provided token
discord_client.run(discord_token)
```

### Handling Messages Sent to the Discord Server

Again, let's focus our attention on the `onMessageReceived()` function, since it's called asynchronously every time a message is posted to the Discord server. As you can see, if the message posted to the server contains the banned word, then we delete the message and warn the sender with a `@mention` message in the same channel where the offending message was posted.

### Success Again! Running Your Second Discord Bot:

#### `content_moderator_bot_dumb.py`

Now, let's run our second Python Discord bot. After executing the app, be sure to return back to your Discord server and type a message in any channel that contains the offending word. Figure 6-16 shows the bot in action.

**Figure 6-16.** *This Bot Has a Strict Rule About Discussing “Puppies”; However, Discussing “Kittens” Is Perfectly Fine*

#### Conclusion

We just went through all the steps necessary to create two functioning Discord bots in Python. For those who are unfamiliar with the process of creating a Discord server, we showed the process on how to set up a server to manage our community.

As you can see, we took a much different approach compared to our Slack bot that we did in Chapter 4! The Slack bot that we created was pretty much focused on user productivity within the workplace. The two Discord bots, on the other hand, are truly focused on community management. We have everything in place for these bots to be artificially intelligent with the help of OpenAI's APIs. This is all accomplished in the final two chapters.

### Exercises Left for the Reader

In the next chapters, we're going to make our “dumb” bots to be intelligent, but there's at least one thing we can do right now. Rather than using the command line to report status messages, it's better for the bots to have their own channel that's exclusively used for status reports. This way, when the bot starts up, shuts down, or has anything important to inform the administrators, it's all logged and recorded in a central location.

## ADDING INTELLIGENCE TO OUR DISCORD BOTS,

### PART 1: USING THE CHAT MODEL FOR Q&A

At this point, we have all the structure in place to make both our Discord bots that we created in the previous chapter to be fully functional and artificially intelligent, and that's what we're going to do in the final chapters of this book. The focus of this chapter will be on our tech support bot, which was called `tech_support_bot_dumb.py`. Below are the two major changes that we're going to make:

- Modify our `chatgpt_client.py` script so that the Discord bot script can ask questions about specific information that we provide to it. The updated file will be a class called `chatgpt_client_for_qa_and_moderation.py`. It will be used for Q&A purposes in this chapter but will be used in the final chapter of the book as well.
- Modify our `tech_support_bot.py` script (formerly named, `tech_support_bot_dumb.py`) so that it can load an external text file that contains frequently asked questions with the answers. The script will then provide the contents of the text file to the `ChatGPTClient` class inside the `chatgpt_client_for_qa_and_moderation.py` script who is responsible for creating the prompt and of course invoking the chat object of the OpenAI API for Python.

Making `tech_support_bot.py` More Intelligent

Listing 7-1 contains the full contents of the Frequently Asked Questions that the fictional customer support team has created based upon support tickets from users of the newly launched mobile banking application.

#### **Listing 7-1.** FAQ.txt

1. What is the Crooks Bank Mobile App?  
The Crooks Bank Mobile App is a cutting-edge mobile banking app that allows you to manage your finances, make transactions, and access a wide range of banking services conveniently from your mobile device.
2. How can I download the Crooks Bank Mobile App?  
You can download the Crooks Bank Mobile App from the App Store for iOS devices and Google Play for Android devices. Simply search for the "Crooks Bank Mobile App" and click the "Install" button.
3. Is the Crooks Bank Mobile App safe and secure?  
Yes, the Crooks Bank Mobile App prioritizes your security. We use state-of-the-art encryption and security protocols to protect your data and transactions. Your information is safe with us.
4. What features does the Crooks Bank Mobile App offer?  
The Crooks Bank Mobile App provides a variety of features, including:

- Account Management: View account balances, transaction history, and more.
  - Transfer Funds: Easily transfer money between your accounts or to other bank accounts.
  - Bill Payments: Pay bills and manage recurring payments.
  - Deposit Checks: Snap photos of checks for remote deposit.
  - ATM Locator: Find nearby ATMs and branches.
  - Notifications: Receive alerts for account activity and important updates.
5. Can I link external accounts to the Crooks Bank Mobile App?  
Yes, the Crooks Bank Mobile App supports linking external accounts from other financial institutions. You can monitor and manage your accounts from different banks in one place.
  6. How can I reset my password if I forget it?  
If you forget your password, simply click the "Forgot Password" option on the login screen. You'll receive instructions on how to reset your password.
  7. What are the fees associated with the Crooks Bank Mobile App?The Crooks Bank Mobile App aims to be transparent with its fees. You can find information on account fees, transaction charges, and other costs in the "Fees" section within the app or on our website.
  8. Can I get customer support through the Crooks Bank Mobile App?  
Absolutely! We offer customer support through our in-app messaging feature. You can also find our customer service contact information on our website.
  9. Is the Crooks Bank Mobile App available for business accounts?  
The Crooks Bank Mobile App primarily caters to personal banking needs. However, we have plans to introduce business banking services in the future.
  10. How can I provide feedback or suggestions for the Crooks Bank Mobile App?  
We welcome your feedback! You can submit suggestions and feedback through the "Contact Us" section in the app or on our website.  
As you can see in theFAQ.txt file in Listing 7-1, there's no magic involved here. It's simply a list of questions and the answers. Now, let's see the newly modifiedtech\_support\_bot.py class. This is represented in Listing 7-2.
- Listing 7-2.** tech\_support\_smart\_bot.py
- ```
"""
tech_support_bot.py
A Discord bot integrated with ChatGPT for automated responses in a
designated channel.
This script initializes a Discord bot using the discord.Client() class and
listens for events such as messages being sent. When a message is
```

received in the specified channel, the bot calls ChatGPT to generate a response based on the message content, and sends the response back to the same channel.

Requirements:

- discord (<https://pypi.org/project/discord.py/>)
- chatgpt_client (Assumed to be a custom module providing interaction with ChatGPT) Usage:
 1. Replace the DISCORD_TOKEN variable with your bot's token obtained from the Discord Developer Portal.
 2. Adjust the CHANNEL_TO_WATCH variable to specify the name of the channel the bot should monitor and interact with.
 3. Ensure that the chatgpt_client module is properly implemented and accessible.

Note: This script assumes the presence of a chatgpt_client module for interaction with the ChatGPT API.

```

"""
import discord from
chatgpt_client_for_qa_and_moderation import
ChatGPTClient
# Bot's token for authentication
DISCORD_TOKEN = "
# Name of the channel the bot should monitor and interact with
CHANNEL_TO_WATCH = 'q-and-a'
# Initialize the Discord
client discord_client =
discord.Client()
# Create the system message for ChatGPT
system_message_to_chatgpt = "You are a virtual assistant
that provides support for the Crooks Bank banking app."
with open('FAQ.txt', 'r', encoding='utf-8') as file:
# Read the entire content of the file into a variable as a
single string
file_contents = file.read()
# Initialize the ChatGPT client chatgpt_client_for_qa_and_moderation =
ChatGPTClient(system_message_to_chatgpt, file_contents)
# Event handler for when the bot is ready
@discord_client.event async
def on_ready():
"""

Event handler triggered when the bot is successfully logged in and
ready to receive events.
"""

print('Logged in as',
discord_client.user)          print('-----')
    
```

Event handler for when a message is received

@discord_client.event

async def

on_message(message

):

"""

Event handler triggered when a message is received.

Parameters:

message (discord.Message): The message received by the bot.

Returns:

None

"""

Ignore messages sent by the bot to prevent self-responses

if message.author == discord_client.user:

return

Ignore messages not in the specified

"tech-support" channel if isinstance(message.channel,

discord.TextChannel) and \ message.channel.name !=

CHANNEL_TO_WATCH:

return

async with message.channel.typing():

Call ChatGPT to generate a response based on the received message response_from_chatgpt =

chatgpt_client_for_qa_and_moderation.send_

message_from_discord(message.content)

Construct a reply mentioning the message author and appending

ChatGPT's response reply =

f'{message.author.mention}

{response_from_chatgpt}'

Send the reply to the same channel where the original message was received

await message.channel.send(reply)

Run the bot with the provided token

discord_client.run(DISCORD_TOKEN)

Important Changes to Note from the Previous Version of the Tech Support Bot

Let's briefly analyze `tech_support_smart_bot.py` and discuss the changes that were made. The code snippet Listing 7-3 contains a portion of the full listing.

Listing 7-3. Setting the System Message and FAQ Info for `tech_support_bot.py`

```
# Create the system message for ChatGPT
system_message_to_chatgpt = "You are a Q and A bot for a
discord group dedicated to the banking app, Crook's Bank"
with open('FAQ.txt', 'r', encoding='utf-8') as file:
# Read the entire content of the file into a variable as
a single string
    file_contents = file.read()
```

As you can see, we're doing a few things at once.

First, as we have learned from the previous chapters in the book, you can dramatically set the tone of the conversation by providing a specific message to the system itself in your prompt. Therefore, we have a variable here containing the system message. 162

Next, we're defining another variable that provides a reference to the file path location where the frequently asked questions file is stored. Then we simply read the file – easy peasy.

Updates to the `on_message(message)` Function

When a message is received, be sure to notice the following lines:

```
@discord_client.event async
def
on_message(
message):
```

This function is the core part of our Discord bot. Whenever a message is posted in our Discord server, the function `on_message()` is called. The `async` keyword is important here because this enables `on_message()` to be called asynchronously, which is what we want, i.e., whenever a message is posted.

Note a quick reminder about synchronous vs. asynchronous communication. a typical example of synchronous communication is like accessing a web server using a web browser. once you send your request, you need to wait until you get a response. in order to view a different website, you need to send another request and wait again for the response.

a good example of asynchronous communication is emailing a friend or colleague. You can send 1, 2, 5, 10, or even 1000 emails to your friend, but they aren't necessarily waiting for your email. in other words, they aren't left in a "holding pattern" and stuck waiting for you to send something due to the way that email simply works. if and when an email arrives, your friend will get a notification.

Therefore, whenever a message is posted in our Discord server, the function `on_message()` is called with the message itself in the function parameters.

Now, let's further examine the following code from `tech_support_bot.py` :

```

    async with message.channel.typing():
        # Call ChatGPT to generate a response based on the received
        message response_from_chatgpt =
        chatgpt_client_for_qa_and_moderation.send_
        message_from_discord(message.content)
        # Construct a reply mentioning the message author and
        appending
        ChatGPT's response reply =
        f'{message.author.mention}
        {response_from_chatgpt}'

```

Here, we provide a nice user experience and show the user that the bot is “typing,” while the user’s question is being sent to ChatGPT. When the response comes back, we provide the reply back to the user.

Analyzing `chatgpt_client_for_qa_and_moderation.py`

Now, as we look further into Listing 7-2 above, `tech_support_bot.py` instantiates the

`ChatGPTClient` object in the `chatgpt_client_for_qa_and_moderation.py` script which (as we stated previously) is very similar to the `chatgpt_client.py` script that we have used before. The complete source for `chatgpt_client_for_qa_and_moderation.py` is shown in Listing 7-4.

Listing 7-4. `chatgpt_client_for_QA_and_moderation.py`

```

import os from dotenv import load_dotenv from openai
import OpenAI class ChatGPTClient: def __init__(self,
system_message, initial_instructions_to_chatgpt):
    # Load environment variables from .env
    load_dotenv()
    # Use the API key from the
environment variable self.client
= OpenAI() self.system_message =
system_message
self.initial_instructions_to_chatgpt = initial_instructions_
to_chatgpt
    def send_message_from_discord(self,
user_message): response =
self.client.chat.completions.create(
    model="gpt-4",
    messages=[
                                {
                                "role": "system",
                                "content":
f'{self.system_message}'

```



```

        },
        {
            "role": "user",
            "content":
f'{self.initial_instructions_to_chatgpt}'
        },
        {
            "role": "user",
            "content": f'{user_message}'
        }
    ],

    temperature=0.85,

    max_tokens=1921,
    top_p=1,

    frequency_penalty=0,
    presence_penalty=0

)
response = requests.get(url, params=params)

```

Breaking Down the ChatGPTClient Class in Python So Our Bot Can Utilize ChatGPT

The Python class we've created, ChatGPTClient, serves as a crucial role in integrating the OpenAI API into our Discord bot. Let's take a look at the benefits:

The class is designed with modularity in mind, encapsulating all the functionalities for interacting with ChatGPT. In the `__init__` method, we initialize the client passing in system message and contents of the FAQ.txt, as well as loading the OpenAI API key from the .env file.

Through the `send_message_from_discord()` method, the class makes communication with ChatGPT easier. Since the object is already constructed with the system message's initial instructions, the message from the Discord user is placed into the prompt as the user message, so that we can send the user's questions to ChatGPT programmatically.

The `send_message_from_discord()` method doesn't limit our use cases for this class. At the moment, we're using it for Q&A; however, any bot that uses this class would be able to prompt ChatGPT in any way for any purpose.

Finally, we're efficiently handling the response from the OpenAI API by extracting the content of the generated message and returning a condensed version. This allows our Discord bot to reply with just the information we're looking for, not the entire response that the Chat

model returns. This is accomplished in a single line of code:

```
condensed_response = response.choices[0].message.content
```

Running Our Intelligent Q&A Bot: tech_support_bot.py

Now, let's run our new and improved Q&A Python Discord bot, tech_support_bot.py. After executing the app, be sure to return back to your Discord server and start asking questions. Figure 7-1 shows the bot in action.

Figure 7-1. *Having a Discussion with Our Intelligent Discord Q&A Bot, tech_support_bot.py*

Listing 7-6 shows the conversation between us and the Discord bot.

Listing 7-5. Our First Conversation with the Intelligent Q&A Discord Bot
 Me: Hi, I'm new to this channel. What kind of questions can I ask?
 Q and A Bot: Hello! Welcome to the Crooks Bank Mobile App support channel. Here, you can ask any questions regarding our banking app. For example, you can ask about:

1. How to install and use the app.
2. Features and services within the app.
3. Troubleshooting any issues you might encounter.
4. Security measures to protect your data and transactions.
5. Instructions for resetting your password.
6. Costs and fees associated with using the app.
7. How to link external bank accounts.
8. Customer support availability.
9. Feedback and suggestions for improving the app.

Feel free to ask any other questions you might have related to the Crooks Bank Mobile App. We're here to assist!

Me: I have an old Blackberry phone, is the app supported on my device?

Q and A Bot: I'm sorry for the inconvenience, but at the moment, the Crooks Bank Mobile App is currently only available for iOS and Android devices. It is not supported on Blackberry or any other operating systems. You may still manage your bank accounts by accessing our website through your phone's browser.

Me: I'm an iPhone user. Can I sign into the app with face or touch ID?

Q and A Bot: Sure, here's how you can set up the Crooks Bank Mobile App on your iPhone:

1. Open the App Store on your iPhone.
2. In the search bar at the top, type in "Crooks Bank Mobile App" and tap "Search".
3. Once you can see the app in the search results, simply tap on the "Get" button to download it.
4. After the app downloads, tap "Open".
5. You'll have to agree to our Terms and Conditions upon the first launch.
6. If you're an existing Crooks Bank member, you can log in using your online banking user ID and password. If you're a new member, you'll need to register for an account.
7. Once you're logged in, you can set up Touch ID or Face ID for quicker, secure login in the future.

8. Now you can start using your Crooks Bank Mobile App to manage your finances, pay bills, make transactions, and more!

Please make sure you update the app regularly to access new features and improvements. If you have any challenges during setup, please reach out to us through the app's support feature or call our customer service.

We Have a Monumental Achievement...with One Slight Flaw

OK, if you step back and examine what we have achieved so far, you should realize that we're looking at what is nothing short of a monumental achievement. We have the following:

- A system made up of a few scripts that allows users to type in questions and get answers about how to use our mobile application.
- Using a simple text file, we can teach our bot how to answer the questions from our users. This text file can be edited by anyone in the company and can be used as a knowledge base to help improve the intelligence of the bot day by day. This is amazing stuff.

Our system allows customers to type their questions using natural language, and the bot provides an intelligent answer back to them. And do you know what? Customers don't like reading FAQs – especially really long ones. However, using this system, they don't need to! All they have to do is ask the question that is relevant to them.

So, with all this that we have accomplished, there's one *gigantic* flaw that we can't ignore. In Listing 7-5 above, the bot said to the user: Once you're logged in, you can set up Touch ID or Face ID for quicker, secure login in the future.

No, no, no! Bad bot! In case you didn't read the Frequently Asked Questions file completely, please allow me to explain what went wrong here:

1. The FAQ.txt file in Listing 7-1 clearly states that the Crook's Bank mobile app is a mobile app. Currently, Touch ID can only be used on Apple desktop and laptop computers. So, this doesn't even make sense.
2. The FAQ.txt file has no mention of Face ID for authentication to the mobile app at all.

So unfortunately, ChatGPT is making up things that are simply not true and is assuming that certain features exist that are not present. So, for the purposes of true tech support, this is absolutely unacceptable.

So, how do we solve this problem? Think about how *you* could solve this problem for a second before reading the next section.

Update the System Message to ChatGPT, and Let's Try Again

Have you come up with a solution yet? One way to resolve this issue is to update the system message. Refer to the following line in Listing 7-2 above:

system_message_to_chatgpt = "You are a virtual assistant that provides support for the Crooks Bank banking app. ";

Remember that the system message indicates to ChatGPT what its role is in the chat conversation. Right now, it is told to be a helpful assistant, and therefore, it has taken the liberty in order to guess that certain features exist within the mobile application. Now, let's ensure that ChatGPT does not provide any assumptions. Update the system message to be as shown below:

```
system_message_to_chatgpt = "You are a virtual assistant that provides support for the Crooks Bank banking app. You will be provided Information about the application, and users will ask you questions about the application. Your role is to answer questions explicitly from the training information. If you were not given an answer to a question, then provide an apologetic message to the user explaining that you do not know the answer.";
```

And that's it. TheFAQ.txt file remains exactly the same as before, and all we needed to do is to update the system message to let ChatGPT know its role in the conversation. Listing 7-6 has the new conversation between us and the Discord bot.

Listing 7-6. Our Second (and Better) Conversation with the Intelligent Q&A Discord Bot

Me: How are you today?

Q and A Bot: As a virtual assistant, I don't have feelings or emotions, but I'm here and ready to assist you. How may I help you with the Crooks Bank Mobile App today?

Me: I have an old Blackberry phone, is the app supported on my device?

Q and A Bot: I'm sorry, but from the information provided, the Crooks Bank

Mobile App can only be downloaded from the App Store for iOS devices and Google Play for Android devices. BlackBerry devices are not mentioned, therefore I cannot confirm that it would be compatible.

Me: So, if I use an iPhone, how do I setup the app?

Q and A Bot: To set up the Crooks Bank Mobile App on your iPhone, you can download it from the App Store. Simply search for "Crooks Bank Mobile App" and click the "Install" button. After the installation is finished, open the app and follow the prompts to set up your account. If you already have an account, use your login information to access your account through the app.

Now, that's a lot better! Our tech support bot is "sticking to the script" and is not allowed to make up or assume anything.

Conclusion

In this chapter, we've accomplished a lot! We have a fully functioning Discord bot that can be trained by anyone without any experience in AI, NLP, or machine learning using a simple text file. We learned that a "helpful assistant" may at times become *too helpful* and assume facts that are not true. However, we also reiterated the concept and the value of the system message, which is an important part of prompt engineering.

Now that we have made our Q&A Discord bot to be intelligent, let's find out how to make our content moderator bot to be intelligent as well!

ADDING INTELLIGENCE TO OUR DISCORD BOTS, PART 2: USING CHAT AND MODERATIONS MODELS FOR CONTENT MODERATION

In this chapter, we're going to take the steps necessary in order to make our content moderator discord bot artificially intelligent. Let's overview the changes that we're going to make:

- Create a new script for ourselves, `moderation_client.py`, to invoke the `OpenAI.moderations.create()` method of the OpenAI API. The Moderations model allows us to be aware when any textual content fits any of the following categories:
 - Hate
 - Hate/threatening
 - Harassment
 - Harassment/threatening
 - Self-harm
 - Self-harm/intent
 - Self-harm/instructions

© Lydia Evelyn, Bruce Hopkins 2024

L. Evelyn and B. Hopkins, *Beginning ChatGPT for Python*,
https://doi.org/10.1007/979-8-8688-0929-3_8

- Sexual
- Sexual/minors
- Violence
- Violence/graphic
- Reuse our `chatgpt_client_for_qa_and_moderation.py` script from the previous chapter. In Chapter 7, it was used to invoke the OpenAI Chat class for Q&A purposes from our users. In this chapter, it will be used to invoke the Chat class again, but this time for moderation purposes. This is why the script is aptly named “`chatgpt_client_for_qa_and_moderation`” because it's used for Q&A in Chapter 7 but also for moderation in this chapter.
- Modify our `content_moderator_bot.py` script (formerly named, `content_moderator_bot_dumb.py`) so that it can invoke both the `moderation_client.py` and `chatgpt_client_for_qa_and_moderation.py`. If either script indicates that the content typed in the Discord channel is objectionable, then it will delete the message from that Discord channel. Remember, this bot watches all content in all channels of the Discord server!

Now, at this point, you may be asking yourself, if the Moderations class already knows how to flag any harmful content, then why do we need to use the Chat class as well? Good question.

Yes, the Moderations class will allow us to know about harmful content, but it **does not** inform us about any other types of unwanted content for our use case, such as when unscrupulous individuals try to lure our users into a scam. Remember, this is a Discord server for a banking app, so scammers would definitely love to target all the members of this Discord server since it's a central location full of bank users!

Therefore, we'll use the `moderation_client.py` to invoke the Moderations class to know if any content in the Discord server is harmful, and we'll reuse the `chatgpt_client_for_qa_and_moderation.py` from the last chapter in order to invoke the Chat method in order to be made aware of any other undesirable content is posted in the Discord server, such as scam attempts.

Using `OpenAI.moderations.create()` to Moderate Content

By using the Moderations models, developers are able to submit a string of text and to subsequently know if it's violent, hateful, and/or threatening or contains any form of harassment.

Table 8-1 describes the format of the parameters necessary to call the `OpenAI.moderations.create()` method.

The service is very simple to use, since only one parameter is required to properly invoke the service.

Examining the Method Parameters

Table 8-1. *The Request Body for the Moderations Model*

inputString or listrequired the text that needs to be classified. there are model String Optional

multiple content moderation models available for use, for example:

Default: "omni-moderation-latest"

- "omni-moderation-latest"
- "text-moderation-stable"
- "text-moderation-latest"

By default, this is set to "omni-moderationlatest". It will be automatically upgraded over time, which ensures you're always using the most accurate model.

If you decide to use any of the text based moderation models, then you are only able to submit text to be evaluated.

the omni moderation models, however, are able to evaluate content as text and images. therefore choose the model that works best for your use case.

Handling the Response

After successfully invoking the Moderations model, the method will provide a response with the structure shown in Table 8-2.

Moderation (Dictionary)

Table 8-2. *The Structure of the Moderation Response*

| Field | Type | Description |
|---|---|---|
| id
model
results
↳ flagged
↳ categories | String
String
List
Boolean
List | a unique identifier for the moderation request.
the model used to perform the moderation request.
a list of moderation objects.
Flags if the content violates OpenAI's usage policies.
a list of the categories and whether they're being flagged or not. |
| ↳ hate
↳ hate/threatening
↳ harassment
↳ harassment/
threatening
↳ self-harm | Boolean
Boolean
Boolean
Boolean
Boolean
Boolean
Boolean | this indicates whether or not the text given expresses, incites, or promotes hate based on race, gender, religion, ethnicity, nationality, disability status, sexual orientation, or caste.
this indicates whether or not the text given contains hateful content that also threatens violence or serious harm toward the targeted group based on biases expressed above.
this indicates whether or not the text given contains content that expresses, incites, or promotes harassing language toward any target.
this indicates whether or not the text given contains harassment content that also threatens violence or serious harm toward any target.
this indicates whether or not the text given contains content that promotes, encourages, or depicts acts of self-harm, for example, suicide, cutting, and eating disorders. |

(continued)

Table 8-2. *(continued)*

Type Boolean ~~this indicates whether or not the text given contains content~~
self-harm/intent
The listing below is an example of the Moderation response after invoking the ^{Number} Moderations model. Table [r 8-2](#) looks a little complex, but as you can see, if any of the categories are labeled as “true,” then the **Number results.flagged** node is labeled as “true.” Take a look at Listing [r 8-1](#) for a practical example of the Moderation response. **Listing 8-1.** The Moderation Response


```
{
  "id": "modr-XXXXX",
  "model": "text-moderation-005",
  "results": [
    {
      "flagged": true,
      "categories": {
        "sexual": false, "hate": false, "harassment": false,
        "self-harm": false, "sexual/minors": false,
        "hate/threatening": false,
        "violence/graphic": false, "self harm/intent": false,
        "self-harm/instructions": false,

        "harassment/threatening": true,
        "violence": true,
      },
      "category_scores": {
        "sexual": 1.2282071e-06, "hate":
0.010696256, "harassment": 0.29842457,
        "self-harm": 1.5236925e-08, "sexual/minors":
5.7246268e-08, "hate/threatening": 0.0060676364,
        "violence/graphic": 4.435014e-06, "self harm/intent":
8.098441e-10, "self harm/instructions": 2.8498655e-11,
        "harassment/threatening": 0.63055265,
        "violence": 0.99011886,
      }
    }
  ]
}
```

Creating Our Client for the Moderations Model:

moderation_client.py

Listing 8-2 is our client to invoke the Moderations model. Take a look at it, and then we'll discuss the important parts afterward.

Listing 8-2. moderation_client.py

```
import os from dotenv import load_dotenv from openai import OpenAI
class ModerationResponse:
    def __init__(self):
        load_dotenv()
        self.client = OpenAI()
    def moderate_text(self, text):
        moderation = self.client.moderations.create(input=text)
    return moderation
```

In the previous chapters in this book, we created client scripts for the various methods of the classes of the OpenAI API. Therefore, the class above, `ModerationResponse` in `moderation_client.py`, should look quite familiar. At the end of the day, we have a simple function, `moderate_text()`, which allows us to pass the text that we want to evaluate and returns the response.

Making content_moderator_bot.py

More Intelligent

Now that we have `moderation_client.py` to invoke the `Moderations` model, let's take a look at the updated `content_moderator_bot.py` (formerly named, `content_moderator_bot_dumb.py`) that will use the `moderation_client.py` to check for harmful content and the `chatgpt_client_for_qa_and_moderation.py` (unmodified from the previous chapter) to check for potential scams.

Listing 8-3 is the full source code for our intelligent Discord moderator bot, `content_moderator_bot.py`.

Listing 8-3. `content_moderator_bot.py`

```
"""
```

```
content_moderation_bot.py
```

A Discord bot integrated with ChatGPT and a moderation service for automated content moderation in Discord servers.

This script initializes a Discord bot using the `discord.Client()` class and listens for events such as messages being sent. When a message is received, it calls both the ChatGPT API and a moderation service to analyze the message content for rule violations. If the message is flagged by either service, it deletes the message and sends a notification to the user explaining why it was deemed inappropriate.

Requirements:

- discord (<https://pypi.org/project/discord.py/>)
- chatgpt_client_for_qa_and_moderation (Assumed to be a custom module providing interaction with ChatGPT)
- moderation_client (Assumed to be a custom module providing interaction with a moderation service)

Usage:

1. Replace the `DISCORD_TOKEN` variable with your bot's token obtained from the Discord Developer Portal.

2. Ensure that the `chatgpt_client_for_qa_and_moderation` and `moderation_client` modules are properly implemented and accessible.

Note: This script assumes the presence of modules for interaction with ChatGPT and a moderation service.

```
"""
```

```
import discord from chatgpt_client_for_qa_and_moderation import
ChatGPTClient from moderation_client import ModerationResponse
# Bot's token for authentication
DISCORD_TOKEN = "
```

```
# Initialize the Discord client discord_client = discord.Client() # Create
the system message for ChatGPT system_message_to_chatgpt = ""
    You are the automated moderator assistant for a Discord
server.
```

Review each message for the following rule violations:

1. Sensitive information
2. Abuse
3. Inappropriate comments
4. Spam, for example; a message in all capital letters, the same phrase or word being repeated over and over, more than 3 exclamation marks or question marks.
5. Advertisement
6. External links 7. Political messages or debate 8. Religious messages or debate

If any of these violations are detected, respond with "FLAG" (in uppercase without quotation marks). If the message adheres to the rules, respond with "SAFE" (in uppercase without quotation marks). ""

```
initial_instructions_to_chatgpt = "Analyze the following message for
rule violations:"
```

```
# Initialize the ChatGPT client chatgpt_client_for_qa_and_moderation =
ChatGPTClient(system_message_to_ chatgpt,
initial_instructions_to_chatgpt)
```

```
# Initialize the Moderation client moderation_client =
ModerationResponse()
```

```
# Event handler for when the bot is ready
@discord_client.event async def on_ready():
    ""
```

Event handler triggered when the bot is successfully logged in and ready to receive events.

```
    ""
    print('Logged in as',
discord_client.user)
print('-----')
```

```
# Event handler for when a message is received
@discord_client.
event async def
on_message(mes
sage):
    ""
```

Event handler triggered when a message is received.

Parameters:

message (discord.Message): The message received by the bot.

Returns:

```

        None
    """
    # Ignore messages sent by the bot to prevent
    self-responses if message.author ==
    discord_client.user:
        return
    # Call the Moderation method to check for harmful content
    moderation_response =
    moderation_client.moderate_text(message.content)
    # Call ChatGPT to generate a response based on the received
    message response_from_chatgpt =
    chatgpt_client_for_qa_and_moderation.send_
    message_from_discord(message.content)
    # Check for if the message from ChatGPT is "FLAG" or if the
    moderation

    response indicates that the input has been flagged if
    response_from_chatgpt == "FLAG" or
    moderation_response.results[0].

    flagged:
    # Delete the message

        await message.delete()
    # Mention the user who sent the inappropriate message
    author_mention = message.author.mention
    # Send a message mentioning the user and explaining why it was
    inappropriate await message.channel.send(f"{author_mention} This
    comment was

        deemed inappropriate for this channel. " +
    " If you believe this to be in error, please contact one of the

    human server moderators.")
    # Run the bot with the provided token
    discord_client.run(DISCORD_TOKEN)
    Updates to the on_message(message) Function
    After a message is received in any channel of the Discord server, the
    on_message(message) function is invoked asynchronously. Here's the
    most important change to be aware of:
    # Call the Moderation model to check for harmful content
    moderation_response =
    moderation_client.moderate_text(message.content)
    # Call ChatGPT to generate a response based on the received
    message response_from_chatgpt =
    chatgpt_client_for_qa_and_moderation.send_
    message_from_discord(message.content)

```

```
# Check for if the message from ChatGPT is "FLAG" or if the
moderation response indicates that the input has been flagged
    if response_from_chatgpt == "FLAG" or
moderation_response.results[0].
flagged:
# Delete the message
```

```
    await message.delete()
    # Mention the user who sent the inappropriate message
    author_mention = message.author.mention
# Send a message mentioning the user and explaining why it
was inappropriate
    await
message.channel.send(f'{author_mention} This comment was
    deemed inappropriate for this channel. " +
    "If you believe this to be in error, please contact one of the
    human server moderators.")
```

Here, we take each message that was posted in the Discord server and check it with both the Moderations class and the Chat method. If either method return to inform us that the message is flagged, then we delete the message in the channel and inform the user that their message violated the rules.

Now that our content moderator Discord bot is intelligent, let's give it a try!

Running Our Intelligent Content Moderator Bot:

content_moderator_bot.py

Now let's run our new and improved content moderator Python Discord bot, content_moderator_bot.py. After executing the app, be sure to return back to your Discord server and start asking questions. Figure 8-1 shows the bot in action.

Figure 8-1. *Having a Discussion with Our Intelligent Discord Content Moderator*

Bot: content_moderator_bot.py

Listing 8-4 shows a conversation between us and the Discord bot in order to test and see what it can do.

Listing 8-4. Our Offensive Conversation with the Intelligent Moderator Discord Bot

Me: Hi everyone, I love the Crooks Bank app!

Me: This app is awesome!

Me: Come to my website! <http://www.google.com>

Content Mod Bot: @PythonChatGPT This comment was deemed inappropriate for this channel. If you believe this to be in error, please contact one of the human server moderators.

Me: I'm sorry for breaking the rules. I'm a different person now

Me: But I have some sad news for you

Me: I want to ☠ everyone

Content Mod Bot: @PythonChatGPT This comment was deemed inappropriate for this channel. If you believe this to be in error, please contact one of the human server moderators.

In both cases when unwanted content was posted in any channel of the Discord server, not only was the offending user called out but the bad message was deleted.

Good bot!

Did you notice that the Moderation and Chat methods are able to read emojis as well?

Conclusion

In this chapter, we created a fully functioning content moderator for our entire Discord server! We leveraged both the Moderations and Chat methods from OpenAI to create a custom content moderator that not only flags unsafe content like hateful and threatening messages but also prevents the users of the Discord server from being subject to unwanted solicitations.

Exercises Left for the Reader

Although we accomplished a lot in this chapter (as well as in this book!), there's still one more thing that we can do to improve the code. For example:

- The individual Discord bots that we created are aware to not respond to messages that they send themselves. However, the bots are not yet aware that they shouldn't respond to messages sent by **other bots**. In other words, if you run both bots at the same time, and someone posts something bad in the "q-and-a" channel, the content moderator will, of course, delete the message and inform everyone that the message was deleted. However, since the tech support bot doesn't know that it shouldn't respond to other bots, it will try to create a response. Of course, bots should not talk to other bots.

DEVELOPING A PYTHON APPLICATION USING THE OPENSKILLS API TO DISPLAY REQUIRED SKILLS FOR A SPECIFIED JOB ROLE

In today's rapidly evolving job market, aligning personal skills with job expectations is crucial for both job seekers and employers. OpenSkills API offers a powerful way to fetch and display the skills required for specific job roles. This chapter will guide you through developing a Python application that utilizes the **OpenSkills API** to fetch and present the necessary skills for a specified job role.

By the end of this chapter, you'll have a working Python application that integrates with OpenSkills API and provides job seekers with valuable insights into the skills required for different job roles.

PREREQUISITES

Before we start, make sure you have the following:

- A working installation of Python (version 3.6 or higher).
- An **OpenSkills API key** (you can obtain this by signing up at the OpenSkills platform).
- Python's requests library installed for making HTTP requests.
Install it by running:
bash
Copy code
pip install requests
- Basic understanding of Python and working with APIs.

1. UNDERSTANDING THE OPENSKILLS API

The OpenSkills API provides endpoints that allow developers to access information about job roles and the skills associated with them. It is designed to assist in career development by categorizing skills based on job titles.

KEY ENDPOINT

The primary endpoint we will be working with is `/skills`, which requires a **job title** as a parameter. It returns a structured list of required skills, including:

- Technical skills
- Soft skills
- Certifications or qualifications

EXAMPLE REQUEST

An example of a request to fetch skills for the "Software Developer" job role using the OpenSkills API might look like this:

```
python  
Copy code
```

```
response =
requests.get("https://api.openskills.com/skills?job_title=Software+Devel
oper", headers={"Authorization": "Bearer YOUR_API_KEY"})
```

EXAMPLE RESPONSE

A successful response from the API might look like this:

json

Copy code

```
{
  "technical_skills": ["Java", "Spring Boot", "REST APIs", "SQL",
"Git"],
  "soft_skills": ["Communication", "Problem-solving", "Teamwork"],
  "certifications": ["Certified Java Programmer", "AWS Certified
Solutions Architect"]
}
```

This JSON response includes lists of technical skills, soft skills, and certifications needed for the job role.

2. SETTING UP THE APPLICATION

Now, let's begin coding our Python application that will fetch and display job skills based on user input.

STEP 1: IMPORT NECESSARY LIBRARIES

We'll start by importing the required libraries for making HTTP requests and parsing the JSON response.

python

Copy code

import requests

import json

STEP 2: CREATE FUNCTION TO FETCH SKILLS

Next, we'll create a function that takes a job title as input, sends an HTTP request to the OpenSkills API, and returns the list of required skills.

python

Copy code

```
def get_skills_for_job(job_title):
    api_url = "https://api.openskills.com/skills"
    api_key = "YOUR_API_KEY" # Replace this with your actual API
key
    headers = {"Authorization": f"Bearer {api_key}"}

    params = {
        "job_title": job_title
    }
```

try:

```
response = requests.get(api_url, headers=headers, params=params)
response.raise_for_status() # Check if request was successful
skills_data = response.json() # Convert response to JSON format
```



```

    return skills_data
except requests.exceptions.RequestException as e:
    print(f'Error fetching skills data: {e}')
    return None

```

EXPLANATION

- `api_url`: This is the OpenSkills API endpoint for retrieving skills data.
- `api_key`: The API key you received after registering for OpenSkills.
- `headers`: This contains the Authorization information to authenticate your request.
- `params`: We pass the `job_title` as a query parameter.
- `response.json()`: Converts the JSON response from the API into a Python dictionary, which we can then use.

STEP 3: DISPLAY SKILLS IN A READABLE FORMAT

Now that we have the skills data, we need a function to present it in a user-friendly format. This function will check for different categories of skills and display them appropriately.

python

Copy code

```

def display_skills(skills_data):
    if not skills_data:
        print("No skills data found.")
        return

    print("Skills required for the job role:")

    # Technical Skills
    if 'technical_skills' in skills_data:
        print("\nTechnical Skills:")
        for skill in skills_data['technical_skills']:
            print(f"- {skill}")

    # Soft Skills
    if 'soft_skills' in skills_data:
        print("\nSoft Skills:")
        for skill in skills_data['soft_skills']:
            print(f"- {skill}")

    # Certifications
    if 'certifications' in skills_data:
        print("\nCertifications:")
        for cert in skills_data['certifications']:
            print(f"- {cert}")

```

EXPLANATION

- This function first checks if any skills data is present.
- It then checks for three categories of skills: **Technical**, **Soft**, and **Certifications**. If any category is present, it prints the corresponding skills.

STEP 4: PUTTING IT ALL TOGETHER

Now, let's create the main part of the program where the user inputs the job title, and the program fetches and displays the relevant skills.

python

Copy code

```
def main():
```

```
    job_title = input("Enter the job title: ")
    skills_data = get_skills_for_job(job_title)
    display_skills(skills_data)
```

```
if __name__ == "__main__":
    main()
```

EXPLANATION

- `input("Enter the job title: ")`: The user is prompted to enter the job role.
- `get_skills_for_job(job_title)`: The function fetches the skills for the entered job title.
- `display_skills(skills_data)`: This function displays the skills in an easy-to-read format.

3. RUNNING THE APPLICATION

To run the application, simply execute the Python script. The program will prompt the user for a job title and display the skills associated with that role.

Example output for the job title "Software Developer":

diff

Copy code

Enter the job title: Software Developer

Skills required for the job role:

Technical Skills:

- Java
- Spring Boot
- REST APIs
- SQL
- Git

Soft Skills:

- Communication
- Problem-solving
- Teamwork

Certifications:

- Certified Java Programmer
- AWS Certified Solutions Architect

4. ERROR HANDLING AND EDGE CASES

We need to ensure the application is robust. If the API request fails or the job title entered by the user is not recognized, we should handle such scenarios gracefully.

HANDLING EMPTY OR INVALID DATA

If the API returns no data or if there's an issue with the request, we can display an appropriate error message.

python

Copy code

```
def display_skills(skills_data):
    if not skills_data:
        print("No skills data found for the entered job title.")
        return
    # (Rest of the function remains unchanged)
```

HANDLING INVALID JOB TITLES

If a job title is not found, it's good practice to notify the user that the entered role could not be processed. The error handling in `get_skills_for_job()` will catch these cases.

5. CONCLUSION

In this chapter, we walked through the steps of developing a Python application that uses the **OpenSkills API** to fetch and display the skills required for a specific job role. By using the OpenSkills API, we provided job seekers with insights into the technical, soft skills, and certifications needed for success in different careers.

The code we developed can be expanded with additional features, such as searching for multiple job titles at once or saving the skills data to a file. This application serves as a great starting point for anyone looking to integrate skill-based job information into a Python project.

APPENDIX 1

LIST OF OPENAI MODELS

After executing the code in Listing 2-4, `model_lister.py`, you will be presented with a response object that has a list of the OpenAI models available to you. The table below shows a snapshot of the response.

| ID | Object | Created | Owned By |
|--------------|--------|-----------|----------------|
| gpt-4-turbo | model | 171236144 | system |
| gpt-4-turbo- | model | 1 | system |
| 2024-04-09 | model | 171260167 | openaiinternal |
| tts-1 | model | 7 | system |
| tts-1-1106 | model | 168194095 | system |
| chatgpt-4o-l | model | 1 | system |
| atest | model | 169905324 | openaiinternal |
| dall-e-2 | model | 1 | system openai |
| whisper-1 | model | 172351513 | system |
| gpt-3.5-turb | model | 1 | system |
| o-instruct | model | 169879817 | system |
| gpt-3.5-turb | model | 7 | system |
| o | model | 167753238 | system |
| gpt-3.5-turb | model | 4 | system |
| o-0125 | model | 169290142 | system |
| babba | model | 7 | system |
| ge-002 | model | 167761060 | |
| davinci-002 | | 2 | |
| gpt-4o-mini- | | | |
| 2024-07-18 | | | |
| gpt-4o | | | |
| dall-e-3 | | | |
| gpt-4o-mini | | | |
| gpt-4o-2024- | | | |
| 08-06 | | | |

APPENDIX II

LIST OF OPEN SOURCE AI TOOLS

| AI Tool | Description | Primary Use Case | Programming Language(s) | GitHub Link |
|--------------|--|--|-------------------------|------------------------------|
| TensorFlow | A comprehensive open-source machine learning framework for building and training models. | Deep learning, neural networks | Python, C++ | TensorFlow |
| PyTorch | A deep learning framework offering flexibility and speed for research and production deployment. | Deep learning, computer vision, NLP | Python, C++ | PyTorch |
| scikit-learn | A simple and efficient tool for data mining and data analysis built on top of Python. | Machine learning, statistical modeling, classification | Python | scikit-learn |

| | | | | |
|---------------------------|---|---|-------------|------------------------------|
| Keras | An open-source neural network library written in Python that runs on top of TensorFlow. | Deep learning, prototyping neural networks | Python | Keras |
| OpenCV | A computer vision library that provides real-time image processing and vision solutions. | Image and video processing, object detection | C++, Python | OpenCV |
| Hugging Face Transformers | A popular library for NLP tasks, including text classification, translation, and text generation. | Natural language processing (NLP), transformers | Python | Transformers |
| FastAI | A library built on top of PyTorch that simplifies the process of training deep learning models. | Deep learning, computer vision, NLP | Python | FastAI |
| Apache Mahout | A machine learning library that provides scalable algorithms | Machine learning, recommendation systems | Java, Scala | Mahout |

| | | | | |
|----------------------|--|--------------------------------------|---------------------|---------------------------|
| | for clustering, classification , and recommendation. | | | |
| Gensim | A library for unsupervised topic modeling and natural language processing, particularly known for word embeddings. | NLP, topic modeling, word embeddings | Python | Gensim |
| spaCy | A library designed for fast, production-ready NLP tasks. | Natural language processing (NLP) | Python | spaCy |
| Tesseract | An OCR engine for recognizing text in images and documents. | Optical character recognition (OCR) | C++, Python | Tesseract |
| Chainer | A flexible and intuitive deep learning framework. | Deep learning, neural networks | Python | Chainer |
| MLlib (Apache Spark) | A scalable machine learning library built into the Apache | Machine learning, big data analysis | Java, Scala, Python | MLlib |

| | | | | |
|----------------------------|--|-----------------------------------|------------------|-------------------------|
| | Spark ecosystem. | | | |
| DVC (Data Version Control) | A version control system for machine learning projects, managing datasets and models efficiently. | Data management , version control | Python | DVC |
| Jupyter Notebook | An open-source web application for creating and sharing documents containing live code, equations, visualizations. | Data science, interactive coding | Python, R, Julia | Jupyter |
| Rasa | An open-source conversational AI platform that helps build contextual chatbots. | NLP, chatbot development | Python | Rasa |

APPENDIX III

LIST OF POPULAR NO CODE DEVELOPMENT TOOLS

| No-Code Tool | Description | Primary Use Case | Supported Platforms | Website Link |
|-------------------|--|--|---------------------|----------------------------|
| Bubble | A no-code platform for building fully customizable web applications with complex workflows. | Web app development, marketplaces, social networks | Web | Bubble |
| Adalo | A no-code platform for building native mobile apps with drag-and-drop interface and backend integration. | Mobile app development | Web, iOS, Android | Adalo |
| Webflow | A visual web design tool that enables users to build responsive websites without writing code. | Website design, web development | Web | Webflow |
| OutSystems | A no-code/low-code platform for building enterprise-grade applications with drag-and-drop features. | Enterprise app development, mobile apps, web apps | Web, iOS, Android | OutSystems |
| Glide | A platform for building mobile apps from Google Sheets without coding. | Mobile app development, productivity tools | Web, iOS, Android | Glide |

| | | | | |
|--------------------------|--|---|-------------------|----------------------------|
| Zapier | An automation tool that connects web apps to automate workflows and tasks without coding. | Workflow automation, integrations | Web | Zapier |
| Airtable | A no-code database management tool that combines the features of a spreadsheet and a database. | Project management, CRM, task management, data tracking | Web, iOS, Android | Airtable |
| Integromat (Make) | A powerful tool for automating workflows by integrating apps and services with no coding required. | Workflow automation, app integration | Web | Integromat |
| Appgyver | A no-code platform for building mobile and web apps with extensive visual components and logic. | Mobile and web app development | Web, iOS, Android | Appgyver |
| Thunkable | A no-code app builder focused on creating native mobile apps with a drag-and-drop interface. | Mobile app development | Web, iOS, Android | Thunkable |
| Wix | A website builder that allows users to create stunning websites without code using pre-designed templates. | Website development, eCommerce | Web | Wix |
| Retool | A platform for building internal tools by connecting various data sources with a simple drag-and-drop interface. | Internal tools, dashboards, admin panels | Web | Retool |

| | | | | |
|------------------|---|---|-------------------|---------------------------|
| Kodular | A no-code platform for creating Android apps with a drag-and-drop interface. | Android app development | Android | Kodular |
| Coda | A flexible document builder that combines documents, spreadsheets, and apps into one platform. | Document management, team collaboration | Web, iOS, Android | Coda |
| Voiceflow | A no-code platform for building conversational AI interfaces like voice apps and chatbots. | Voice app development, chatbot creation | Web | Voiceflow |
| Tilda | A web design platform with pre-designed blocks and a focus on creating beautiful, content-focused websites. | Website design, content creation | Web | Tilda |

APPENDIX IV**LIST OF OPEN SOURCE LLMs**

| Model Name | Developed By | Parameters | Use Case | Repository |
|------------|-----------------|---------------------|--|-----------------------------------|
| GPT-Neo | EleutherAI | Various (up to 20B) | Text generation, dialogue systems, NLP tasks | GPT-Neo on GitHub |
| GPT-J | EleutherAI | 6B | Language generation, general-purpose NLP tasks | GPT-J on GitHub |
| LLaMA | Meta | 7B to 65B | General-purpose NLP, text generation, summarization | LLaMA on GitHub |
| BLOOM | BigScience | 176B | Multilingual NLP tasks, text generation, translation | BLOOM on GitHub |
| T5 | Google Research | Various (up to 11B) | Text-to-text tasks, summarization, translation, QA | T5 on GitHub |

| | | | | |
|-----------|----------|-----------------------|--|-------------------------------------|
| OPT | Meta | 125M to 175B | Text generation, NLP tasks similar to GPT-3 | OPT on GitHub |
| Alpaca | Stanford | 7B (fine-tuned LLaMA) | Instruction-following tasks, conversational agents | Alpaca on GitHub |
| Mistral | Mistral | 7B, others | Efficient NLP tasks, text generation | Mistral on GitHub |
| OpenLLaMA | OpenLM | 7B, others | Text generation, NLP research | OpenLLaMA on GitHub |
| RWKV | BlinkDL | Various (up to 7B) | Text generation, summarization, language modeling | RWKV on GitHub |
| Gopher | DeepMind | 280B | General-purpose NLP, text generation | Gopher on GitHub |

| | | | | |
|------|-----------------|----------------------|---|--------------------------------|
| BERT | Google Research | 110M to 340M | Text classification, information retrieval, QA | BERT on GitHub |
| CLIP | OpenAI | Various (up to 400M) | Multimodal tasks (image and text), zero-shot classification | CLIP on GitHub |
| MELD | Declare Lab | Various | Emotion recognition, sentiment analysis, dialogue NLP | MELD on GitHub |